

Marek Cygan, Fedor V. Fomin,
Łukasz Kowalik, Daniel Lokshtanov,
Dániel Marx, Marcin Pilipczuk,
Michał Pilipczuk and Saket Saurabh

Parameterized Algorithms

May 30, 2016

Springer

Preface

The goal of this textbook is twofold. First, the book serves as an introduction to the field of parameterized algorithms and complexity accessible to graduate students and advanced undergraduate students. Second, it contains a clean and coherent account of some of the most recent tools and techniques in the area.

Parameterized algorithmics analyzes running time in finer detail than classical complexity theory: instead of expressing the running time as a function of the input size only, dependence on one or more parameters of the input instance is taken into account. While there were examples of nontrivial parameterized algorithms in the literature, such as Lenstra’s algorithm for integer linear programming [319] or the disjoint paths algorithm of Robertson and Seymour [402], it was only in the late 1980s that Downey and Fellows [149], building on joint work with Langston [180, 182, 183], proposed the systematic exploration of parameterized algorithms. Downey and Fellows laid the foundations of a fruitful and deep theory, suitable for reasoning about the complexity of parameterized algorithms. Their early work demonstrated that fixed-parameter tractability is a ubiquitous phenomenon, naturally arising in various contexts and applications. The parameterized view on algorithms has led to a theory that is both mathematically beautiful and practically applicable. During the 30 years of its existence, the area has transformed into a mainstream topic of theoretical computer science. A great number of new results have been achieved, a wide array of techniques have been created, and several open problems have been solved. At the time of writing, Google Scholar gives more than 4000 papers containing the term “fixed-parameter tractable”. While a full overview of the field in a single volume is no longer possible, our goal is to present a selection of topics at the core of the field, providing a key for understanding the developments in the area.

Why This Book?

The idea of writing this book arose after we decided to organize a summer school on parameterized algorithms and complexity in Będlewo in August 2014. While planning the school, we realized that there is no textbook that contains the material that we wanted to cover. The classical book of Downey and Fellows [153] summarizes the state of the field as of 1999. This book was the starting point of a new wave of research in the area, which is obviously not covered by this classical text. The area has been developing at such a fast rate that even the two books that appeared in 2006, by Flum and Grohe [189] and Niedermeier [376], do not contain some of the new tools and techniques that we feel need to be taught in a modern introductory course. Examples include the lower bound techniques developed for kernelization in 2008, methods introduced for faster dynamic programming on tree decompositions (starting with *Cut & Count* in 2011), and the use of algebraic tools for problems such as LONGEST PATH. The book of Flum and Grohe [189] focuses to a large extent on complexity aspects of parameterized algorithmics from the viewpoint of logic, while the material we wanted to cover in the school is primarily algorithmic, viewing complexity as a tool for proving that certain kinds of algorithms do not exist. The book of Niedermeier [376] gives a gentle introduction to the field and some of the basic algorithmic techniques. In 2013, Downey and Fellows [154] published the second edition of their classical text, capturing the development of the field from its nascent stages to the most recent results. However, the book does not treat in detail many of the algorithmic results we wanted to teach, such as how one can apply important separators for EDGE MULTIWAY CUT and DIRECTED FEEDBACK VERTEX SET, linear programming for ALMOST 2-SAT, *Cut & Count* and its deterministic counterparts to obtain faster algorithms on tree decompositions, algorithms based on representative families of matroids, kernels for FEEDBACK VERTEX SET, and some of the reductions related to the use of the Strong Exponential Time Hypothesis.

Our initial idea was to prepare a collection of lecture notes for the school, but we realized soon that a coherent textbook covering all basic topics in equal depth would better serve our purposes, as well as the purposes of those colleagues who would teach a semester course in the future. We have organized the material into chapters according to techniques. Each chapter discusses a certain algorithmic paradigm or lower bound methodology. This means that the same algorithmic problem may be revisited in more than one chapter, demonstrating how different techniques can be applied to it. Thanks to the rapid growth of the field, it is now nearly impossible to cover every relevant result in a single textbook. Therefore, we had to carefully select what to present at the school and include in the book. Our goal was to include a self-contained and teachable exposition of what we believe are the basic techniques of the field, at the expense of giving a complete survey of the area. A consequence of this is that we do not always present the strongest result for

a particular problem. Nevertheless, we would like to point out that for many problems the book actually contains the state of the art and brings the reader to the frontiers of research. We made an effort to present full proofs for most of the results, where this was feasible within the textbook format. We used the opportunity of writing this textbook to revisit some of the results in the literature and, using the benefit of hindsight, to present them in a modern and didactic way.

At the end of each chapter we provide sections with exercises, hints to exercises and bibliographical notes. Many of the exercises complement the main narrative and cover important results which have to be omitted due to space constraints. We use (🍃) and (💀) to identify easy and challenging exercises. Following the common practice for textbooks, we try to minimize the occurrence of bibliographical and historical references in the main text by moving them to bibliographic notes. These notes can also guide the reader on to further reading.

Organization of the Book

The book is organized into three parts. The first seven chapters give the basic toolbox of parameterized algorithms, which, in our opinion, every course on the subject should cover. The second part, consisting of Chapters 8-12, covers more advanced algorithmic techniques that are featured prominently in current research, such as important separators and algebraic methods. The third part introduces the reader to the theory of lower bounds: the intractability theory of parameterized complexity, lower bounds based on the Exponential Time Hypothesis, and lower bounds on kernels. We adopt a very pragmatic viewpoint in these chapters: our goal is to help the algorithm designer by providing evidence that certain algorithms are unlikely to exist, without entering into complexity theory in deeper detail. Every chapter is accompanied by exercises, with hints for most of them. Bibliographic notes point to the original publications, as well as to related work.

- Chapter 1 motivates parameterized algorithms and the notion of fixed-parameter tractability with some simple examples. Formal definitions of the main concepts are introduced.
- Kernelization is the first algorithmic paradigm for fixed-parameter tractability that we discuss. Chapter 2 gives an introduction to this technique.
- Branching and bounded-depth search trees are the topic of Chapter 3. We discuss both basic examples and more advanced applications based on linear programming relaxations, showing the fixed-parameter tractability of, e.g., ODD CYCLE TRANSVERSAL and ALMOST 2-SAT.
- Iterative compression is a very useful technique for deletion problems. Chapter 4 introduces the technique through three examples, including FEEDBACK VERTEX SET and ODD CYCLE TRANSVERSAL.

- Chapter 5 discusses techniques for parameterized algorithms that use randomization. The classic color coding technique for LONGEST PATH will serve as an illustrative example.
- Chapter 6 presents a collection of techniques that belong to the basic toolbox of parameterized algorithms: dynamic programming over subsets, integer linear programming (ILP), and the use of well-quasi-ordering results from graph minors theory.
- Chapter 7 introduces treewidth, which is a graph measure that has important applications for parameterized algorithms. We discuss how to use dynamic programming and Courcelle’s theorem to solve problems on graphs of bounded treewidth and how these algorithms are used more generally, for example, in the context of bidimensionality for planar graphs.
- Chapter 8 presents results that are based on a combinatorial bound on the number of so-called “important separators”. We use this bound to show the fixed-parameter tractability of problems such as EDGE MULTICUT and DIRECTED FEEDBACK VERTEX SET. We also discuss randomized sampling of important cuts.
- The kernels presented in Chapter 9 form a representative sample of more advanced kernelization techniques. They demonstrate how the use of min-max results from graph theory, the probabilistic method, and the properties of planar graphs can be exploited in kernelization.
- Two different types of algebraic techniques are discussed in Chapter 10: algorithms based on the inclusion–exclusion principle and on polynomial identity testing. We use these techniques to present the fastest known parameterized algorithms for STEINER TREE and LONGEST PATH.
- In Chapter 11, we return to dynamic programming algorithms on graphs of bounded treewidth. This chapter presents three methods (subset convolution, *Cut & Count*, and a rank-based approach) for speeding up dynamic programming on tree decompositions.
- The notion of matroids is a fundamental concept in combinatorics and optimization. Recently, matroids have also been used for kernelization and parameterized algorithms. Chapter 12 gives a gentle introduction to some of these developments.
- Chapter 13 presents tools that allow us to give evidence that certain problems are not fixed-parameter tractable. The chapter introduces parameterized reductions and the W-hierarchy, and gives a sample of hardness results for various concrete problems.
- Chapter 14 uses the (Strong) Exponential Time Hypothesis to give running time lower bounds that are more refined than the bounds in Chapter 13. In many cases, these stronger complexity assumptions allow us to obtain lower bounds essentially matching the best known algorithms.
- Chapter 15 gives the tools for showing lower bounds for kernelization algorithms. We use methods of composition and polynomial-parameter transformations to show that certain problem, such as LONGEST PATH, do not admit polynomial kernels.

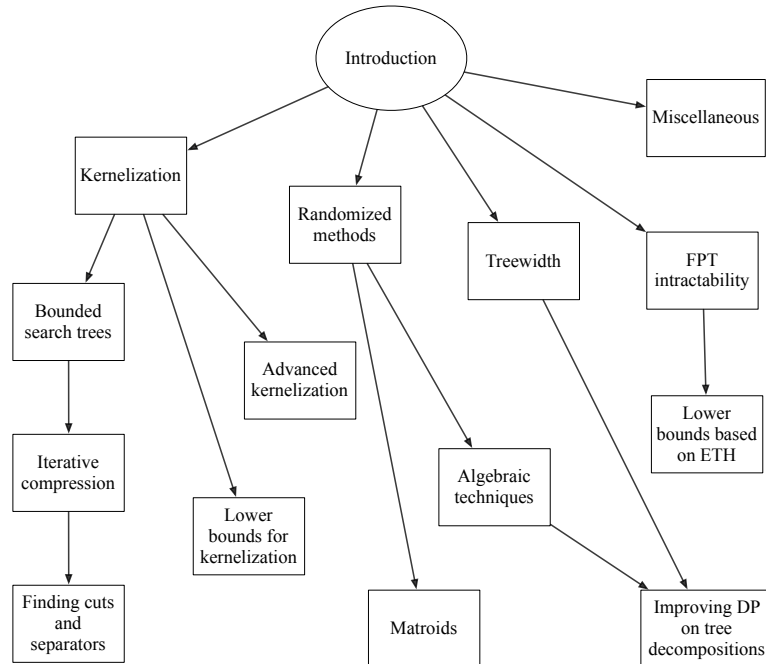


Fig. 0.1: Dependencies between the chapters

As in any textbook we will assume that the reader is familiar with the content of one chapter before moving to the next. On the other hand, for most chapters it is not necessary for the reader to have read *all* preceding chapters. Thus the book does not have to be read linearly from beginning to end. Figure 0.1 depicts the dependencies between the different chapters. For example, the chapters on Iterative Compression and Bounded Search Trees are considered necessary prerequisites to understand the chapter on finding cuts and separators.

Using the Book for Teaching

A course on parameterized algorithms should cover most of the material in Part I, except perhaps the more advanced sections marked with an asterisk. In Part II, the instructor may choose which chapters and which sections to teach based on his or her preferences. Our suggestion for a coherent set of topics from Part II is the following:

- All of Chapter 8, as it is relatively easily teachable. The sections of this chapter are based on each other and hence should be taught in this order, except that perhaps Section 8.4 and Sections 8.5–8.6 are interchangeable.
- Chapter 9 contains four independent sections. One could select Section 9.1 (FEEDBACK VERTEX SET) and Section 9.3 (CONNECTED VERTEX COVER on planar graphs) in a first course.
- From Chapter 10, we suggest presenting Section 10.1 (inclusion–exclusion principle), and Section 10.4.1 (LONGEST PATH in time $2^k \cdot n^{\mathcal{O}(1)}$).
- From Chapter 11, we recommend teaching Sections 11.2.1 and *11.2.2, as they are most illustrative for the recent developments on algorithms on tree decompositions.
- From Chapter 12 we recommend teaching Section 12.3. If the students are unfamiliar with matroids, Section 12.1 provides a brief introduction to the topic.

Part III gives a self-contained exposition of the lower bound machinery. In this part, the sections not marked with an asterisk give a set of topics that can form the complexity part of a course on parameterized algorithms. In some cases, we have presented multiple reductions showcasing the same kind of lower bounds; the instructor can choose from these examples according to the needs of the course. Section 14.4.1 contains some more involved proofs, but one can give a coherent overview of this section even while omitting most of the proofs.

Bergen, Budapest, Chennai, Warsaw

June 2015

*Marek Cygan, Fedor V. Fomin, Łukasz Kowalik, Daniel Lokshtanov,
Dániel Marx, Marcin Pilipczuk, Michał Pilipczuk and Saket Saurabh*

Acknowledgements

Many of our colleagues helped with valuable comments, corrections and suggestions. We are grateful for feedback to Yixin Cao, Chandra Chekuri, Kyril Elagin, Mike Fellows, Tobias Friedrich, Serge Gaspers, Petr Golovach, Fabrizio Grandoni, Jiong Guo, Gregory Gutin, Danny Hermelin, Pim van 't Hof, Philip Klein, Sudeshna Kolay, Alexander Kulikov, Michael Lampis, Neeldhara Misra, Jesper Nederlof, Sang-il Oum, Fahad Panolan, Venkatesh Raman, M.S. Ramanujan, Lajos Rónyai, Dimitrios M. Thilikos, and Magnus Wahlström. We would like to especially thank Bart M.P. Jansen for his enormous number of very insightful comments, as well as his help in organizing the FPT School in Będlewo.

We are greatly indebted to Felix Reidl, the creator of the squirrel, the platypus, the hamster, and the woodpecker, for the magnificent illustrations embellishing this book.

Our work has received funding from the European Research Council under the European Union's Seventh Framework Programme (FP/2007-2013)/ERC Grant Agreements No. 267959 (Rigorous Theory of Preprocessing), 306992 (Parameterized Approximation), and 280152 (PARAMTIGHT: Parameterized complexity and the search for tight complexity results), from the Bergen Research Foundation and the University of Bergen through project "BeHard", as well as from Polish National Science Centre grant UMO-2012/05/D/ST6/03214.

An important part of this work was done while subsets of the authors attended Dagstuhl Seminar 14071 "Graph Modification Problems" and ICERM Research Cluster "Towards Efficient Algorithms Exploiting Graph Structure".

We also acknowledge the great help we have received from the staff of the Conference Center of the Institute of Mathematics of the Polish Academy of Science when organizing our school in Będlewo.

Saket would like to thank his mentor Venkatesh Raman for introducing him to this area and bearing with him for the last fourteen years. To him, he says "Thank you, sir!"

Daniel, Fedor, and Saket express their deepest gratitude to Mike Fellows, for teaching them parameterized complexity and being generous in sharing new ideas.

Last but not least, we would like to thank our families and other people who supported us during the writing of this book, including among others: Agata, Olga, Ania, Szymon, Monika, Zosia, Piotruś, Sushmita, Rebecca, Tanya, Nikolas, Shurik, Gosha, Zsuzsa, Veronika, and Zsolt.

Contents

Part I Basic toolbox	1
1 Introduction	3
1.1 Formal definitions	12
2 Kernelization	17
2.1 Formal definitions	18
2.2 Some simple kernels	20
2.2.1 VERTEX COVER	21
2.2.2 FEEDBACK ARC SET IN TOURNAMENTS	22
2.2.3 EDGE CLIQUE COVER	25
2.3 Crown decomposition	26
2.3.1 VERTEX COVER	29
2.3.2 MAXIMUM SATISFIABILITY	29
2.4 Expansion lemma	30
2.5 Kernels based on linear programming	33
2.6 Sunflower lemma	38
2.6.1 d -HITTING SET	39
3 Bounded search trees	51
3.1 VERTEX COVER	53
3.2 How to solve recursive relations	55
3.3 FEEDBACK VERTEX SET	57
3.4 VERTEX COVER ABOVE LP	60
3.5 CLOSEST STRING	67
4 Iterative compression	77
4.1 Illustration of the basic technique	78
4.1.1 A few generic steps	80
4.2 FEEDBACK VERTEX SET IN TOURNAMENTS	81
4.2.1 Solving DISJOINT FEEDBACK VERTEX SET IN TOURNAMENTS in polynomial time	83
4.3 FEEDBACK VERTEX SET	86
4.3.1 First algorithm for DISJOINT FEEDBACK VERTEX SET	87
*4.3.2 Faster algorithm for DISJOINT FEEDBACK VERTEX SET	88
4.4 ODD CYCLE TRANSVERSAL	91

5	Randomized methods in parameterized algorithms	99
5.1	A simple randomized algorithm for FEEDBACK VERTEX SET	101
5.2	Color coding	103
5.2.1	A color coding algorithm for LONGEST PATH	104
5.3	Random separation	106
*5.4	A divide and color algorithm for LONGEST PATH	108
5.5	A chromatic coding algorithm for d -CLUSTERING	113
5.6	Derandomization	117
5.6.1	Basic pseudorandom objects	118
5.6.2	Derandomization of algorithms based on variants of color coding	120
6	Miscellaneous	129
6.1	Dynamic programming over subsets	130
6.1.1	SET COVER	130
6.1.2	STEINER TREE	131
6.2	INTEGER LINEAR PROGRAMMING	135
6.2.1	The example of IMBALANCE	136
6.3	Graph minors and the Robertson-Seymour theorem	140
7	Treewidth	151
7.1	Trees, narrow grids, and dynamic programming	153
7.2	Path and tree decompositions	157
7.3	Dynamic programming on graphs of bounded treewidth	162
7.3.1	WEIGHTED INDEPENDENT SET	162
7.3.2	DOMINATING SET	168
7.3.3	STEINER TREE	172
7.4	Treewidth and monadic second-order logic	177
7.4.1	Monadic second-order logic on graphs	178
7.4.2	Courcelle's theorem	183
7.5	Graph searching, interval and chordal graphs	185
7.6	Computing treewidth	190
7.6.1	Balanced separators and separations	192
7.6.2	An FPT approximation algorithm for treewidth	195
7.7	Win/win approaches and planar problems	199
7.7.1	Grid theorems	200
7.7.2	Bidimensionality	203
7.7.3	Shifting technique	211
*7.8	Irrelevant vertex technique	216
7.9	Beyond treewidth	228
	Part II Advanced algorithmic techniques	245
8	Finding cuts and separators	247
8.1	Minimum cuts	249

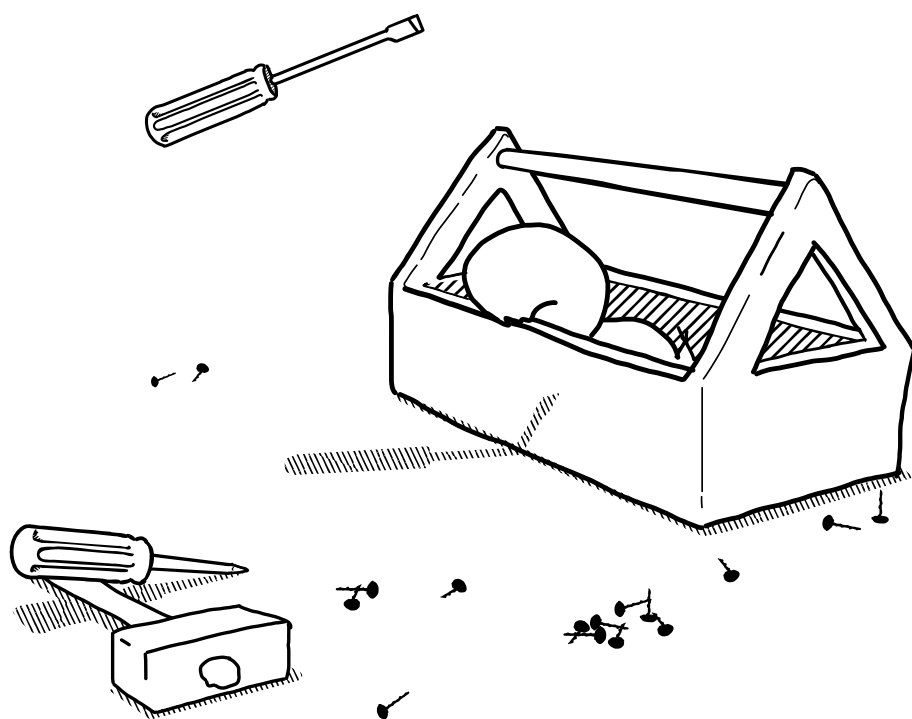
8.2	Important cuts	254
8.3	EDGE MULTIWAY CUT	261
8.4	(p, q) -clustering	264
8.5	Directed graphs	272
8.6	DIRECTED FEEDBACK VERTEX SET	274
8.7	Vertex-deletion problems	278
9	Advanced kernelization algorithms	285
9.1	A quadratic kernel for FEEDBACK VERTEX SET	287
9.1.1	Proof of Gallai's theorem	290
9.1.2	Detecting flowers with Gallai's theorem	295
9.1.3	Exploiting the blocker	296
*9.2	Moments and MAX-ER-SAT	299
9.2.1	Algebraic representation	301
9.2.2	Tools from probability theory	302
9.2.3	Analyzing moments of $X(\psi)$	304
9.3	CONNECTED VERTEX COVER in planar graphs	307
9.3.1	Plane graphs and Euler's formula	308
9.3.2	A lemma on planar bipartite graphs	309
9.3.3	The case of CONNECTED VERTEX COVER	310
9.4	Turing kernelization	313
9.4.1	A polynomial Turing kernel for MAX LEAF SUBTREE	315
10	Algebraic techniques: sieves, convolutions, and polynomials	321
10.1	Inclusion–exclusion principle	322
10.1.1	HAMILTONIAN CYCLE	323
10.1.2	STEINER TREE	324
10.1.3	CHROMATIC NUMBER	326
10.2	Fast zeta and Möbius transforms	328
10.3	Fast subset convolution and cover product	331
10.3.1	Counting colorings via fast subset convolution	334
10.3.2	Convolutions and cover products in min-sum semirings	334
10.4	Multivariate polynomials	337
10.4.1	LONGEST PATH in time $2^k n^{\mathcal{O}(1)}$	340
10.4.2	LONGEST PATH in time $2^{k/2} n^{\mathcal{O}(1)}$ for undirected bi-partite graphs	346
*10.4.3	LONGEST PATH in time $2^{3k/4} n^{\mathcal{O}(1)}$ for undirected graphs	349
11	Improving dynamic programming on tree decompositions	357
11.1	Applying fast subset convolution	358
11.1.1	Counting perfect matchings	358
11.1.2	DOMINATING SET	359
11.2	Connectivity problems	361
11.2.1	Cut & Count	361
*11.2.2	Deterministic algorithms by Gaussian elimination	365

12 Matroids	377
12.1 Classes of matroids	379
12.1.1 Linear matroids and matroid representation	379
12.1.2 Representation of uniform matroids	380
12.1.3 Graphic matroids	381
12.1.4 Transversal matroids	382
12.1.5 Direct sum and partition matroids	383
12.2 Algorithms for matroid problems	383
12.2.1 Matroid intersection and matroid parity	386
12.2.2 FEEDBACK VERTEX SET in subcubic graphs	389
12.3 Representative sets	392
12.3.1 Playing on a matroid	394
12.3.2 Kernel for d -HITTING SET	398
12.3.3 Kernel for d -SET PACKING	399
12.3.4 ℓ -MATROID INTERSECTION	401
12.3.5 LONG DIRECTED CYCLE	403
12.4 Representative families for uniform matroids	409
12.5 Faster LONG DIRECTED CYCLE	410
12.6 LONGEST PATH	413
 Part III Lower bounds	 419
13 Fixed-parameter intractability	421
13.1 Parameterized reductions	424
13.2 Problems at least as hard as CLIQUE	426
13.3 The W-hierarchy	435
*13.4 Turing machines	439
*13.5 Problems complete for $W[1]$ and $W[2]$	443
13.6 Parameterized reductions: further examples	448
13.6.1 Reductions keeping the structure of the graph	448
13.6.2 Reductions with vertex representation	451
13.6.3 Reductions with vertex and edge representation	453
 14 Lower bounds based on the Exponential-Time Hypothesis	 467
14.1 The Exponential-Time Hypothesis: motivation and basic results	468
14.2 ETH and classical complexity	473
14.3 ETH and fixed-parameter tractable problems	475
14.3.1 Immediate consequences for parameterized complexity	476
*14.3.2 Slightly super-exponential parameterized complexity	477
*14.3.3 Double exponential parameterized complexity	484
14.4 ETH and $W[1]$ -hard problems	485
14.4.1 Planar and geometric problems	489
*14.5 Lower bounds based on the Strong Exponential-Time Hypothesis	502

14.5.1 HITTING SET parameterized by the size of the universe	503
14.5.2 Dynamic programming on treewidth	508
14.5.3 A refined lower bound for DOMINATING SET	514
15 Lower bounds for kernelization	523
15.1 Compositionality	524
15.1.1 Distillation	525
15.1.2 Composition	529
15.1.3 AND-distillations and AND-compositions	533
15.2 Examples	534
15.2.1 Instance selector: SET SPLITTING	534
15.2.2 Polynomial parameter transformations: COLORFUL GRAPH MOTIF and STEINER TREE	537
15.2.3 A more involved one: SET COVER	540
15.2.4 Structural parameters: CLIQUE parameterized by the vertex cover number	544
15.3 Weak compositions	547
References	556
Appendix	576
Notation	577
Problem definitions	581
Index	599
Author index	609

Part I

Basic toolbox



Chapter 1

Introduction



A squirrel, a platypus and a hamster walk into a bar...

Imagine that you are an exceptionally tech-savvy security guard of a bar in an undisclosed small town on the west coast of Norway. Every Friday, half of the inhabitants of the town go out, and the bar you work at is well known for its nightly brawls. This of course results in an excessive amount of work for you; having to throw out intoxicated guests is tedious and rather unpleasant labor. Thus you decide to take preemptive measures. As the town is small, you know everyone in it, and you also know who will be likely to fight with whom if they are admitted to the bar. So you wish to plan ahead, and only admit people if they will not be fighting with anyone else at the bar. At the same time, the management wants to maximize profit and is not too happy if you on any given night reject more than k people at the door. Thus, you are left with the following optimization problem. You have a list of all of the n people who will come to the bar, and for each pair of people a prediction of whether or not they will fight if they both are admitted. You need to figure out whether it is possible to admit everyone except for at most k troublemakers, such that no fight breaks out among the admitted guests. Let us call this problem the `BAR FIGHT PREVENTION` problem. Figure 1.1 shows an instance of the problem and a solution for $k = 3$. One can easily check that this instance has no solution with $k = 2$.

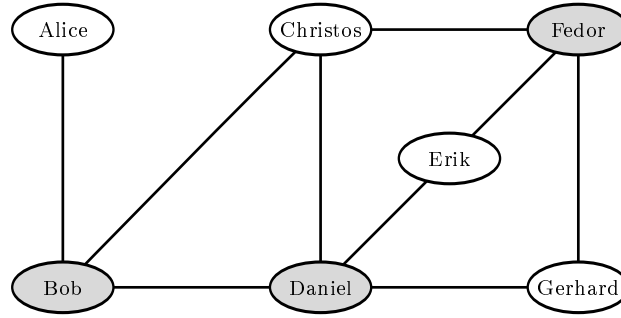


Fig. 1.1: An instance of the BAR FIGHT PREVENTION problem with a solution for $k = 3$. An edge between two guests means that they will fight if both are admitted

Efficient algorithms for BAR FIGHT PREVENTION

Unfortunately, BAR FIGHT PREVENTION is a classic NP-complete problem (the reader might have heard of it under the name VERTEX COVER), and so the best way to solve the problem is by trying all possibilities, right? If there are $n = 1000$ people planning to come to the bar, then you can quickly code up the brute-force solution that tries each of the $2^{1000} \approx 1.07 \cdot 10^{301}$ possibilities. Sadly, this program won't terminate before the guests arrive, probably not even before the universe implodes on itself. Luckily, the number k of guests that should be rejected is not that large, $k \leq 10$. So now the program only needs to try $\binom{1000}{10} \approx 2.63 \cdot 10^{23}$ possibilities. This is much better, but still quite infeasible to do in one day, even with access to supercomputers.

So should you give up at this point, and resign yourself to throwing guests out after the fights break out? Well, at least you can easily identify some peaceful souls to accept, and some troublemakers you need to refuse at the door for sure. Anyone who does not have a potential conflict with anyone else can be safely moved to the list of people to accept. On the other hand, if some guy will fight with at least $k + 1$ other guests you have to reject him — as otherwise you will have to reject all of his $k + 1$ opponents, thereby upsetting the management. If you identify such a troublemaker (in the example of Fig. 1.1, Daniel is such a troublemaker), you immediately strike him from the guest list, and decrease the number k of people you can reject by one.¹

If there is no one left to strike out in this manner, then we know that each guest will fight with at most k other guests. Thus, rejecting any single guest will resolve at most k potential conflicts. And so, if there are more than k^2

¹ The astute reader may observe that in Fig. 1.1, after eliminating Daniel and setting $k = 2$, Fedor still has three opponents, making it possible to eliminate him and set $k = 1$. Then Bob, who is in conflict with Alice and Christos, can be eliminated, resolving all conflicts.

potential conflicts, you know that there is no way to ensure a peaceful night at the bar by rejecting only k guests at the door. As each guest who has not yet been moved to the accept or reject list participates in at least one and at most k potential conflicts, and there are at most k^2 potential conflicts, there are at most $2k^2$ guests whose fate is yet undecided. Trying all possibilities for these will need approximately $\binom{2k^2}{k} \leq \binom{200}{10} \approx 2.24 \cdot 10^{16}$ checks, which is feasible to do in less than a day on a modern supercomputer, but quite hopeless on a laptop.

If it is safe to admit anyone who does not participate in any potential conflict, what about those who participate in exactly one? If Alice has a conflict with Bob, but with no one else, then it is always a good idea to admit Alice. Indeed, you cannot accept both Alice and Bob, and admitting Alice cannot be any worse than admitting Bob: if Bob is in the bar, then Alice has to be rejected for sure and potentially some other guests as well. Therefore, it is safe to accept Alice, reject Bob, and decrease k by one in this case. This way, you can always decide the fate of any guest with only one potential conflict. At this point, each guest you have not yet moved to the accept or reject list participates in at least two and at most k potential conflicts. It is easy to see that with this assumption, having at most k^2 unresolved conflicts implies that there are only at most k^2 guests whose fate is yet undecided, instead of the previous upper bound of $2k^2$. Trying all possibilities for which of those to refuse at the door requires $\binom{k^2}{k} \leq \binom{100}{10} \approx 1.73 \cdot 10^{13}$ checks. With a clever implementation, this takes less than half a day on a laptop, so if you start the program in the morning you'll know who to refuse at the door by the time the bar opens. Therefore, instead of using brute force to go through an enormous search space, we used simple observations to reduce the search space to a manageable size. This algorithmic technique, using reduction rules to decrease the size of the instance, is called *kernelization*, and will be the subject of Chapter 2 (with some more advanced examples appearing in Chapter 9).

It turns out that a simple observation yields an even faster algorithm for BAR FIGHT PREVENTION. The crucial point is that every conflict has to be resolved, and that the only way to resolve a conflict is to refuse at least one of the two participants. Thus, as long as there is at least one unresolved conflict, say between Alice and Bob, we proceed as follows. Try moving Alice to the reject list and run the algorithm recursively to check whether the remaining conflicts can be resolved by rejecting at most $k - 1$ guests. If this succeeds you already have a solution. If it fails, then move Alice back onto the undecided list, move Bob to the reject list and run the algorithm recursively to check whether the remaining conflicts can be resolved by rejecting at most $k - 1$ additional guests (see Fig. 1.2). If this recursive call also fails to find a solution, then you can be sure that there is no way to avoid a fight by rejecting at most k guests.

What is the running time of this algorithm? All it does is to check whether all conflicts have been resolved, and if not, it makes two recursive calls. In

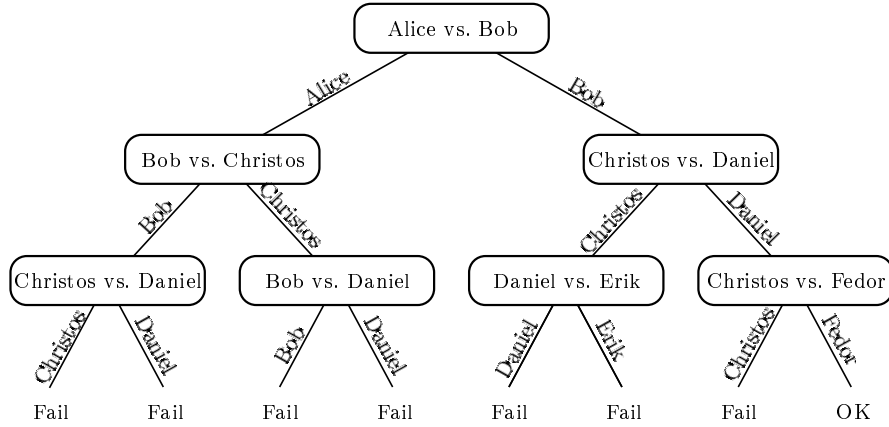


Fig. 1.2: The search tree for BAR FIGHT PREVENTION with $k = 3$. In the leaves marked with “Fail”, the parameter k is decreased to zero, but there are still unresolved conflicts. The rightmost branch of the search tree finds a solution: after rejecting Bob, Daniel, and Fedor, no more conflicts remain

both of the recursive calls the value of k decreases by 1, and when k reaches 0 all the algorithm has to do is to check whether there are any unresolved conflicts left. Hence there is a total of 2^k recursive calls, and it is easy to implement each recursive call to run in linear time $\mathcal{O}(n + m)$, where m is the total number of possible conflicts. Let us recall that we already achieved the situation where every undecided guest has at most k conflicts with other guests, so $m \leq nk/2$. Hence the total number of operations is approximately $2^k \cdot n \cdot k \leq 2^{10} \cdot 10,000 = 10,240,000$, which takes a fraction of a second on today’s laptops. Or cell phones, for that matter. You can now make the BAR FIGHT PREVENTION app, and celebrate with a root beer. This simple algorithm is an example of another algorithmic paradigm: the technique of *bounded search trees*. In Chapter 3, we will see several applications of this technique to various problems.

The algorithm above runs in time $\mathcal{O}(2^k \cdot k \cdot n)$, while the naive algorithm that tries every possible subset of k people to reject runs in time $\mathcal{O}(n^k)$. Observe that if k is considered to be a constant (say $k = 10$), then both algorithms run in polynomial time. However, as we have seen, there is a quite dramatic difference between the running times of the two algorithms. The reason is that even though the naive algorithm is a polynomial-time algorithm for every fixed value of k , the exponent of the polynomial depends on k . On the other hand, the final algorithm we designed runs in linear time for every fixed value of k ! This difference is what parameterized algorithms and complexity is all about. In the $\mathcal{O}(2^k \cdot k \cdot n)$ -time algorithm, the combinatorial explosion is restricted to the parameter k : the running time is exponential

in k , but depends only polynomially (actually, linearly) on n . Our goal is to find algorithms of this form.

Algorithms with running time $f(k) \cdot n^c$, for a constant c independent of both n and k , are called *fixed-parameter algorithms*, or FPT algorithms. Typically the goal in parameterized algorithmics is to design FPT algorithms, trying to make both the $f(k)$ factor and the constant c in the bound on the running time as small as possible. FPT algorithms can be put in contrast with less efficient XP algorithms (for *slice-wise polynomial*), where the running time is of the form $f(k) \cdot n^{g(k)}$, for some functions f, g . There is a tremendous difference in the running times $f(k) \cdot n^{g(k)}$ and $f(k) \cdot n^c$.

In parameterized algorithmics, k is simply a *relevant secondary measurement* that encapsulates some aspect of the input instance, be it the size of the solution sought after, or a number describing how “structured” the input instance is.

A negative example: vertex coloring

Not every choice for what k measures leads to FPT algorithms. Let us have a look at an example where it does not. Suppose the management of the hypothetical bar you work at doesn’t want to refuse anyone at the door, but still doesn’t want any fights. To achieve this, they buy $k - 1$ more bars across the street, and come up with the following brilliant plan. Every night they will compile a list of the guests coming, and a list of potential conflicts. Then you are to split the guest list into k groups, such that no two guests with a potential conflict between them end up in the same group. Then each of the groups can be sent to one bar, keeping everyone happy. For example, in Fig. 1.1, we may put Alice and Christos in the first bar, Bob, Erik, and Gerhard in the second bar, and Daniel and Fedor in the third bar.

We model this problem as a graph problem, representing each person as a vertex, and each conflict as an edge between two vertices. A partition of the guest list into k groups can be represented by a function that assigns to each vertex an integer between 1 and k . The objective is to find such a function that, for every edge, assigns different numbers to its two endpoints. A function that satisfies these constraints is called a *proper k -coloring* of the graph. Not every graph has a proper k -coloring. For example, if there are $k + 1$ vertices with an edge between every pair of them, then each of these vertices needs to be assigned a unique integer. Hence such a graph does not have a proper k -coloring. This gives rise to a computational problem, called VERTEX COLORING. Here we are given as input a graph G and an integer k , and we need to decide whether G has a proper k -coloring.

It is well known that VERTEX COLORING is NP-complete, so we do not hope for a polynomial-time algorithm that works in all cases. However, it is fair to assume that the management does not want to own more than $k = 5$ bars on the same street, so we will gladly settle for a $\mathcal{O}(2^k \cdot n^c)$ -time algorithm for some constant c , mimicking the success we had with our first problem. Unfortunately, deciding whether a graph G has a proper 5-coloring is NP-complete, so any $f(k) \cdot n^c$ -time algorithm for VERTEX COLORING for any function f and constant c would imply that $P = NP$; indeed, suppose such an algorithm existed. Then, given a graph G , we can decide whether G has a proper 5-coloring in time $f(5) \cdot n^c = \mathcal{O}(n^c)$. But then we have a polynomial-time algorithm for an NP-hard problem, implying $P = NP$. Observe that even an XP algorithm with running time $f(k) \cdot n^{g(k)}$ for any functions f and g would imply that $P = NP$ by an identical argument.

A hard parameterized problem: finding cliques

The example of VERTEX COLORING illustrates that parameterized algorithms are not all-powerful: there are parameterized problems that do not seem to admit FPT algorithms. But very importantly, in this specific example, we could explain very precisely why we are not able to design efficient algorithms, even when the number of bars is small. From the perspective of an algorithm designer such insight is very useful; she can now stop wasting time trying to design efficient algorithms based only on the fact that the number of bars is small, and start searching for other ways to attack the problem instances. If we are trying to make a polynomial-time algorithm for a problem and failing, it is quite likely that this is because the problem is NP-hard. Is the theory of NP-hardness the right tool also for giving negative evidence for fixed-parameter tractability? In particular, if we are trying to make an $f(k) \cdot n^c$ -time algorithm and fail to do so, is it because the problem is NP-hard for some fixed constant value of k , say $k = 100$? Let us look at another example problem.

Now that you have a program that helps you decide who to refuse at the door and who to admit, you are faced with a different problem. The people in the town you live in have friends who might get upset if their friend is refused at the door. You are quite skilled at martial arts, and you can handle at most $k - 1$ angry guys coming at you, but probably not k . What you are most worried about are groups of at least k people where everyone in the group is friends with everyone else. These groups tend to have an “all for one and one for all” mentality — if one of them gets mad at you, they all do. Small as the town is, you know exactly who is friends with whom, and you want to figure out whether there is a group of at least k people where everyone is friends with everyone else. You model this as a graph problem where every person is a vertex and two vertices are connected by an edge if the corresponding persons are friends. What you are looking for is a *clique* on k vertices, that

is, a set of k vertices with an edge between every pair of them. This problem is known as the **CLIQUE** problem. For example, if we interpret now the edges of Fig. 1.1 as showing friendships between people, then Bob, Christos, and Daniel form a clique of size 3.

There is a simple $\mathcal{O}(n^k)$ -time algorithm to check whether a clique on at least k vertices exists; for each of the $\binom{n}{k} = \mathcal{O}\left(\frac{n^k}{k^k}\right)$ subsets of vertices of size k , we check in time $\mathcal{O}(k^2)$ whether every pair of vertices in the subset is adjacent. Unfortunately, this XP algorithm is quite hopeless to run for $n = 1000$ and $k = 10$. Can we design an FPT algorithm for this problem? So far, no one has managed to find one. Could it be that this is because finding a k -clique is NP-hard for some fixed value of k ? Suppose the problem was NP-hard for $k = 100$. We just gave an algorithm for finding a clique of size 100 in time $\mathcal{O}(n^{100})$, which is polynomial time. We would then have a polynomial-time algorithm for an NP-hard problem, implying that $P = NP$. So we cannot expect to be able to use NP-hardness in this way in order to rule out an FPT algorithm for **CLIQUE**. More generally, it seems very difficult to use NP-hardness in order to explain why a problem that does have an XP algorithm does not admit an FPT algorithm.

Since NP-hardness is insufficient to differentiate between problems with $f(k) \cdot n^{g(k)}$ -time algorithms and problems with $f(k) \cdot n^c$ -time algorithms, we resort to stronger complexity theoretical assumptions. The theory of W[1]-hardness (see Chapter 13) allows us to prove (under certain complexity assumptions) that even though a problem is polynomial-time solvable for every fixed k , the parameter k has to appear in the exponent of n in the running time, that is, the problem is not FPT. This theory has been quite successful for identifying which parameterized problems are FPT and which are unlikely to be. Besides this qualitative classification of FPT versus W[1]-hard, more recent developments give us also (an often surprisingly tight) quantitative understanding of the time needed to solve a parameterized problem. Under reasonable assumptions about the hardness of CNF-SAT (see Chapter 14), it is possible to show that there is no $f(k) \cdot n^c$, or even a $f(k) \cdot n^{o(k)}$ -time algorithm for finding a clique on k vertices. Thus, up to constant factors in the exponent, the naive $\mathcal{O}(n^k)$ -time algorithm is optimal! Over the past few years, it has become a rule, rather than an exception, that whenever we are unable to significantly improve the running time of a parameterized algorithm, we are able to show that the existing algorithms are asymptotically optimal, under reasonable assumptions. For example, under the same assumptions that we used to rule out an $f(k) \cdot n^{o(k)}$ -time algorithm for solving **CLIQUE**, we can also rule out a $2^{o(k)} \cdot n^{\mathcal{O}(1)}$ -time algorithm for the **BAR FIGHT PREVENTION** problem from the beginning of this chapter.

Any algorithmic theory is incomplete without an accompanying complexity theory that establishes intractability of certain problems. There

Problem	Good news	Bad news
BAR FIGHT PREVENTION	$\mathcal{O}(2^k \cdot k \cdot n)$ -time algorithm	NP-hard (probably not in P)
CLIQUE with Δ	$\mathcal{O}(2^\Delta \cdot \Delta^2 \cdot n)$ -time algorithm	NP-hard (probably not in P)
CLIQUE with k	$n^{\mathcal{O}(k)}$ -time algorithm	W[1]-hard (probably not FPT)
VERTEX COLORING		NP-hard for $k = 3$ (probably not XP)

Fig. 1.3: Overview of the problems in this chapter

is such a complexity theory providing lower bounds on the running time required to solve parameterized problems.

Finding cliques — with a different parameter

OK, so there probably is no algorithm for solving CLIQUE with running time $f(k) \cdot n^{o(k)}$. But what about those scary groups of people that might come for you if you refuse the wrong person at the door? They do not care at all about the computational hardness of CLIQUE, and neither do their fists. What can you do? Well, in Norway most people do not have too many friends. In fact, it is quite unheard of that someone has more than $\Delta = 20$ friends. That means that we are trying to find a k -clique in a graph of maximum degree Δ . This can be done quite efficiently: if we guess one vertex v in the clique, then the remaining vertices in the clique must be among the Δ neighbors of v . Thus we can try all of the 2^Δ subsets of the neighbors of v , and return the largest clique that we found. The total running time of this algorithm is $\mathcal{O}(2^\Delta \cdot \Delta^2 \cdot n)$, which is quite feasible for $\Delta = 20$. Again it is possible to use complexity theoretic assumptions on the hardness of CNF-SAT to show that this algorithm is asymptotically optimal, up to multiplicative constants in the exponent.

What the algorithm above shows is that the CLIQUE problem is FPT when the parameter is the maximum degree Δ of the input graph. At the same time CLIQUE is probably not FPT when the parameter is the solution size k . Thus, the classification of the problem into “tractable” or “intractable” crucially depends on the choice of parameter. This makes a lot of sense; the more we know about our input instances, the more we can exploit algorithmically!

The art of parameterization

For typical optimization problems, one can immediately find a relevant parameter: the size of the solution we are looking for. In some cases, however, it is not completely obvious what we mean by the size of the solution. For example, consider the variant of **BAR FIGHT PREVENTION** where we want to reject at most k guests such that the number of conflicts is reduced to at most ℓ (as we believe that the bouncers at the bar can handle ℓ conflicts, but not more). Then we can parameterize either by k or by ℓ . We may even parameterize by both: then the goal is to find an FPT algorithm with running time $f(k, \ell) \cdot n^c$ for some computable function f depending only on k and ℓ . Thus the theory of parameterization and FPT algorithms can be extended to considering a set of parameters at the same time. Formally, however, one can express parameterization by k and ℓ simply by defining the value $k + \ell$ to be the parameter: an $f(k, \ell) \cdot n^c$ algorithm exists if and only if an $f(k + \ell) \cdot n^c$ algorithm exists.

The parameters k and ℓ in the extended **BAR FIGHT PREVENTION** example of the previous paragraph are related to the *objective* of the problem: they are parameters explicitly given in the input, defining the properties of the solution we are looking for. We get more examples of this type of parameter if we define variants of **BAR FIGHT PREVENTION** where we need to reject at most k guests such that, say, the number of conflicts decreases by p , or such that each accepted guest has conflicts with at most d other accepted guests, or such that the average number of conflicts per guest is at most a . Then the parameters p , d , and a are again explicitly given in the input, telling us what kind of solution we need to find. The parameter Δ (maximum degree of the graph) in the **CLIQUE** example is a parameter of a very different type: it is not given explicitly in the input, but it is a *measure* of some property of the input instance. We defined and explored this particular measure because we believed that it is typically small in the input instances we care about: this parameter expresses some structural property of typical instances. We can identify and investigate any number of such parameters. For example, in problems involving graphs, we may parameterize by any structural parameter of the graph at hand. Say, if we believe that the problem is easy on planar graphs and the instances are “almost planar”, then we may explore the parameterization by the genus of the graph (roughly speaking, a graph has genus g if it can be drawn without edge crossings on a sphere with g holes in it). A large part of Chapter 7 (and also Chapter 11) is devoted to parameterization by treewidth, which is a very important parameter measuring the “tree-likeness” of the graph. For problems involving satisfiability of Boolean formulas, we can have such parameters as the number of variables, or clauses, or the number of clauses that need to be satisfied, or that are allowed not to be satisfied. For problems involving a set of strings, one can parameterize by the maximum length of the strings, by the size of the alphabet, by the maximum number of distinct symbols appearing in each string, etc. In

a problem involving a set of geometric objects (say, points in space, disks, or polygons), one may parameterize by the maximum number of vertices of each polygon or the dimension of the space where the problem is defined. For each problem, with a bit of creativity, one can come up with a large number of (combinations of) parameters worth studying.

For the same problem there can be multiple choices of parameters. Selecting the right parameter(s) for a particular problem is an art.

Parameterized complexity allows us to study how different parameters influence the complexity of the problem. A successful parameterization of a problem needs to satisfy two properties. First, we should have some reason to believe that the selected parameter (or combination of parameters) is typically small on input instances in some application. Second, we need efficient algorithms where the combinatorial explosion is restricted to the parameter(s), that is, we want the problem to be FPT with this parameterization. Finding good parameterizations is an art on its own and one may spend quite some time on analyzing different parameterizations of the same problem. However, in this book we focus more on explaining algorithmic techniques via carefully chosen illustrative examples, rather than discussing every possible aspect of a particular problem. Therefore, even though different parameters and parameterizations will appear throughout the book, we will not try to give a complete account of all known parameterizations and results for any concrete problem.

1.1 Formal definitions

We finish this chapter by leaving the realm of pub jokes and moving to more serious matters. Before we start explaining the techniques for designing parameterized algorithms, we need to introduce formal foundations of parameterized complexity. That is, we need to have rigorous definitions of what a parameterized problem is, and what it means that a parameterized problem belongs to a specific complexity class.

Definition 1.1. A *parameterized problem* is a language $L \subseteq \Sigma^* \times \mathbb{N}$, where Σ is a fixed, finite alphabet. For an instance $(x, k) \in \Sigma^* \times \mathbb{N}$, k is called the *parameter*.

For example, an instance of CLIQUE parameterized by the solution size is a pair (G, k) , where we expect G to be an undirected graph encoded as a string over Σ , and k is a positive integer. That is, a pair (G, k) belongs to the CLIQUE parameterized language if and only if the string G correctly encodes an undirected graph, which we will also denote by G , and moreover the graph

G contains a clique on k vertices. Similarly, an instance of the CNF-SAT problem (satisfiability of propositional formulas in CNF), parameterized by the number of variables, is a pair (φ, n) , where we expect φ to be the input formula encoded as a string over Σ and n to be the number of variables of φ . That is, a pair (φ, n) belongs to the CNF-SAT parameterized language if and only if the string φ correctly encodes a CNF formula with n variables, and the formula is satisfiable.

We define the size of an instance (x, k) of a parameterized problem as $|x| + k$. One interpretation of this convention is that, when given to the algorithm on the input, the parameter k is encoded in unary.

Definition 1.2. A parameterized problem $L \subseteq \Sigma^* \times \mathbb{N}$ is called *fixed-parameter tractable* (FPT) if there exists an algorithm $\mathcal{A}$ (called a *fixed-parameter algorithm*), a computable function $f: \mathbb{N} \rightarrow \mathbb{N}$, and a constant c such that, given $(x, k) \in \Sigma^* \times \mathbb{N}$, the algorithm $\mathcal{A}$ correctly decides whether $(x, k) \in L$ in time bounded by $f(k) \cdot |(x, k)|^c$. The complexity class containing all fixed-parameter tractable problems is called FPT.

Before we go further, let us make some remarks about the function f in this definition. Observe that we assume f to be computable, as otherwise we would quickly run into trouble when developing complexity theory for fixed-parameter tractability. For technical reasons, it will be convenient to assume, from now on, that f is also nondecreasing. Observe that this assumption has no influence on the definition of fixed-parameter tractability as stated in Definition 1.2, since for every computable function $f: \mathbb{N} \rightarrow \mathbb{N}$ there exists a computable nondecreasing function $\tilde{f}$ that is never smaller than f : we can simply take $\tilde{f}(k) = \max_{i=0,1,\dots,k} f(i)$. Also, for standard algorithmic results it is always the case that the bound on the running time is a nondecreasing function of the complexity measure, so this assumption is indeed satisfied in practice. However, the assumption about f being nondecreasing is formally needed in various situations, for example when performing reductions.

We now define the complexity class XP.

Definition 1.3. A parameterized problem $L \subseteq \Sigma^* \times \mathbb{N}$ is called *slice-wise polynomial* (XP) if there exists an algorithm $\mathcal{A}$ and two computable functions $f, g: \mathbb{N} \rightarrow \mathbb{N}$ such that, given $(x, k) \in \Sigma^* \times \mathbb{N}$, the algorithm $\mathcal{A}$ correctly decides whether $(x, k) \in L$ in time bounded by $f(k) \cdot |(x, k)|^{g(k)}$. The complexity class containing all slice-wise polynomial problems is called XP.

Again, we shall assume that the functions f, g in this definition are nondecreasing.

The definition of a parameterized problem, as well as the definitions of the classes FPT and XP, can easily be generalized to encompass multiple parameters. In this setting we simply allow k to be not just one nonnegative

integer, but a vector of d nonnegative integers, for some fixed constant d . Then the functions f and g in the definitions of the complexity classes FPT and XP can depend on all these parameters.

Just as “polynomial time” and “polynomial-time algorithm” usually refer to time polynomial in the input size, the terms “FPT time” and “FPT algorithms” refer to time $f(k)$ times a polynomial in the input size. Here f is a computable function of k and the degree of the polynomial is independent of both n and k . The same holds for “XP time” and “XP algorithms”, except that here the degree of the polynomial is allowed to depend on the parameter k , as long as it is upper bounded by $g(k)$ for some computable function g .

Observe that, given some parameterized problem L , the algorithm designer has essentially two different optimization goals when designing FPT algorithms for L . Since the running time has to be of the form $f(k) \cdot n^c$, one can:

- optimize the *parametric dependence* of the running time, i.e., try to design an algorithm where function f grows as slowly as possible; or
- optimize the *polynomial factor* in the running time, i.e., try to design an algorithm where constant c is as small as possible.

Both these goals are equally important, from both a theoretical and a practical point of view. Unfortunately, keeping track of and optimizing both factors of the running time can be a very difficult task. For this reason, most research on parameterized algorithms concentrates on optimizing one of the factors, and putting more focus on each of them constitutes one of the two dominant trends in parameterized complexity. Sometimes, when we are not interested in the exact value of the polynomial factor, we use the $\mathcal{O}^*$ -notation, which suppresses factors polynomial in the input size. More precisely, a running time $\mathcal{O}^*(f(k))$ means that the running time is upper bounded by $f(k) \cdot n^{\mathcal{O}(1)}$, where n is the input size.

The theory of parameterized complexity has been pioneered by Downey and Fellows over the last two decades [148, 149, 150, 151, 153]. The main achievement of their work is a comprehensive complexity theory for parameterized problems, with appropriate notions of reduction and completeness. The primary goal is to understand the qualitative difference between fixed-parameter tractable problems, and problems that do not admit such efficient algorithms. The theory contains a rich “positive” toolkit of techniques for developing efficient parameterized algorithms, as well as a corresponding “negative” toolkit that supports a theory of parameterized intractability. This textbook is mostly devoted to a presentation of the positive toolkit: in Chapters 2 through 12 we present various algorithmic techniques for designing fixed-parameter tractable algorithms. As we have argued, the process of algorithm design has to use both toolkits in order to be able to conclude that certain research directions are pointless. Therefore, in Part III we give an introduction to lower bounds for parameterized problems.

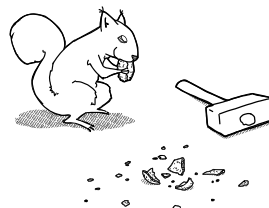
Bibliographic notes

Downey and Fellows laid the foundation of parameterized complexity in the series of papers [1, 149, 150, 151]. The classic reference on parameterized complexity is the book of Downey and Fellows [153]. The new edition of this book [154] is a comprehensive overview of the state of the art in many areas of parameterized complexity. The book of Flum and Grohe [189] is an extensive introduction to the area with a strong emphasis on the complexity viewpoint. An introduction to basic algorithmic techniques in parameterized complexity up to 2006 is given in the book of Niedermeier [376]. The recent book [51] contains a collection of surveys on different areas of parameterized complexity.

Chapter 2

Kernelization

Kernelization is a systematic approach to study polynomial-time preprocessing algorithms. It is an important tool in the design of parameterized algorithms. In this chapter we explain basic kernelization techniques such as crown decomposition, the expansion lemma, the sunflower lemma, and linear programming. We illustrate these techniques by obtaining kernels for VERTEX COVER, FEEDBACK ARC SET IN TOURNAMENTS, EDGE CLIQUE COVER, MAXIMUM SATISFIABILITY, and d -HITTING SET.



Preprocessing (data reduction or kernelization) is used universally in almost every practical computer implementation that aims to deal with an NP-hard problem. The goal of a preprocessing subroutine is to solve efficiently the “easy parts” of a problem instance and reduce it (shrink it) to its computationally difficult “core” structure (the *problem kernel* of the instance). In other words, the idea of this method is to reduce (but not necessarily solve) the given problem instance to an equivalent “smaller sized” instance in time polynomial in the input size. A slower exact algorithm can then be run on this smaller instance.

How can we measure the effectiveness of such a preprocessing subroutine? Suppose we define a useful preprocessing algorithm as one that runs in polynomial time and replaces an instance I with an equivalent instance that is at least one bit smaller. Then the existence of such an algorithm for an NP-hard problem would imply $P = NP$, making it unlikely that such an algorithm can be found. For a long time, there was no other suggestion for a formal definition of useful preprocessing, leaving the mathematical analysis of polynomial-time preprocessing algorithms largely neglected. But in the language of parameterized complexity, we can formulate a definition of useful preprocessing by demanding that large instances with a small parameter should be shrunk, while instances that are small compared to their parameter

do not have to be processed further. These ideas open up the “lost continent” of polynomial-time algorithms called kernelization.

In this chapter we illustrate some commonly used techniques to design kernelization algorithms through concrete examples. The next section, Section 2.1, provides formal definitions. In Section 2.2 we give kernelization algorithms based on so-called natural reduction rules. Section 2.3 introduces the concepts of crown decomposition and the expansion lemma, and illustrates it on MAXIMUM SATISFIABILITY. Section 2.5 studies tools based on linear programming and gives a kernel for VERTEX COVER. Finally, we study the sunflower lemma in Section 2.6 and use it to obtain a polynomial kernel for d -HITTING SET.

2.1 Formal definitions

We now turn to the formal definition that captures the notion of kernelization. A *data reduction rule*, or simply, reduction rule, for a parameterized problem Q is a function $\phi: \Sigma^* \times \mathbb{N} \rightarrow \Sigma^* \times \mathbb{N}$ that maps an instance (I, k) of Q to an equivalent instance (I', k') of Q such that ϕ is computable in time polynomial in $|I|$ and k . We say that two instances of Q are *equivalent* if $(I, k) \in Q$ if and only if $(I', k') \in Q$; this property of the reduction rule ϕ , that it translates an instance to an equivalent one, is sometimes referred to as the *safeness* or *soundness* of the reduction rule. In this book, we stick to the phrases: *a rule is safe* and *the safeness of a reduction rule*.

The general idea is to design a *preprocessing algorithm* that consecutively applies various data reduction rules in order to shrink the instance size as much as possible. Thus, such a preprocessing algorithm takes as input an instance $(I, k) \in \Sigma^* \times \mathbb{N}$ of Q , works in polynomial time, and returns an equivalent instance (I', k') of Q . In order to formalize the requirement that the output instance has to be small, we apply the main principle of Parameterized Complexity: The complexity is measured in terms of the parameter. Consequently, the *output size* of a preprocessing algorithm $\mathcal{A}$ is a function $\text{size}_{\mathcal{A}}: \mathbb{N} \rightarrow \mathbb{N} \cup \{\infty\}$ defined as follows:

$$\text{size}_{\mathcal{A}}(k) = \sup\{|I'| + k' : (I', k') = \mathcal{A}(I, k), I \in \Sigma^*\}.$$

In other words, we look at all possible instances of Q with a fixed parameter k , and measure the supremum of the sizes of the output of $\mathcal{A}$ on these instances. Note that this supremum may be infinite; this happens when we do not have any bound on the size of $\mathcal{A}(I, k)$ in terms of the input parameter k only. *Kernelization algorithms* are exactly these preprocessing algorithms whose output size is finite and bounded by a computable function of the parameter.

Definition 2.1 (Kernelization, kernel). A *kernelization algorithm*, or simply a *kernel*, for a parameterized problem Q is an algorithm $\mathcal{A}$ that, given

an instance (I, k) of Q , works in polynomial time and returns an equivalent instance (I', k') of Q . Moreover, we require that $\text{size}_{\mathcal{A}}(k) \leq g(k)$ for some computable function $g: \mathbb{N} \rightarrow \mathbb{N}$.

The size requirement in this definition can be reformulated as follows: There exists a computable function $g(\cdot)$ such that whenever (I', k') is the output for an instance (I, k) , then it holds that $|I'| + k' \leq g(k)$. If the upper bound $g(\cdot)$ is a polynomial (linear) function of the parameter, then we say that Q admits a *polynomial (linear) kernel*. We often abuse the notation and call the output of a kernelization algorithm the “reduced” equivalent instance, also a kernel.

In the course of this chapter, we will often encounter a situation when in some boundary cases we are able to completely resolve the considered problem instance, that is, correctly decide whether it is a yes-instance or a no-instance. Hence, for clarity, we allow the reductions (and, consequently, the kernelization algorithm) to return a yes/no answer instead of a reduced instance. Formally, to fit into the introduced definition of a kernel, in such cases the kernelization algorithm should instead return a constant-size trivial yes-instance or no-instance. Note that such instances exist for every parameterized language except for the empty one and its complement, and can be therefore hardcoded into the kernelization algorithm.

Recall that, given an instance (I, k) of Q , the size of the kernel is defined as the number of *bits* needed to encode the reduced equivalent instance I' plus the parameter value k' . However, when dealing with problems on graphs, hypergraphs, or formulas, often we would like to emphasize other aspects of output instances. For example, for a graph problem Q , we could say that Q admits a kernel with $\mathcal{O}(k^3)$ vertices and $\mathcal{O}(k^5)$ edges to emphasize the upper bound on the number of vertices and edges in the output instances. Similarly, for a problem defined on formulas, we could say that the problem admits a kernel with $\mathcal{O}(k)$ variables.

It is important to mention here that the early definitions of kernelization required that $k' \leq k$. On an intuitive level this makes sense, as the parameter k measures the complexity of the problem — thus the larger the k , the harder the problem. This requirement was subsequently relaxed, notably in the context of lower bounds. An advantage of the more liberal notion of kernelization is that it is robust with respect to polynomial transformations of the kernel. However, it limits the connection with practical preprocessing. All the kernels mentioned in this chapter respect the fact that the output parameter is at most the input parameter, that is, $k' \leq k$.

While usually in Computer Science we measure the efficiency of an algorithm by estimating its running time, the central measure of the efficiency of a kernelization algorithm is a bound on its output size. Although the actual running time of a kernelization algorithm is of-

ten very important for practical applications, in theory a kernelization algorithm is only required to run in polynomial time.

If we have a kernelization algorithm for a problem for which there is some algorithm (with any running time) to decide whether (I, k) is a yes-instance, then clearly the problem is FPT, as the size of the reduced instance I is simply a function of k (and independent of the input size n). However, a surprising result is that the converse is also true.

Lemma 2.2. *If a parameterized problem Q is FPT then it admits a kernelization algorithm.*

Proof. Since Q is FPT, there is an algorithm $\mathcal{A}$ deciding if $(I, k) \in Q$ in time $f(k) \cdot |I|^c$ for some computable function f and a constant c . We obtain a kernelization algorithm for Q as follows. Given an input (I, k) , the kernelization algorithm runs $\mathcal{A}$ on (I, k) , for at most $|I|^{c+1}$ steps. If it terminates with an answer, use that answer to return either that (I, k) is a yes-instance or that it is a no-instance. If $\mathcal{A}$ does not terminate within $|I|^{c+1}$ steps, then return (I, k) itself as the output of the kernelization algorithm. Observe that since $\mathcal{A}$ did not terminate in $|I|^{c+1}$ steps, we have that $f(k) \cdot |I|^c > |I|^{c+1}$, and thus $|I| < f(k)$. Consequently, we have $|I| + k \leq f(k) + k$, and we obtain a kernel of size at most $f(k) + k$; note that this upper bound is computable as $f(k)$ is a computable function. $\square$

Lemma 2.2 implies that a decidable problem admits a kernel if and only if it is fixed-parameter tractable. Thus, in a sense, kernelization can be another way of defining fixed-parameter tractability.

However, kernels obtained by this theoretical result are usually of exponential (or even worse) size, while problem-specific data reduction rules often achieve quadratic ($g(k) = \mathcal{O}(k^2)$) or even linear-size ($g(k) = \mathcal{O}(k)$) kernels. So a natural question for any concrete FPT problem is whether it admits a problem kernel that is bounded by a polynomial function of the parameter ($g(k) = k^{\mathcal{O}(1)}$). In this chapter we give polynomial kernels for several problems using some elementary methods. In Chapter 9, we give more advanced methods for obtaining kernels.

2.2 Some simple kernels

In this section we give kernelization algorithms for VERTEX COVER and FEEDBACK ARC SET IN TOURNAMENTS (FAST) based on a few natural reduction rules.

2.2.1 VERTEX COVER

Let G be a graph and $S \subseteq V(G)$. The set S is called a *vertex cover* if for every edge of G at least one of its endpoints is in S . In other words, the graph $G - S$ contains no edges and thus $V(G) \setminus S$ is an *independent set*. In the VERTEX COVER problem, we are given a graph G and a positive integer k as input, and the objective is to check whether there exists a vertex cover of size at most k .

The first reduction rule is based on the following simple observation. For a given instance (G, k) of VERTEX COVER, if the graph G has an isolated vertex, then this vertex does not cover any edge and thus its removal does not change the solution. This shows that the following rule is safe.

Reduction VC.1. If G contains an isolated vertex v , delete v from G . The new instance is $(G - v, k)$.

The second rule is based on the following natural observation:

If G contains a vertex v of degree more than k , then v should be in every vertex cover of size at most k .

Indeed, this is because if v is not in a vertex cover, then we need at least $k + 1$ vertices to cover edges incident to v . Thus our second rule is the following.

Reduction VC.2. If there is a vertex v of degree at least $k + 1$, then delete v (and its incident edges) from G and decrement the parameter k by 1. The new instance is $(G - v, k - 1)$.

Observe that exhaustive application of reductions VC.1 and VC.2 completely removes the vertices of degree 0 and degree at least $k + 1$. The next step is the following observation.

If a graph has maximum degree d , then a set of k vertices can cover at most kd edges.

This leads us to the following lemma.

Lemma 2.3. *If (G, k) is a yes-instance and none of the reduction rules VC.1, VC.2 is applicable to G , then $|V(G)| \leq k^2 + k$ and $|E(G)| \leq k^2$.*

Proof. Because we cannot apply Reductions VC.1 anymore on G , G has no isolated vertices. Thus for every vertex cover S of G , every vertex of $G - S$ should be adjacent to some vertex from S . Since we cannot apply Reductions VC.2, every vertex of G has degree at most k . It follows that

$|V(G - S)| \leq k|S|$ and hence $|V(G)| \leq (k + 1)|S|$. Since (G, k) is a yes-instance, there is a vertex cover S of size at most k , so $|V(G)| \leq (k + 1)k$. Also every edge of G is covered by some vertex from a vertex cover and every vertex can cover at most k edges. Hence if G has more than k^2 edges, this is again a no-instance. $\square$

Lemma 2.3 allows us to claim the final reduction rule that explicitly bounds the size of the kernel.

Reduction VC.3. Let (G, k) be an input instance such that Reductions VC.1 and VC.2 are not applicable to (G, k) . If $k < 0$ and G has more than $k^2 + k$ vertices, or more than k^2 edges, then conclude that we are dealing with a no-instance.

Finally, we remark that all reduction rules are trivially applicable in linear time. Thus, we obtain the following theorem.

Theorem 2.4. VERTEX COVER admits a kernel with $\mathcal{O}(k^2)$ vertices and $\mathcal{O}(k^2)$ edges.

2.2.2 FEEDBACK ARC SET IN TOURNAMENTS

In this section we discuss a kernel for the FEEDBACK ARC SET IN TOURNAMENTS problem. A *tournament* is a directed graph T such that for every pair of vertices $u, v \in V(T)$, exactly one of (u, v) or (v, u) is a directed edge (also often called an *arc*) of T . A set of edges A of a directed graph G is called a *feedback arc set* if every directed cycle of G contains an edge from A . In other words, the removal of A from G turns it into a directed acyclic graph. Very often, acyclic tournaments are called *transitive* (note that then $E(G)$ is a transitive relation). In the FEEDBACK ARC SET IN TOURNAMENTS problem we are given a tournament T and a nonnegative integer k . The objective is to decide whether T has a feedback arc set of size at most k .

For tournaments, the deletion of edges results in directed graphs which are not tournaments anymore. Because of that, it is much more convenient to use the characterization of a feedback arc set in terms of “reversing edges”. We start with the following well-known result about *topological orderings* of directed acyclic graphs.

Lemma 2.5. A directed graph G is acyclic if and only if it is possible to order its vertices in such a way such that for every directed edge (u, v) , we have $u < v$.

We leave the proof of Lemma 2.5 as an exercise; see Exercise 2.1. Given a directed graph G and a subset $F \subseteq E(G)$ of edges, we define $G \circledast F$ to be the directed graph obtained from G by reversing all the edges of F . That is, if $\text{rev}(F) = \{(u, v) : (v, u) \in F\}$, then for $G \circledast F$ the vertex set is $V(G)$

and the edge set $E(G \circledast F) = (E(G) \cup \text{rev}(F)) \setminus F$. Lemma 2.5 implies the following.

Observation 2.6. Let G be a directed graph and let F be a subset of edges of G . If $G \circledast F$ is a directed acyclic graph then F is a feedback arc set of G .

The following lemma shows that, in some sense, the opposite direction of the statement in Observation 2.6 is also true. However, the minimality condition in Lemma 2.7 is essential, see Exercise 2.2.

Lemma 2.7. *Let G be a directed graph and F be a subset of $E(G)$. Then F is an inclusion-wise minimal feedback arc set of G if and only if F is an inclusion-wise minimal set of edges such that $G \circledast F$ is an acyclic directed graph.*

Proof. We first prove the forward direction of the lemma. Let F be an inclusion-wise minimal feedback arc set of G . Assume to the contrary that $G \circledast F$ has a directed cycle C . Then C cannot contain only edges of $E(G) \setminus F$, as that would contradict the fact that F is a feedback arc set. Let $f_1, f_2, \dots, f_\ell$ be the edges of $C \cap \text{rev}(F)$ in the order of their appearance on the cycle C , and let $e_i \in F$ be the edge f_i reversed. Since F is inclusion-wise minimal, for every e_i , there exists a directed cycle C_i in G such that $F \cap C_i = \{e_i\}$. Now consider the following closed walk W in G : we follow the cycle C , but whenever we are to traverse an edge $f_i \in \text{rev}(F)$ (which is not present in G), we instead traverse the path $C_i - e_i$. By definition, W is a closed walk in G and, furthermore, note that W does not contain any edge of F . This contradicts the fact that F is a feedback arc set of G .

The minimality follows from Observation 2.6. That is, every set of edges F such that $G \circledast F$ is acyclic is also a feedback arc set of G , and thus, if F is not a minimal set such that $G \circledast F$ is acyclic, then it will contradict the fact that F is a minimal feedback arc set.

For the other direction, let F be an inclusion-wise minimal set of edges such that $G \circledast F$ is an acyclic directed graph. By Observation 2.6, F is a feedback arc set of G . Moreover, F is an inclusion-wise minimal feedback arc set, because if a proper subset F' of F is an inclusion-wise minimal feedback arc set of G , then by the already proved implication of the lemma, $G \circledast F'$ is an acyclic directed graph, a contradiction with the minimality of F . $\square$

We are ready to give a kernel for FEEDBACK ARC SET IN TOURNAMENTS.

Theorem 2.8. FEEDBACK ARC SET IN TOURNAMENTS *admits a kernel with at most $k^2 + 2k$ vertices.*

Proof. Lemma 2.7 implies that a tournament T has a feedback arc set of size at most k if and only if it can be turned into an acyclic tournament by reversing directions of at most k edges. We will use this characterization for the kernel.

In what follows by a *triangle* we mean a directed cycle of length three. We give two simple reduction rules.

Reduction FAST.1. If an edge e is contained in at least $k + 1$ triangles, then reverse e and reduce k by 1.

Reduction FAST.2. If a vertex v is not contained in any triangle, then delete v from T .

The rules follow similar guidelines as in the case of VERTEX COVER. In Reduction FAST.1, we greedily take into a solution an edge that participates in $k + 1$ otherwise disjoint forbidden structures (here, triangles). In Reduction FAST.2, we discard vertices that do not participate in any forbidden structure, and should be irrelevant to the problem.

However, a formal proof of the safeness of Reduction FAST.2 is not immediate: we need to verify that deleting v and its incident edges does not make a yes-instance out of a no-instance.

Note that after applying any of the two rules, the resulting graph is again a tournament. The first rule is safe because if we do not reverse e , we have to reverse at least one edge from each of $k + 1$ triangles containing e . Thus e belongs to every feedback arc set of size at most k .

Let us now prove the safeness of the second rule. Let $X = N^+(v)$ be the set of heads of directed edges with tail v and let $Y = N^-(v)$ be the set of tails of directed edges with head v . Because T is a tournament, X and Y is a partition of $V(T) \setminus \{v\}$. Since v is not a part of any triangle in T , we have that there is no edge from X to Y (with head in Y and tail in X). Consequently, for any feedback arc set A_1 of tournament $T[X]$ and any feedback arc set A_2 of tournament $T[Y]$, the set $A_1 \cup A_2$ is a feedback arc set of T . As the reverse implication is trivial (for any feedback arc set A in T , $A \cap E(T[X])$ is a feedback arc set of $T[X]$, and $A \cap E(T[Y])$ is a feedback arc set of $T[Y]$), we have that (T, k) is a yes-instance if and only if $(T - v, k)$ is.

Finally, we show that every reduced yes-instance T , an instance on which none of the presented reduction rules are applicable, has at most $k(k + 2)$ vertices. Let A be a feedback arc set of a reduced instance T of size at most k . For every edge $e \in A$, aside from the two endpoints of e , there are at most k vertices that are in triangles containing e — otherwise we would be able to apply Reduction FAST.1. Since every triangle in T contains an edge of A and every vertex of T is in a triangle, we have that T has at most $k(k + 2)$ vertices.

Thus, given (T, k) we apply our reduction rules exhaustively and obtain an equivalent instance (T', k') . If T' has more than $k'^2 + k'$ vertices, then the algorithm returns that (T, k) is a no-instance, otherwise we get the desired kernel. This completes the proof of the theorem. $\square$

2.2.3 EDGE CLIQUE COVER

Not all FPT problems admit polynomial kernels. In the EDGE CLIQUE COVER problem, we are given a graph G and a nonnegative integer k , and the goal is to decide whether the edges of G can be covered by at most k cliques. In this section we give an exponential kernel for EDGE CLIQUE COVER. In Theorem 14.20 of Section *14.3.3, we remark that this simple kernel is essentially optimal.

Let us recall the reader that we use $N(v) = \{u : uv \in E(G)\}$ to denote the neighborhood of vertex v in G , and $N[v] = N(v) \cup \{v\}$ to denote the closed neighborhood of v . We apply the following data reduction rules in the given order (i.e., we always use the lowest-numbered rule that modifies the instance).

Reduction ECC.1. Remove isolated vertices.

Reduction ECC.2. If there is an isolated edge uv (a connected component that is just an edge), delete it and decrease k by 1. The new instance is $(G - \{u, v\}, k - 1)$.

Reduction ECC.3. If there is an edge uv whose endpoints have exactly the same closed neighborhood, that is, $N[u] = N[v]$, then delete v . The new instance is $(G - v, k)$.

The crux of the presented kernel for EDGE CLIQUE COVER is an observation that two true twins (vertices u and v with $N[u] = N[v]$) can be treated in exactly the same way in some optimum solution, and hence we can reduce them. Meanwhile, the vertices that are contained in exactly the same set of cliques in a feasible solution *have to* be true twins. This observation bounds the size of the kernel.

The safeness of the first two reductions is trivial, while the safeness of Reduction ECC.3 follows from the observation that a solution in $G - v$ can be extended to a solution in G by adding v to all the cliques containing u (see Exercise 2.3).

Theorem 2.9. EDGE CLIQUE COVER admits a kernel with at most 2^k vertices.

Proof. We start with the following claim.

Claim. If (G, k) is a reduced yes-instance, on which none of the presented reduction rules can be applied, then $|V(G)| \leq 2^k$.

Proof. Let $C_1, \dots, C_k$ be an edge clique cover of G . We claim that G has at most 2^k vertices. Targeting a contradiction, let us assume that G has more

than 2^k vertices. We assign to each vertex $v \in V(G)$ a binary vector b_v of length k , where bit i , $1 \leq i \leq k$, is set to 1 if and only if v is contained in clique C_i . Since there are only 2^k possible vectors, there must be $u \neq v \in V(G)$ with $b_u = b_v$. If b_u and b_v are zero vectors, the first rule applies; otherwise, u and v are contained in the same cliques. This means that u and v are adjacent and have the same neighborhood; thus either Reduction ECC.2 or Reduction ECC.3 applies. Hence, if G has more than 2^k vertices, at least one of the reduction rules can be applied to it, which is a contradiction to the initial assumption that G is reduced. This completes the proof of the claim. $\square$

The kernelization algorithm works as follows. Given an instance (G, k) , it applies Reductions ECC.1, ECC.2, and ECC.3 exhaustively. If the resulting graph has more than 2^k vertices the kernelization algorithm outputs that the input instance is a no-instance, else it outputs the reduced instance. $\square$

2.3 Crown decomposition

Crown decomposition is a general kernelization technique that can be used to obtain kernels for many problems. The technique is based on the classical matching theorems of Kőnig and Hall.

Recall that for disjoint vertex subsets U, W of a graph G , a matching M is called a *matching of U into W* if every edge of M connects a vertex of U and a vertex of W and, moreover, every vertex of U is an endpoint of some edge of M . In this situation, we also say that M *saturates U* .

Definition 2.10 (Crown decomposition). A *crown decomposition* of a graph G is a partitioning of $V(G)$ into three parts C , H and R , such that

1. C is nonempty.
2. C is an independent set.
3. There are no edges between vertices of C and R . That is, H separates C and R .
4. Let E' be the set of edges between vertices of C and H . Then E' contains a matching of size $|H|$. In other words, G contains a matching of H into C .

The set C can be seen as a crown put on head H of the remaining part R , see Fig. 2.1. Note that the fact that E' contains a matching of size $|H|$ implies that there is a matching of H into C . This is a matching in the subgraph G' , with the vertex set $C \cup H$ and the edge set E' , saturating all the vertices of H .

For finding a crown decomposition in polynomial time, we use the following well known structural and algorithmic results. The first is a mini-max theorem due to Kőnig.

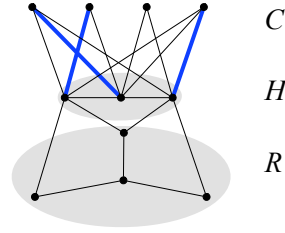


Fig. 2.1: Example of a crown decomposition. Set C is an independent set, H separates C and R , and there is a matching of H into C

Theorem 2.11 (König's theorem, [303]). *In every undirected bipartite graph the size of a maximum matching is equal to the size of a minimum vertex cover.*

Let us recall that a matching *saturates* a set of vertices S when every vertex in S is incident to an edge in the matching. The second classic result states that in bipartite graphs, a trivial necessary condition for the existence of a matching is also sufficient.

Theorem 2.12 (Hall's theorem, [256]). *Let G be an undirected bipartite graph with bipartition (V_1, V_2) . The graph G has a matching saturating V_1 if and only if for all $X \subseteq V_1$, we have $|N(X)| \geq |X|$.*

The following theorem is due to Hopcroft and Karp [268]. The proof of the (nonstandard) second claim of the theorem is deferred to Exercise 2.21.

Theorem 2.13 (Hopcroft-Karp algorithm, [268]). *Let G be an undirected bipartite graph with bipartition V_1 and V_2 , on n vertices and m edges. Then we can find a maximum matching as well as a minimum vertex cover of G in time $\mathcal{O}(m\sqrt{n})$. Furthermore, in time $\mathcal{O}(m\sqrt{n})$ either we can find a matching saturating V_1 or an inclusion-wise minimal set $X \subseteq V_1$ such that $|N(X)| < |X|$.*

The following lemma is the basis for kernelization algorithms using crown decomposition.

Lemma 2.14 (Crown lemma). *Let G be a graph without isolated vertices and with at least $3k + 1$ vertices. There is a polynomial-time algorithm that either*

- *finds a matching of size $k + 1$ in G ; or*
- *finds a crown decomposition of G .*

Proof. We first find an inclusion-maximal matching M in G . This can be done by a greedy algorithm. If the size of M is $k + 1$, then we are done.

Hence, we assume that $|M| \leq k$, and let V_M be the endpoints of M . We have $|V_M| \leq 2k$. Because M is a maximal matching, the remaining set of vertices $I = V(G) \setminus V_M$ is an independent set.

Consider the bipartite graph G_{I, V_M} formed by edges of G between V_M and I . We compute a minimum-sized vertex cover X and a maximum sized matching M' of the bipartite graph G_{I, V_M} in polynomial time using Theorem 2.13. We can assume that $|M'| \leq k$, for otherwise we are done. Since $|X| = |M'|$ by König's theorem (Theorem 2.11), we infer that $|X| \leq k$.

If no vertex of X is in V_M , then $X \subseteq I$. We claim that $X = I$. For a contradiction assume that there is a vertex $w \in I \setminus X$. Because G has no isolated vertices there is an edge, say wz , incident to w in G_{I, V_M} . Since G_{I, V_M} is bipartite, we have that $z \in V_M$. However, X is a vertex cover of G_{I, V_M} such that $X \cap V_M = \emptyset$, which implies that $w \in X$. This is contrary to our assumption that $w \notin X$, thus proving that $X = I$. But then $|I| \leq |X| \leq k$, and G has at most

$$|I| + |V_M| \leq k + 2k = 3k$$

vertices, which is a contradiction.

Hence, $X \cap V_M \neq \emptyset$. We obtain a crown decomposition (C, H, R) as follows. Since $|X| = |M'|$, every edge of the matching M' has exactly one endpoint in X . Let M^* denote the subset of M' such that every edge from M^* has exactly one endpoint in $X \cap V_M$ and let V_{M^*} denote the set of endpoints of edges in M^* . We define head $H = X \cap V_M = X \cap V_{M^*}$, crown $C = V_{M^*} \cap I$, and the remaining part $R = V(G) \setminus (C \cup H) = V(G) \setminus V_{M^*}$. In other words, H is the set of endpoints of edges of M^* that are present in V_M and C is the set of endpoints of edges of M^* that are present in I . Obviously, C is an independent set and by construction, M^* is a matching of H into C . Furthermore, since X is a vertex cover of G_{I, V_M} , every vertex of C can be adjacent only to vertices of H and thus H separates C and R . This completes the proof. $\square$

The crown lemma gives a relatively strong structural property of graphs with a small vertex cover (equivalently, a small maximum matching). If in a studied problem the parameter upper bounds the size of a vertex cover (maximum matching), then there is a big chance that the structural insight given by the crown lemma would help in developing a small kernel — quite often with number of vertices bounded linearly in the parameter.

We demonstrate the application of crown decompositions on kernelization for VERTEX COVER and MAXIMUM SATISFIABILITY.

2.3.1 VERTEX COVER

Consider a VERTEX COVER instance (G, k) . By an exhaustive application of Reduction VC.1, we may assume that G has no isolated vertices. If $|V(G)| > 3k$, we may apply the crown lemma to the graph G and integer k , obtaining either a matching of size $k + 1$, or a crown decomposition $V(G) = C \cup H \cup R$. In the first case, the algorithm concludes that (G, k) is a no-instance.

In the latter case, let M be a matching of H into C . Observe that the matching M witnesses that, for every vertex cover X of the graph G , X contains at least $|M| = |H|$ vertices of $H \cup C$ to cover the edges of M . On the other hand, the set H covers all edges of G that are incident to $H \cup C$. Consequently, there exists a minimum vertex cover of G that contains H , and we may reduce (G, k) to $(G - H, k - |H|)$. Note that in the instance $(G - H, k - |H|)$, the vertices of C are isolated and will be reduced by Reduction VC.1.

As the crown lemma promises us that $H \neq \emptyset$, we can always reduce the graph as long as $|V(G)| > 3k$. Thus, we obtain the following.

Theorem 2.15. VERTEX COVER admits a kernel with at most $3k$ vertices.

2.3.2 MAXIMUM SATISFIABILITY

For a second application of the crown decomposition, we look at the following parameterized version of MAXIMUM SATISFIABILITY. Given a CNF formula F , and a nonnegative integer k , decide whether F has a truth assignment satisfying at least k clauses.

Theorem 2.16. MAXIMUM SATISFIABILITY admits a kernel with at most k variables and $2k$ clauses.

Proof. Let φ be a CNF formula with n variables and m clauses. Let ψ be an arbitrary assignment to the variables and let $\neg\psi$ be the assignment obtained by complementing the assignment of ψ . That is, if ψ assigns $\delta \in \{\top, \perp\}$ to some variable x then $\neg\psi$ assigns $\neg\delta$ to x . Observe that either ψ or $\neg\psi$ satisfies at least $m/2$ clauses, since every clause is satisfied by ψ or $\neg\psi$ (or by both). This means that, if $m \geq 2k$, then (φ, k) is a yes-instance. In what follows we give a kernel with $n < k$ variables.

Let G_φ be the *variable-clause* incidence graph of φ . That is, G_φ is a bipartite graph with bipartition (X, Y) , where X is the set of the variables of φ and Y is the set of clauses of φ . In G_φ there is an edge between a variable $x \in X$ and a clause $c \in Y$ if and only if either x , or its negation, is in c . If there is a matching of X into Y in G_φ , then there is a truth assignment satisfying at least $|X|$ clauses: we can set each variable in X in such a way that the clause matched to it becomes satisfied. Thus at least $|X|$ clauses are satisfied. Hence, in this case, if $k \leq |X|$, then (φ, k) is a yes-instance. Otherwise,

$k > |X| = n$, and we get the desired kernel. We now show that, if φ has at least $n \geq k$ variables, then we can, in polynomial time, either reduce φ to an equivalent smaller instance, or find an assignment to the variables satisfying at least k clauses (and conclude that we are dealing with a yes-instance).

Suppose φ has at least k variables. Using Hall's theorem and a polynomial-time algorithm computing a maximum-size matching (Theorems 2.12 and 2.13), we can in polynomial time find either a matching of X into Y or an inclusion-wise minimal set $C \subseteq X$ such that $|N(C)| < |C|$. As discussed in the previous paragraph, if we found a matching, then the instance is a yes-instance and we are done. So suppose we found a set C as described. Let H be $N(C)$ and $R = V(G_\varphi) \setminus (C \cup H)$. Clearly, $N(C) \subseteq H$, there are no edges between vertices of C and R and $G[C]$ is an independent set. Select an arbitrary $x \in C$. We have that there is a matching of $C \setminus \{x\}$ into H since $|N(C')| \geq |C'|$ for every $C' \subseteq C \setminus \{x\}$. Since $|C| > |H|$, we have that the matching from $C \setminus \{x\}$ to H is in fact a matching of H into C . Hence (C, H, R) is a crown decomposition of G_φ .

We prove that all clauses in H are satisfied in every assignment satisfying the maximum number of clauses. Indeed, consider any assignment ψ that does not satisfy all clauses in H . Fix any variable $x \in C$. For every variable y in $C \setminus \{x\}$ set the value of y so that the clause in H matched to y is satisfied. Let ψ' be the new assignment obtained from ψ in this manner. Since $N(C) \subseteq H$ and ψ' satisfies all clauses in H , more clauses are satisfied by ψ' than by ψ . Hence ψ cannot be an assignment satisfying the maximum number of clauses.

The argument above shows that (φ, k) is a yes-instance to MAXIMUM SATISFIABILITY if and only if $(\varphi \setminus H, k - |H|)$ is. This gives rise to the following simple reduction.

Reduction MSat.1. Let (φ, k) and H be as above. Then remove H from φ and decrease k by $|H|$. That is, $(\varphi \setminus H, k - |H|)$ is the new instance.

Repeated applications of Reduction MSat.1 and the arguments described above give the desired kernel. This completes the proof of the theorem. $\square$

2.4 Expansion lemma

In the previous subsection, we described crown decomposition techniques based on the classical Hall's theorem. In this section, we introduce a powerful variation of Hall's theorem, which is called the expansion lemma. This lemma captures a certain property of neighborhood sets in graphs and can be used to obtain polynomial kernels for many graph problems. We apply this result to get an $\mathcal{O}(k^2)$ kernel for FEEDBACK VERTEX SET in Chapter 9.

A q -star, $q \geq 1$, is a graph with $q+1$ vertices, one vertex of degree q , called the *center*, and all other vertices of degree 1 adjacent to the center. Let G be

a bipartite graph with vertex bipartition (A, B) . For a positive integer q , a set of edges $M \subseteq E(G)$ is called by a *q -expansion of A into B* if

- every vertex of A is incident to exactly q edges of M ;
- M saturates exactly $q|A|$ vertices in B .

Let us emphasize that a q -expansion saturates all vertices of A . Also, for every $u, v \in A$, $u \neq v$, the set of vertices E_u adjacent to u by edges of M does not intersect the set of vertices E_v adjacent to v via edges of M , see Fig. 2.2. Thus every vertex $v \in A$ could be thought of as the center of a star with its q leaves in B , with all these $|A|$ stars being vertex-disjoint. Furthermore, a collection of these stars is also a family of q edge-disjoint matchings, each saturating A .

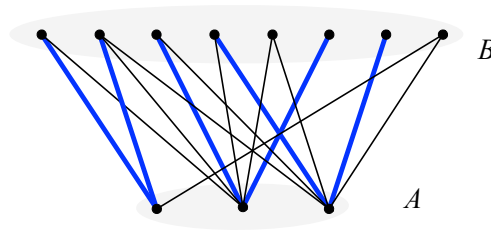


Fig. 2.2: Set A has a 2-expansion into B

Let us recall that, by Hall's theorem (Theorem 2.12), a bipartite graph with bipartition (A, B) has a matching of A into B if and only if $|N(X)| \geq |X|$ for all $X \subseteq A$. The following lemma is an extension of this result.

Lemma 2.17. *Let G be a bipartite graph with bipartition (A, B) . Then there is a q -expansion from A into B if and only if $|N(X)| \geq q|X|$ for every $X \subseteq A$. Furthermore, if there is no q -expansion from A into B , then a set $X \subseteq A$ with $|N(X)| < q|X|$ can be found in polynomial time.*

Proof. If A has a q -expansion into B , then trivially $|N(X)| \geq q|X|$ for every $X \subseteq A$.

For the opposite direction, we construct a new bipartite graph G' with bipartition (A', B) from G by adding $(q - 1)$ copies of all the vertices in A . For every vertex $v \in A$ all copies of v have the same neighborhood in B as v . We would like to prove that there is a matching M from A' into B in G' . If we prove this, then by identifying the endpoints of M corresponding to the copies of vertices from A , we obtain a q -expansion in G . It suffices to check that the assumptions of Hall's theorem are satisfied in G' . Assume otherwise, that there is a set $X \subseteq A'$ such that $|N_{G'}(X)| < |X|$. Without loss of generality, we can assume that if X contains some copy of a vertex v , then it contains all the copies of v , since including all the remaining copies increases $|X|$ but

does not change $|N_{G'}(X)|$. Hence, the set X in A' naturally corresponds to the set X_A of size $|X|/q$ in A , the set of vertices whose copies are in X . But then $|N_G(X_A)| = |N_{G'}(X)| < |X| = q|X_A|$, which is a contradiction. Hence A' has a matching into B and thus A has a q -expansion into B .

For the algorithmic claim, note that, if there is no q -expansion from A into B , then we can use Theorem 2.13 to find a set $X \subseteq A'$ such that $|N_{G'}(X)| < |X|$, and the corresponding set X_A satisfies $|N_G(X_A)| < q|X_A|$. $\square$

Finally, we are ready to prove a lemma analogous to Lemma 2.14.

Lemma 2.18. (Expansion lemma) *Let $q \geq 1$ be a positive integer and G be a bipartite graph with vertex bipartition (A, B) such that*

- (i) $|B| \geq q|A|$, and
- (ii) *there are no isolated vertices in B .*

Then there exist nonempty vertex sets $X \subseteq A$ and $Y \subseteq B$ such that

- *there is a q -expansion of X into Y , and*
- *no vertex in Y has a neighbor outside X , that is, $N(Y) \subseteq X$.*

Furthermore, the sets X and Y can be found in time polynomial in the size of G .

Note that the sets X , Y and $V(G) \setminus (X \cup Y)$ form a crown decomposition of G with a stronger property — every vertex of X is not only matched into Y , but there is a q -expansion of X into Y . We proceed with the proof of expansion lemma.

Proof. We proceed recursively, at every step decreasing the cardinality of A . When $|A| = 1$, the claim holds trivially by taking $X = A$ and $Y = B$.

We apply Lemma 2.17 to G . If A has a q -expansion into B , then we are done as we may again take $X = A$ and $Y = B$. Otherwise, we can in polynomial time find a (nonempty) set $Z \subseteq A$ such that $|N(Z)| < q|Z|$. We construct the graph G' by removing Z and $N(Z)$ from G . We claim that G' satisfies the assumptions of the lemma. Indeed, because we removed less than q times more vertices from B than from A , we have that (i) holds for G' . Moreover, every vertex from $B \setminus N(Z)$ has no neighbor in Z , and thus (ii) also holds for G' . Note that $Z \neq A$, because otherwise $N(A) = B$ (there are no isolated vertices in B) and $|B| \geq q|A|$. Hence, we recurse on the graph G' with bipartition $(A \setminus Z, B \setminus N(Z))$, obtaining nonempty sets $X \subseteq A \setminus Z$ and $Y \subseteq B \setminus N(Z)$ such that there is a q -expansion of X into Y and such that $N_{G'}(Y) \subseteq X$. Because $Y \subseteq B \setminus N(Z)$, we have that no vertex in Y has a neighbor in Z . Hence, $N_{G'}(Y) = N_G(Y) \subseteq X$ and the pair (X, Y) satisfies all the required properties. $\square$

The expansion lemma is useful when the matching saturating the head part H in the crown lemma turns out to be not sufficient for a reduction, and we would like to have a few vertices of the crown C matched to a single vertex of the head H . For example, this is the case for the FEEDBACK VERTEX SET kernel presented in Section 9.1, where we need the case $q = 2$.

2.5 Kernels based on linear programming

In this section we design a $2k$ -vertex kernel for VERTEX COVER exploiting the solution to a linear programming formulation of VERTEX COVER.

Many combinatorial problems can be expressed in the language of INTEGER LINEAR PROGRAMMING (ILP). In an INTEGER LINEAR PROGRAMMING instance, we are given a set of integer-valued variables, a set of linear inequalities (called *constraints*) and a linear *cost function*. The goal is to find an (integer) evaluation of the variables that satisfies all constraints, and minimizes or maximizes the value of the cost function.

Let us give an example on how to encode a VERTEX COVER instance (G, k) as an INTEGER LINEAR PROGRAMMING instance. We introduce $n = |V(G)|$ variables, one variable x_v for each vertex $v \in V(G)$. Setting variable x_v to 1 means that v is in the vertex cover, while setting x_v to 0 means that v is not in the vertex cover. To ensure that every edge is covered, we can introduce constraints $x_u + x_v \geq 1$ for every edge $uv \in E(G)$. The size of the vertex cover is given by $\sum_{v \in V(G)} x_v$. In the end, we obtain the following ILP formulation:

$$\begin{aligned} & \text{minimize} && \sum_{v \in V(G)} x_v \\ & \text{subject to} && x_u + x_v \geq 1 \quad \text{for every } uv \in E(G), \\ & && 0 \leq x_v \leq 1 \quad \text{for every } v \in V(G), \\ & && x_v \in \mathbb{Z} \quad \text{for every } v \in V(G). \end{aligned} \tag{2.1}$$

Clearly, the optimal value of (2.1) is at most k if and only if G has a vertex cover of size at most k .

As we have just seen, INTEGER LINEAR PROGRAMMING is at least as hard as VERTEX COVER, so we do not expect it to be polynomial-time solvable. In fact, it is relatively easy to express many NP-hard problems in the language of INTEGER LINEAR PROGRAMMING. In Section 6.2 we discuss FPT algorithms for INTEGER LINEAR PROGRAMMING and their application in proving fixed-parameter tractability of other problems.

Here, we proceed in a different way: we relax the integrality requirement of INTEGER LINEAR PROGRAMMING, which is the main source of the hardness of this problem, to obtain LINEAR PROGRAMMING. That is, in LINEAR

PROGRAMMING the instance looks exactly the same as in INTEGER LINEAR PROGRAMMING, but the variables are allowed to take arbitrary real values, instead of just integers.

In the case of VERTEX COVER, we relax (2.1) by dropping the constraint $x_v \in \mathbb{Z}$ for every $v \in V(G)$. In other words, we obtain the following LINEAR PROGRAMMING instance. For a graph G , we call this relaxation LPVC(G).

$$\begin{aligned} & \text{minimize} && \sum_{v \in V(G)} x_v \\ & \text{subject to} && x_u + x_v \geq 1 \quad \text{for every } uv \in E(G), \\ & && 0 \leq x_v \leq 1 \quad \text{for every } v \in V(G). \end{aligned} \tag{2.2}$$

Note that constraints $x_v \leq 1$ can be omitted because every optimal solution of LPVC(G) satisfies these constraints.

Observe that in LPVC(G), a variable x_v can take fractional values in the interval $[0, 1]$, which corresponds to taking “part of the vertex v ” into a vertex cover. Consider an example of G being a triangle. A minimum vertex cover of a triangle is of size 2, whereas in LPVC(G) we can take $x_v = \frac{1}{2}$ for every $v \in V(G)$, obtaining a feasible solution of cost $\frac{3}{2}$. Thus, LPVC(G) does not express exactly the VERTEX COVER problem on graph G , but its optimum solution can still be useful to learn something about minimum vertex covers in G .

The main source of utility of LINEAR PROGRAMMING comes from the fact that LINEAR PROGRAMMING can be solved in polynomial time, even in some general cases where there are exponentially many constraints, accessed through an oracle. For this reason, LINEAR PROGRAMMING has found abundant applications in approximation algorithms (for more on this topic, we refer to the book of Vazirani [427]). In this section, we use LP to design a small kernel for VERTEX COVER. In Section 3.4, we will use LPVC(G) to obtain an FPT branching algorithm for VERTEX COVER.

Let us now have a closer look at the relaxation LPVC(G). Fix an optimal solution $(x_v)_{v \in V(G)}$ of LPVC(G). In this solution the variables corresponding to vertices of G take values in the interval $[0, 1]$. We partition $V(G)$ according to these values into three sets as follows.

- $V_0 = \{v \in V(G) : x_v < \frac{1}{2}\},$
- $V_{\frac{1}{2}} = \{v \in V(G) : x_v = \frac{1}{2}\},$
- $V_1 = \{v \in V(G) : x_v > \frac{1}{2}\}.$

Theorem 2.19 (Nemhauser-Trotter theorem). *There is a minimum vertex cover S of G such that*

$$V_1 \subseteq S \subseteq V_1 \cup V_{\frac{1}{2}}.$$

Proof. Let $S^* \subseteq V(G)$ be a minimum vertex cover of G . Define

$$S = (S^* \setminus V_0) \cup V_1.$$

By the constraints of (2.2), every vertex of V_0 can have a neighbor only in V_1 and thus S is also a vertex cover of G . Moreover, $V_1 \subseteq S \subseteq V_1 \cup V_{\frac{1}{2}}$. It suffices to show that S is a *minimum* vertex cover. Assume the contrary, i.e., $|S| > |S^*|$. Since $|S| = |S^*| - |V_0 \cap S^*| + |V_1 \setminus S^*|$ we infer that

$$|V_0 \cap S^*| < |V_1 \setminus S^*|. \quad (2.3)$$

Let us define

$$\varepsilon = \min\{|x_v - \frac{1}{2}| : v \in V_0 \cup V_1\}.$$

We decrease the fractional values of vertices from $V_1 \setminus S^*$ by ε and increase the values of vertices from $V_0 \cap S^*$ by ε . In other words, we define a vector $(y_v)_{v \in V(G)}$ as

$$y_v = \begin{cases} x_v - \varepsilon & \text{if } v \in V_1 \setminus S^*, \\ x_v + \varepsilon & \text{if } v \in V_0 \cap S^*, \\ x_v & \text{otherwise.} \end{cases}$$

Note that $\varepsilon > 0$, because otherwise $V_0 = V_1 = \emptyset$, a contradiction with (2.3). This, together with (2.3), implies that

$$\sum_{v \in V(G)} y_v < \sum_{v \in V(G)} x_v. \quad (2.4)$$

Now we show that $(y_v)_{v \in V(G)}$ is a feasible solution, i.e., it satisfies the constraints of $\text{LPVC}(G)$. Since $(x_v)_{v \in V(G)}$ is a feasible solution, by the definition of ε we get $0 \leq y_v \leq 1$ for every $v \in V(G)$. Consider an arbitrary edge $uv \in E(G)$. If none of the endpoints of uv belong to $V_1 \setminus S^*$, then both $y_u \geq x_u$ and $y_v \geq x_v$, so $y_u + y_v \geq x_u + x_v \geq 1$. Otherwise, by symmetry we can assume that $u \in V_1 \setminus S^*$, and hence $y_u = x_u - \varepsilon$. Because S^* is a vertex cover, we have that $v \in S^*$. If $v \in V_0 \cap S^*$, then

$$y_u + y_v = x_u - \varepsilon + x_v + \varepsilon = x_u + x_v \geq 1.$$

Otherwise, $v \in (V_{\frac{1}{2}} \cup V_1) \cap S^*$. Then $y_v \geq x_v \geq \frac{1}{2}$. Note also that $x_u - \varepsilon \geq \frac{1}{2}$ by the definition of ε . It follows that

$$y_u + y_v = x_u - \varepsilon + y_v \geq \frac{1}{2} + \frac{1}{2} = 1.$$

Thus $(y_v)_{v \in V(G)}$ is a feasible solution of $\text{LPVC}(G)$ and hence (2.4) contradicts the optimality of $(x_v)_{v \in V(G)}$. $\square$

Theorem 2.19 allows us to use the following reduction rule.

Reduction VC.4. Let $(x_v)_{v \in V(G)}$ be an optimum solution to $\text{LPVC}(G)$ in a VERTEX COVER instance (G, k) and let V_0 , V_1 and $V_{\frac{1}{2}}$ be defined as above. If $\sum_{v \in V(G)} x_v > k$, then conclude that we are dealing with a no-instance. Otherwise, greedily take into the vertex cover the vertices of V_1 . That is, delete all vertices of $V_0 \cup V_1$, and decrease k by $|V_1|$.

Let us now formally verify the safeness of Reduction VC.4.

Lemma 2.20. *Reduction VC.4 is safe.*

Proof. Clearly, if (G, k) is a yes-instance, then an optimum solution to $\text{LPVC}(G)$ is of cost at most k . This proves the correctness of the step if we conclude that (G, k) is a no-instance.

Let $G' = G - (V_0 \cup V_1) = G[V_{\frac{1}{2}}]$ and $k' = k - |V_1|$. We claim that (G, k) is a yes-instance of VERTEX COVER if and only if (G', k') is. By Theorem 2.19, we know that G has a vertex cover S of size at most k such that $V_1 \subseteq S \subseteq V_1 \cup V_{\frac{1}{2}}$. Then $S' = S \cap V_{\frac{1}{2}}$ is a vertex cover in G' and the size of S' is at most $k - |V_1| = k'$.

For the opposite direction, let S' be a vertex cover in G' . For every solution of $\text{LPVC}(G)$, every edge with an endpoint from V_0 should have an endpoint in V_1 . Hence, $S = S' \cup V_1$ is a vertex cover in G and the size of this vertex cover is at most $k' + |V_1| = k$. $\square$

Reduction VC.4 leads to the following kernel for VERTEX COVER .

Theorem 2.21. *VERTEX COVER admits a kernel with at most $2k$ vertices.*

Proof. Let (G, k) be an instance of VERTEX COVER . We solve $\text{LPVC}(G)$ in polynomial time, and apply Reduction VC.4 to the obtained solution $(x_v)_{v \in V(G)}$, either concluding that we are dealing with a no-instance or obtaining an instance (G', k') . Lemma 2.20 guarantees the safeness of the reduction. For the size bound, observe that

$$|V(G')| = |V_{\frac{1}{2}}| = \sum_{v \in V_{\frac{1}{2}}} 2x_v \leq 2 \sum_{v \in V(G)} x_v \leq 2k.$$

$\square$

While it is possible to solve linear programs in polynomial time, usually such solutions are less efficient than combinatorial algorithms. The specific structure of the LP-relaxation of the vertex cover problem (2.2) allows us to solve it by reducing to the problem of finding a maximum-size matching in a bipartite graph.

Lemma 2.22. *For a graph G with n vertices and m edges, the optimal (fractional) solution to the linear program $\text{LPVC}(G)$ can be found in time $\mathcal{O}(m\sqrt{n})$.*

Proof. We reduce the problem of solving $\text{LPVC}(G)$ to a problem of finding a minimum-size vertex cover in the following bipartite graph H . Its vertex set consists of two copies V_1 and V_2 of the vertex set of G . Thus, every vertex $v \in V(G)$ has two copies $v_1 \in V_1$ and $v_2 \in V_2$ in H . For every edge $uv \in E(G)$, we have edges u_1v_2 and v_1u_2 in H .

Using the Hopcroft-Karp algorithm (Theorem 2.13), we can find a minimum vertex cover S of H in time $\mathcal{O}(m\sqrt{n})$. We define a vector $(x_v)_{v \in V(G)}$ as follows: if both vertices v_1 and v_2 are in S , then $x_v = 1$. If exactly one of the vertices v_1 and v_2 is in S , we put $x_v = \frac{1}{2}$. We put $x_v = 0$ if none of the vertices v_1 and v_2 are in S . Thus

$$\sum_{v \in V(G)} x_v = \frac{|S|}{2}.$$

Since S is a vertex cover in H , we have that for every edge $uv \in E(G)$ at least two vertices from $\{u_1, u_2, v_1, v_2\}$ should be in S . Thus $x_u + x_v \geq 1$ and vector $(x_v)_{v \in V(G)}$ satisfies the constraints of $\text{LPVC}(G)$.

To show that $(x_v)_{v \in V(G)}$ is an optimal solution of $\text{LPVC}(G)$, we argue as follows. Let $(y_v)_{v \in V(G)}$ be an optimal solution of $\text{LPVC}(G)$. For every vertex v_i , $i \in \{1, 2\}$, of H , we assign the weight $\mathbf{w}(v_i) = y_v$. This weight assignment is a fractional vertex cover of H , i.e., for every edge $v_1 u_2 \in E(H)$, $\mathbf{w}(v_1) + \mathbf{w}(u_2) \geq 1$. We have that

$$\sum_{v \in V(G)} y_v = \frac{1}{2} \sum_{v \in V(G)} (\mathbf{w}(v_1) + \mathbf{w}(v_2)).$$

On the other hand, the value $\sum_{v \in V(H)} \mathbf{w}(v)$ of any fractional solution of $\text{LPVC}(H)$ is at least the size of a maximum matching M in H . A reader familiar with linear programming can see that this follows from weak duality; we also ask you to verify this fact in Exercise 2.24.

By König's theorem (Theorem 2.11), $|M| = |S|$. Hence

$$\sum_{v \in V(G)} y_v = \frac{1}{2} \sum_{v \in V(G)} (\mathbf{w}(v_1) + \mathbf{w}(v_2)) = \frac{1}{2} \sum_{v \in V(H)} \mathbf{w}(v) \geq \frac{|S|}{2} = \sum_{v \in V(G)} x_v.$$

Thus $(x_v)_{v \in V(G)}$ is an optimal solution of $\text{LPVC}(G)$. $\square$

We immediately obtain the following.

Corollary 2.23. *For a graph G with n vertices and m edges, the kernel of Theorem 2.21 can be found in time $\mathcal{O}(m\sqrt{n})$.*

The following proposition is another interesting consequence of the proof of Lemma 2.22.

Proposition 2.24. *Let G be a graph on n vertices and m edges. Then $\text{LPVC}(G)$ has a half-integral optimal solution, i.e., all variables have values in the set $\{0, \frac{1}{2}, 1\}$. Furthermore, we can find a half-integral optimal solution in time $\mathcal{O}(m\sqrt{n})$.*

In short, we have proved properties of $\text{LPVC}(G)$. There exists a half-integral optimal solution $(x_v)_{v \in V(G)}$ to $\text{LPVC}(G)$, and it can be found efficiently. We can look at this solution as a partition of $V(G)$ into parts V_0 , $V_{\frac{1}{2}}$, and V_1 with the following message: greedily take V_1 into a solution, do not take any vertex of V_0 into a solution, and in $V_{\frac{1}{2}}$, we do not know what to do and that is the hard part of the problem. However, as an optimum solution pays $\frac{1}{2}$ for every vertex of $V_{\frac{1}{2}}$, the hard part — the kernel of the problem — cannot have more than $2k$ vertices.

2.6 Sunflower lemma

In this section we introduce a classical result of Erdős and Rado and show some of its applications in kernelization. In the literature it is known as the sunflower lemma or as the Erdős-Rado lemma. We first define the terminology used in the statement of the lemma. A *sunflower* with k *petals* and a *core* Y is a collection of sets $S_1, \dots, S_k$ such that $S_i \cap S_j = Y$ for all $i \neq j$; the sets $S_i \setminus Y$ are petals and we require *none of them to be empty*. Note that a family of pairwise disjoint sets is a sunflower (with an empty core).

Theorem 2.25 (Sunflower lemma). *Let $\mathcal{A}$ be a family of sets (without duplicates) over a universe U , such that each set in $\mathcal{A}$ has cardinality exactly d . If $|\mathcal{A}| > d!(k-1)^d$, then $\mathcal{A}$ contains a sunflower with k petals and such a sunflower can be computed in time polynomial in $|\mathcal{A}|$, $|U|$, and k .*

Proof. We prove the theorem by induction on d . For $d = 1$, i.e., for a family of singletons, the statement trivially holds. Let $d \geq 2$ and let $\mathcal{A}$ be a family of sets of cardinality at most d over a universe U such that $|\mathcal{A}| > d!(k-1)^d$.

Let $\mathcal{G} = \{S_1, \dots, S_\ell\} \subseteq \mathcal{A}$ be an inclusion-wise maximal family of pairwise disjoint sets in $\mathcal{A}$. If $\ell \geq k$ then $\mathcal{G}$ is a sunflower with at least k petals. Thus we assume that $\ell < k$. Let $S = \bigcup_{i=1}^\ell S_i$. Then $|S| \leq d(k-1)$. Because $\mathcal{G}$ is maximal, every set $A \in \mathcal{A}$ intersects at least one set from $\mathcal{G}$, i.e., $A \cap S \neq \emptyset$. Therefore, there is an element $u \in U$ contained in at least

$$\frac{|\mathcal{A}|}{|\mathcal{G}|} > \frac{d!(k-1)^d}{d(k-1)} = (d-1)!(k-1)^{d-1}$$

sets from $\mathcal{A}$. We take all sets of $\mathcal{A}$ containing such an element u , and construct a family $\mathcal{A}'$ of sets of cardinality $d-1$ by removing from each set the element u . Because $|\mathcal{A}'| > (d-1)!(k-1)^{d-1}$, by the induction hypothesis, $\mathcal{A}'$ contains a sunflower $\{S'_1, \dots, S'_k\}$ with k petals. Then $\{S'_1 \cup \{u\}, \dots, S'_k \cup \{u\}\}$ is a sunflower in $\mathcal{A}$ with k petals.

The proof can be easily transformed into a polynomial-time algorithm, as follows. Greedily select a maximal set of pairwise disjoint sets. If the size

of this set is at least k , then return this set. Otherwise, find an element u contained in the maximum number of sets in $\mathcal{A}$, and call the algorithm recursively on sets of cardinality $d - 1$, obtained from deleting u from the sets containing u . $\square$

2.6.1 d -HITTING SET

As an application of the sunflower lemma, we give a kernel for d -HITTING SET. In this problem, we are given a family $\mathcal{A}$ of sets over a universe U , where each set in the family has cardinality at most d , and a positive integer k . The objective is to decide whether there is a subset $H \subseteq U$ of size at most k such that H contains at least one element from each set in $\mathcal{A}$.

Theorem 2.26. *d -HITTING SET admits a kernel with at most $d!k^d$ sets and at most $d!k^d \cdot d^2$ elements.*

Proof. The crucial observation is that if $\mathcal{A}$ contains a sunflower

$$S = \{S_1, \dots, S_{k+1}\}$$

of cardinality $k + 1$, then every hitting set H of $\mathcal{A}$ of cardinality at most k intersects the core Y of the sunflower S . Indeed, if H does not intersect Y , it should intersect each of the $k + 1$ disjoint petals $S_i \setminus Y$. This leads to the following reduction rule.

Reduction HS.1. Let $(U, \mathcal{A}, k)$ be an instance of d -HITTING SET and assume that $\mathcal{A}$ contains a sunflower $S = \{S_1, \dots, S_{k+1}\}$ of cardinality $k + 1$ with core Y . Then return $(U', \mathcal{A}', k)$, where $\mathcal{A}' = (\mathcal{A} \setminus S) \cup \{Y\}$ is obtained from $\mathcal{A}$ by deleting all sets $\{S_1, \dots, S_{k+1}\}$ and by adding a new set Y and $U' = \bigcup_{X \in \mathcal{A}'} X$.

Note that when deleting sets we do not delete the elements contained in these sets but only those which do not belong to any set. Then the instances $(U, \mathcal{A}, k)$ and $(U', \mathcal{A}', k)$ are equivalent, i.e. $(U, \mathcal{A})$ contains a hitting set of size k if and only if $(U, \mathcal{A}')$ does.

The kernelization algorithm is as follows. If for some $d' \in \{1, \dots, d\}$ the number of sets in $\mathcal{A}$ of size exactly d' is more than $d'!k^{d'}$, then the kernelization algorithm applies the sunflower lemma to find a sunflower of size $k + 1$, and applies Reduction HS.1 on this sunflower. It applies this procedure exhaustively, and obtains a new family of sets $\mathcal{A}'$ of size at most $d!k^d \cdot d$. If $\emptyset \in \mathcal{A}'$ (that is, at some point a sunflower with an empty core has been discovered), then the algorithm concludes that there is no hitting set of size at most k and returns that the given instance is a no-instance. Otherwise, every set contains at most d elements, and thus the number of elements in the kernel is at most $d!k^d \cdot d^2$. $\square$

Exercises

2.1 ($\mathcal{E}$). Prove Lemma 2.5: A digraph is acyclic if and only if it is possible to order its vertices in such a way such that for every arc (u, v) , we have $u < v$.

2.2 ($\mathcal{E}$). Give an example of a feedback arc set F in a tournament G , such that $G \oplus F$ is not acyclic.

2.3 ($\mathcal{E}$). Show that Reductions ECC.1, ECC.2, and ECC.3 are safe.

2.4 ($\mathcal{E}$). In the POINT LINE COVER problem, we are given a set of n points in the plane and an integer k , and the goal is to check if there exists a set of k lines on the plane that contain all the input points. Show a kernel for this problem with $\mathcal{O}(k^2)$ points.

2.5. A graph G is a *cluster graph* if every connected component of G is a clique. In the CLUSTER EDITING problem, we are given as input a graph G and an integer k , and the objective is to check whether one can edit (add or delete) at most k edges in G to obtain a cluster graph. That is, we look for a set $F \subseteq \binom{V(G)}{2}$ of size at most k , such that the graph $(V(G), (E(G) \setminus F) \cup (F \setminus E(G)))$ is a cluster graph.

1. Show that a graph G is a cluster graph if and only if it does not have an induced path on three vertices (sequence of three vertices u, v, w such that uv and vw are edges and $uw \notin E(G)$).
2. Show a kernel for CLUSTER EDITING with $\mathcal{O}(k^2)$ vertices.

2.6. In the SET SPLITTING problem, we are given a family of sets $\mathcal{F}$ over a universe U and a positive integer k , and the goal is to test whether there exists a coloring of U with two colors such that at least k sets in $\mathcal{F}$ are nonmonochromatic (that is, they contain vertices of both colors). Show that the problem admits a kernel with at most $2k$ sets and $\mathcal{O}(k^2)$ universe size.

2.7. In the MINIMUM MAXIMAL MATCHING problem, we are given an undirected graph G and a positive integer k , and the objective is to decide whether there exists a maximal matching in G on at most k edges. Obtain a polynomial kernel for the problem (parameterized by k).

2.8. In the MIN-ONES-2-SAT problem, we are given a 2-CNF formula ϕ and an integer k , and the objective is to decide whether there exists a satisfying assignment for ϕ with at most k variables set to true. Show that MIN-ONES-2-SAT admits a polynomial kernel.

2.9. In the d -BOUNDED-DEGREE DELETION problem, we are given an undirected graph G and a positive integer k , and the task is to find at most k vertices whose removal decreases the maximum vertex degree of the graph to at most d . Obtain a kernel of size polynomial in k and d for the problem. (Observe that VERTEX COVER is the case of $d = 0$.)

2.10. Show a kernel with $\mathcal{O}(k^2)$ vertices for the following problem: given a graph G and an integer k , check if G contains a subgraph with exactly k edges, whose vertices are all of odd degree in the subgraph.

2.11. A set of vertices D in an undirected graph G is called a *dominating set* if $N[D] = V(G)$. In the DOMINATING SET problem, we are given an undirected graph G and a positive integer k , and the objective is to test whether there exists a dominating set of size at most k . Show that DOMINATING SET admits a polynomial kernel on graphs where the length of the shortest cycle is at least 5. (What would you do with vertices with degree more than k ? Note that unlike for the VERTEX COVER problem, you cannot delete a vertex once you pick it in the solution.)

2.12. Show that **FEEDBACK VERTEX SET** admits a kernel with $\mathcal{O}(k)$ vertices on undirected regular graphs.

2.13. We say that an n -vertex digraph is *well-spread* if every vertex has indegree at least $\sqrt{n}$. Show that **DIRECTED FEEDBACK ARC SET**, restricted to well-spread digraphs, is FPT by obtaining a polynomial kernel for this problem. Does the problem remain FPT if we replace the lower bound on indegree by any monotonically increasing function of n (like $\log n$ or $\log \log \log n$)? Does the assertion hold if we replace indegree with outdegree? What about **DIRECTED FEEDBACK VERTEX SET**?

2.14. In the **CONNECTED VERTEX COVER** problem, we are given an undirected graph G and a positive integer k , and the objective is to decide whether there exists a vertex cover C of G such that $|C| \leq k$ and $G[C]$ is connected.

1. Explain where the kernelization procedure described in Theorem 2.4 for **VERTEX COVER** breaks down for the **CONNECTED VERTEX COVER** problem.
2. Show that the problem admits a kernel with at most $2^k + \mathcal{O}(k^2)$ vertices.
3. Show that if the input graph G does not contain a cycle of length 4 as a subgraph, then the problem admits a kernel with at most $\mathcal{O}(k^2)$ vertices.

2.15 (🔗). Extend the argument of the previous exercise to show that, for every fixed $d \geq 2$, **CONNECTED VERTEX COVER** admits a kernel of size $\mathcal{O}(k^d)$ if restricted to graphs that do not contain the biclique $K_{d,d}$ as a subgraph.

2.16 (🔗). A graph G is chordal if it contains no induced cycles of length more than 3, that is, every cycle of length at least 4 has a chord. In the **CHORDAL COMPLETION** problem, we are given an undirected graph G and a positive integer k , and the objective is to decide whether we can add at most k edges to G so that it becomes a chordal graph. Obtain a polynomial kernel for **CHORDAL COMPLETION** (parameterized by k).

2.17 (🔗). In the **EDGE DISJOINT CYCLE PACKING** problem, we are given an undirected graph G and a positive integer k , and the objective is to test whether G has k pairwise edge disjoint cycles. Obtain a polynomial kernel for **EDGE DISJOINT CYCLE PACKING** (parameterized by k).

2.18 (🔗). A *bisection* of a graph G with an even number of vertices is a partition of $V(G)$ into V_1 and V_2 such that $|V_1| = |V_2|$. The size of (V_1, V_2) is the number of edges with one endpoint in V_1 and the other in V_2 . In the **MAXIMUM BISECTION** problem, we are given an undirected graph G with an even number of vertices and a positive integer k , and the objective is to test whether there exists a bisection of size at least k .

1. Show that every graph with m edges has a bisection of size at least $\lceil \frac{m}{2} \rceil$. Use this to show that **MAXIMUM BISECTION** admits a kernel with $2k$ edges.
2. Consider the following “above guarantee” variant of **MAXIMUM BISECTION**, where we are given an undirected graph G and a positive integer k , but the objective is to test whether there exists a bisection of size at least $\lceil \frac{m}{2} \rceil + k$. Show that the problem admits a kernel with $\mathcal{O}(k^2)$ vertices and $\mathcal{O}(k^3)$ edges.

2.19 (🔗). Byteland, a country of area exactly n square miles, has been divided by the government into n regions, each of area exactly one square mile. Meanwhile, the army of Byteland divided its area into n military zones, each of area again exactly one square mile. Show that one can build n airports in Byteland, such that each region and each military zone contains one airport.

2.20. A magician and his assistant are performing the following magic trick. A volunteer from the audience picks five cards from a standard deck of 52 cards and then passes the deck to the assistant. The assistant shows to the magician, one by one in some order, four cards from the chosen set of five cards. Then, the magician guesses the remaining fifth card. Show that this magic trick can be performed without any help of magic.

2.21. Prove the second claim of Theorem 2.13.

2.22. In the DUAL-COLORING problem, we are given an undirected graph G on n vertices and a positive integer k , and the objective is to test whether there exists a proper coloring of G with at most $n - k$ colors. Obtain a kernel with $\mathcal{O}(k)$ vertices for this problem using crown decomposition.

2.23 () In the MAX-INTERNAL SPANNING TREE problem, we are given an undirected graph G and a positive integer k , and the objective is to test whether there exists a spanning tree with at least k internal vertices. Obtain a kernel with $\mathcal{O}(k)$ vertices for MAX-INTERNAL SPANNING TREE.

2.24 () Let G be an undirected graph, let $(x_v)_{v \in V(G)}$ be any feasible solution to LPVC(G), and let M be a matching in G . Prove that $|M| \leq \sum_{v \in V(G)} x_v$.

2.25 () Let G be a graph and let $(x_v)_{v \in V(G)}$ be an optimum solution to LPVC(G) (not necessarily a half-integral one). Define a vector $(y_v)_{v \in V(G)}$ as follows:

$$y_v = \begin{cases} 0 & \text{if } x_v < \frac{1}{2} \\ \frac{1}{2} & \text{if } x_v = \frac{1}{2} \\ 1 & \text{if } x_v > \frac{1}{2}. \end{cases}$$

Show that $(y_v)_{v \in V(G)}$ is also an optimum solution to LPVC(G).

2.26 () In the MIN-ONES-2-SAT, we are given a CNF formula, where every clause has exactly two literals, and an integer k , and the goal is to check if there exists a satisfying assignment of the input formula with at most k variables set to true. Show a kernel for this problem with at most $2k$ variables.

2.27 () Consider a restriction of d -HITTING SET, called Ed-HITTING SET, where we require every set in the input family $\mathcal{A}$ to be of size *exactly* d . Show that this problem is not easier than the original d -HITTING SET problem, by showing how to transform a d -HITTING SET instance into an equivalent Ed-HITTING SET instance without changing the number of sets.

2.28. Show a kernel with at most $f(d)k^d$ sets (for some computable function f) for the Ed-HITTING SET problem, defined in the previous exercise.

2.29. In the d -SET PACKING problem, we are given a family $\mathcal{A}$ of sets over a universe U , where each set in the family has cardinality at most d , and a positive integer k . The objective is to decide whether there are sets $S_1, \dots, S_k \in \mathcal{A}$ that are pairwise disjoint. Use the sunflower lemma to obtain a kernel for d -SET PACKING with $f(d)k^d$ sets, for some computable function d .

2.30. Consider a restriction of d -SET PACKING, called Ed-SET PACKING, where we require every set in the input family $\mathcal{A}$ to be of size *exactly* d . Show that this problem is not easier than the original d -SET PACKING problem, by showing how to transform a d -SET PACKING instance into an equivalent Ed-SET PACKING instance without changing the number of sets.

2.31. A *split graph* is a graph in which the vertices can be partitioned into a clique and an independent set. In the VERTEX DISJOINT PATHS problem, we are given an undirected graph G and k pairs of vertices (s_i, t_i) , $i \in \{1, \dots, k\}$, and the objective is to decide whether there exists paths P_i joining s_i to t_i such that these paths are pairwise vertex disjoint. Show that VERTEX DISJOINT PATHS admits a polynomial kernel on split graphs (when parameterized by k).

2.32. Consider now the VERTEX DISJOINT PATHS problem, defined in the previous exercise, restricted, for a fixed integer $d \geq 3$, to a class of graphs that does not contain a d -vertex path as an induced subgraph. Show that in this class the VERTEX DISJOINT PATHS problem admits a kernel with $\mathcal{O}(k^{d-1})$ vertices and edges.

2.33. In the CLUSTER VERTEX DELETION problem, we are given as input a graph G and a positive integer k , and the objective is to check whether there exists a set $S \subseteq V(G)$ of size at most k such that $G - S$ is a cluster graph. Show a kernel for CLUSTER VERTEX DELETION with $\mathcal{O}(k^3)$ vertices.

2.34. An undirected graph G is called *perfect* if for every induced subgraph H of G , the size of the largest clique in H is same as the chromatic number of H . In the ODD CYCLE TRANSVERSAL problem, we are given an undirected graph G and a positive integer k , and the objective is to find at most k vertices whose removal makes the resulting graph bipartite. Obtain a kernel with $\mathcal{O}(k^2)$ vertices for ODD CYCLE TRANSVERSAL on perfect graphs.

2.35. In the SPLIT VERTEX DELETION problem, we are given an undirected graph G and a positive integer k and the objective is to test whether there exists a set $S \subseteq V(G)$ of size at most k such that $G - S$ is a split graph (see Exercise 2.31 for the definition).

1. Show that a graph is split if and only if it has no induced subgraph isomorphic to one of the following three graphs: a cycle on four or five vertices, or a pair of disjoint edges.
2. Give a kernel with $\mathcal{O}(k^5)$ vertices for SPLIT VERTEX DELETION.

2.36 () **2.36.** In the SPLIT EDGE DELETION problem, we are given an undirected graph G and a positive integer k , and the objective is to test whether G can be transformed into a split graph by deleting at most k edges. Obtain a polynomial kernel for this problem (parameterized by k).

2.37 () **2.37.** In the RAMSEY problem, we are given as input a graph G and an integer k , and the objective is to test whether there exists in G an independent set or a clique of size at least k . Show that RAMSEY is FPT.

2.38 () **2.38.** A directed graph D is called *oriented* if there is no directed cycle of length at most 2. Show that the problem of testing whether an oriented digraph contains an induced directed acyclic subgraph on at least k vertices is FPT.

Hints

2.4 Consider the following reduction rule: if there exists a line that contains more than k input points, delete the points on this line and decrease k by 1.

2.5 Consider the following natural reduction rules:

1. delete a vertex that is not a part of any P_3 (induced path on three vertices);
2. if an edge uv is contained in at least $k + 1$ different P_3 s, then delete uv ;
3. if a non-edge uv is contained in at least $k + 1$ different P_3 s, then add uv .

Show that, after exhaustive application of these rules, a yes-instance has $\mathcal{O}(k^2)$ vertices.

2.6 First, observe that one can discard any set in $\mathcal{F}$ that is of size at most 1. Second, observe that if every set in $\mathcal{F}$ is of size at least 2, then a random coloring of U has at least

$|\mathcal{F}|/2$ nonmonochromatic sets on average, and an instance with $|\mathcal{F}| \geq 2k$ is a yes-instance. Moreover, observe that if we are dealing with a yes-instance and $F \in \mathcal{F}$ is of size at least $2k$, then we can always tweak the solution coloring to color F nonmonochromatically: fix two differently colored vertices for $k-1$ nonmonochromatic sets in the solution, and color some two uncolored vertices of F with different colors. Use this observation to design a reduction rule that handles large sets in $\mathcal{F}$.

2.7 Observe that the endpoints of the matching in question form a vertex cover of the input graph. In particular, every vertex of degree larger than $2k$ needs to be an endpoint of a solution matching. Let X be the set of these large-degree vertices. Argue, similarly as in the case of $\mathcal{O}(k^2)$ kernel for VERTEX COVER, that in a yes-instance, $G \setminus X$ has only few edges. Design a reduction rule to reduce the number of isolated vertices of $G \setminus X$.

2.8 Proceed similarly as in the $\mathcal{O}(k^2)$ kernel for VERTEX COVER.

2.9 Proceed similarly as in the case of VERTEX COVER. Argue that the vertices of degree larger than $d+k$ need to be included in the solution. Moreover, observe that you may delete isolated vertices, as well as edges connecting two vertices of degree at most d . Argue that, if no rule is applicable, then a yes-instance is of size bounded polynomially in $d+k$.

2.10 The important observation is that a matching of size k is a good subgraph. Hence, we may restrict ourselves to the case where we are additionally given a vertex cover X of the input graph of size at most $2k$. Moreover, assume that X is inclusion-wise minimal. To conclude, prove that, if a vertex $v \in X$ has at least k neighbors in $V(G) \setminus X$, then (G, k) is a yes-instance.

2.11 The main observation is that, since there is no 3-cycle nor 4-cycle in the graph, if $x, y \in N(v)$, then only v can dominate both x and y at once. In particular, every vertex of degree larger than k needs to be included in the solution.

However, you cannot easily delete such a vertex. Instead, mark it as “obligatory” and mark its neighbors as “dominated”. Note now that you can delete a “dominated” vertex, as long as it has no unmarked neighbor and its deletion does not drop the degree of an “obligatory” vertex to k .

Prove that, in a yes-instance, if no rule is applicable, then the size is bounded polynomially in k . To this end, show that

1. any vertex can dominate at most k unmarked vertices, and, consequently, there are at most k^2 unmarked vertices;
2. there are at most k “obligatory” vertices;
3. every remaining “dominated” vertex can be charged to one unmarked or obligatory vertex in a manner that each unmarked or obligatory vertex is charged at most $k+1$ times.

2.12 Let (G, k) be a FEEDBACK VERTEX SET instance and assume G is d -regular. If $d \leq 2$, then solve (G, k) in polynomial time. Otherwise, observe that G has $dn/2$ edges and, if (G, k) is a yes-instance and X is a feedback vertex set of G of size at most k , then at most dk edges of G are incident to X and $G - X$ contains less than $n - k$ edges (since it is a forest). Consequently, $dn/2 \leq dk + n - k$, which gives $n = \mathcal{O}(k)$ for $d \geq 3$.

2.13 Show, using greedy arguments, that if every vertex in a digraph G has indegree at least d , then G contains d pairwise edge-disjoint cycles.

For the vertex-deletion variant, design a simple reduction that boosts up the indegree of every vertex without actually changing anything in the solution space.

2.14 Let X be the set of vertices of G of degree larger than k . Clearly, any connected vertex cover of G of size at most k needs to contain X . Moreover, as in the case of VERTEX COVER, in a yes-instance there are only $\mathcal{O}(k^2)$ edges in $G - X$. However, we cannot easily discard the isolated vertices of $G - X$, as they may be used to make the solution connected.

To obtain an exponential kernel, note that in a yes-instance, $|X| \leq k$, and if we have two vertices u, v that are isolated in $G - X$, and $N_G(u) = N_G(v)$ (note that $N_G(u) \subseteq X$ for every u that is isolated in $G - X$), then we need only one of the vertices u, v in a CONNECTED VERTEX COVER solution. Hence, in a kernel, we need to keep:

1. $G[X]$, and all edges and non-isolated vertices of $G - X$;
2. for every $x \in X$, some $k + 1$ neighbors of x ;
3. for every $Y \subseteq X$, one vertex u that is isolated in $G - X$ and $N_G(u) = Y$ (if there exists any such vertex).

For the last part of the exercise, note that in the presence of this assumption, no two vertices of X share more than one neighbor and, consequently, there are only $\mathcal{O}(|X|^2)$ sets $Y \subseteq X$ for which there exist $u \notin X$ with $N_G(u) = Y$.

2.15 We repeat the argument of the previous exercise, and bound the number of sets $Y \subseteq X$ for which we need to keep a vertex $u \in V(G) \setminus X$ with $N_G(u) = Y$. First, there are $\mathcal{O}(d|X|^{d-1})$ sets Y of size smaller than d . Second, charge every set Y of size at least d to one of its subset of size d . Since G does not contain $K_{d,d}$ as a subgraph, every subset X of size d is charged less than d times. Consequently, there are at most $(d-1)\binom{|X|}{d}$ vertices $u \in V(G) \setminus X$ such that $N_G(u) \subseteq X$ and $|N_G(u)| \geq d$.

2.16 The main observation is as follows: an induced cycle of length ℓ needs exactly $\ell - 3$ edges to become chordal. In particular, if a graph contains an induced cycle of length larger than $k + 3$, then the input instance is a no-instance, as we need more than k edges to triangulate the cycle in question.

First, prove the safeness of the following two reduction rules:

1. Delete any vertex that is not contained in any induced cycle in G .
2. A vertex x is a *friend* of a non-edge uv , if u, x, v are three consecutive vertices of some induced cycle in G . If $uv \notin E(G)$ has more than $2k$ friends, then add the edge uv and decrease k by one.

Second, consider the following procedure. Initiate A to be the vertex set of any inclusion-wise maximal family of pairwise vertex-disjoint induced cycles of length at most 4 in G . Then, as long as there exists an induced cycle of length at most 4 in G that contains two consecutive vertices in $V(G) \setminus A$, move these two vertices to A . Show, using a charging argument, that, in a yes-instance, the size of A remains $\mathcal{O}(k)$. Conclude that the size of a reduced yes-instance is bounded polynomially in k .

2.17 Design reduction rules that remove vertices of degree at most 2 (you may obtain a multigraph in the process). Prove that every n -vertex multigraph of minimum degree at least 3 has a cycle of length $\mathcal{O}(\log n)$. Use this to show a greedy argument that an n -vertex multigraph of minimum degree 3 has $\Omega(n^\varepsilon)$ pairwise edge-disjoint cycles for some $\varepsilon > 0$.

2.18 Consider the following argument. Let $|V(G)| = 2n$ and pair the vertices of G arbitrarily: $V(G) = \{x_1, y_1, x_2, y_2, \dots, x_n, y_n\}$. Consider the bisection (V_1, V_2) where, in each pair (x_i, y_i) , one vertex goes to V_1 and the other goes to V_2 , where the decision is made uniformly at random and independently of other pairs. Prove that, in expectation, the obtained bisection is of size at least $(m + \ell)/2$, where ℓ is the number of pairs (x_i, y_i) where $x_i y_i \in E(G)$.

Use the arguments in the previous paragraph to show not only the first point of the exercise, but also that the input instance is a yes-instance if it admits a matching of size $2k$. If this is not the case, then let X be the set of endpoints of a maximal matching in G ; note that $|X| \leq 4k$.

First, using a variation of the argument of the first paragraph, prove that, if there exists $x \in X$ that has at least $2k$ neighbors and at least $2k$ non-neighbors outside X , then the input instance is a yes-instance. Second, show that in the absence of such a vertex, all but

$\mathcal{O}(k^2)$ vertices of $V(G) \setminus X$ have exactly the same neighborhood in X , and design a way to reduce them.

2.19 Construct the following bipartite graph: on one side there are regions of Byteland, on the second side there are military zones, and a region R is adjacent to a zone Z if $R \cap Z \neq \emptyset$. Show that this graph satisfies the condition of Hall's theorem and, consequently, contains a perfect matching.

2.20 Consider the following bipartite graph: on one side there are all $\binom{52}{5}$ sets of five cards (possibly chosen by the volunteer), and on the other side there are all $52 \cdot 51 \cdot 50 \cdot 49$ tuples of pairwise different four cards (possibly shown by the assistant). A set S is adjacent to a tuple T if all cards of T belong to S . Using Hall's theorem, show that this graph admits a matching saturating the side with all sets of five cards. This matching induces a strategy for the assistant and the magician.

We now show a relatively simple explicit strategy, so that you can impress your friends and perform this trick at some party. In every set of five cards, there are two cards of the same color, say a and b . Moreover, as there are 13 cards of the same color, the cards a and b differ by at most 6, that is, $a + i = b$ or $b + i = a$ for some $1 \leq i \leq 6$, assuming some cyclic order on the cards of the same color. Without loss of generality, assume $a + i = b$. The assistant first shows the card a to the magician. Then, using the remaining three cards, and some fixed total order on the whole deck of cards, the assistant shows the integer i (there are $3! = 6$ permutations of remaining three cards). Consequently, the magician knows the card b by knowing its color (the same as the first card show by the assistant) and the value of the card a and the number i .

2.21 Let M be a maximum matching, which you can find using the Hopcroft-Karp algorithm (the first part of Theorem 2.13). If M saturates V_1 , then we are done. Otherwise, pick any $v \in V_1 \setminus V(M)$ (i.e., a vertex $v \in V_1$ that is not an endpoint of an edge of M) and consider all vertices of G that are reachable from v using alternating paths. (A path P is *alternating* if every second edge of P belongs to M .) Show that all vertices from V_1 that are reachable from v using alternating paths form an inclusion-wise minimal set X with $|N(X)| < |X|$.

2.22 Apply the crown lemma to $\bar{G}$, the edge complement of G ($\bar{G}$ has vertex set $V(G)$ and $uv \in E(\bar{G})$ if and only if $uv \notin E(G)$) and the parameter $k - 1$. If it returns a matching M_0 of size k , then note that one can color the endpoints of each edge of M_0 with the same color, obtaining a coloring of G with $n - k$ colors. Otherwise, design a way to greedily color the head and the crown of the obtained crown decomposition.

2.23 Your main tool is the following variation of the crown lemma: if $V(G)$ is sufficiently large, then you can find either a matching of size $k + 1$, or a crown decomposition $V(G) = C \cup H \cup R$, such that $G[H \cup C]$ admits a spanning tree where all vertices of H and $|H| - 1$ vertices of C are of degree at least two. Prove it, and use it for the problem in question.

2.24 Observe that for every $uv \in M$ we have $x_u + x_v \geq 1$ and, moreover, all these inequalities for all edges of M contain different variables. In other words,

$$\sum_{v \in V(G)} \mathbf{w}(x_v) \geq \sum_{v \in V(M)} \mathbf{w}(x_v) = \sum_{vu \in M} (\mathbf{w}(x_v) + \mathbf{w}(x_u)) \geq \sum_{vu \in M} 1 = |M|.$$

2.25 Let $V_\delta = \{v \in V(G) : 0 < x_v < \frac{1}{2}\}$ and $V_{1-\delta} = \{v \in V(G) : \frac{1}{2} < x_v < 1\}$. For sufficiently small $\varepsilon > 0$, consider two operations: first, an operation of adding ε to all variables x_v for $v \in V_\delta$ and subtracting ε from x_v for $v \in V_{1-\delta}$, and second, an operation of adding ε to all variables x_v for $v \in V_{1-\delta}$ and subtracting ε from x_v for $v \in V_\delta$. Show that both these operations lead to feasible solutions to $\text{LPVC}(G)$, as long as ε is small enough. Conclude that $|V_\delta| = |V_{1-\delta}|$, and that both operations lead to other *optimal* solutions

to $\text{LPVC}(G)$. Finally, observe that, by repeatedly applying the second operation, one can empty sets V_δ and $V_{1-\delta}$, and reach the vector $(y_v)_{v \in V(G)}$.

2.26 First, design natural reduction rules that enforce the following: for every variable x in the input formula φ , there exists a truth assignment satisfying φ that sets x to false, and a truth assignment satisfying φ that sets x to true. In other words, whenever a value of some variable is fixed in any satisfying assignment, fix this value and propagate it in the formula.

Then, consider the following *closure* of the formula φ : for every two-literal clause C that is satisfied in every truth assignment satisfying φ , add C to φ . Note that testing whether C is such a clause can be done in polynomial time: force two literals in C to be false and check if φ remains satisfiable. Moreover, observe that the sets of satisfying assignments for φ and φ' are equal.

Let φ' be the closure of φ . Consider the following auxiliary graph H : $V(H)$ is the set of variables of φ' , and $xy \in E(H)$ iff the clause $x \vee y$ belongs to φ' . Clearly, if we take any truth assignment ψ satisfying φ , then $\psi^{-1}(\top)$ is a vertex cover of H . A somewhat surprising fact is that a partial converse is true: for every inclusion-wise minimal vertex cover X of H , the assignment ψ defined as $\psi(x) = \top$ if and only if $x \in X$ satisfies φ' (equivalently, φ). Note that such a claim would solve the exercise: we can apply the LP-based kernelization algorithm to VERTEX COVER instance (H, k) , and translate the reductions it makes back to the formula φ .

Below we prove the aforementioned claim in full detail. We encourage you to try to prove it on your own before reading.

Let X be a minimal vertex cover of H , and let ψ be defined as above. Take any clause C in φ' and consider three cases. If $C = x \vee y$, then $xy \in E(H)$, and, consequently, either x or y belongs to X . It follows from the definition of ψ that $\psi(x) = \top$ or $\psi(y) = \top$, and ψ satisfies C .

In the second case, $C = x \vee \neg y$. For a contradiction, assume that ψ does not satisfy C and, consequently, $x \notin X$ and $y \in X$. Since X is a minimal vertex cover, there exists $z \in N_H(y)$ such that $z \notin X$ and the clause $C' = y \vee z$ belongs to φ' . If $z = x$, then any satisfying assignment to φ' sets y to true, a contradiction to our first preprocessing step. Otherwise, the presence of C and C' implies that in any assignment ψ' satisfying φ' we have $\psi'(x) = \top$ or $\psi'(z) = \top$. Thus, $x \vee z$ is a clause of φ' , and $xz \in E(H)$. However, neither x nor z belongs to X , a contradiction.

In the last case, $C = \neg x \vee \neg y$ and, again, we assume that ψ does not satisfy C , that is, $x, y \in X$. Since X is a minimal vertex cover, there exist $s \in N_H(x)$, $t \in N_H(y)$ such that $s, t \notin X$. It follows from the definition of H that the clauses $C_x = x \vee s$ and $C_y = y \vee t$ are present in φ' . If $s = t$, then the clauses C , C_x and C_y imply that t is set to true in any truth assignment satisfying φ' , a contradiction to our first preprocessing step. If $s \neq t$, then observe that the clauses C , C_x and C_y imply that either s or t is set to true in any truth assignment satisfying φ' and, consequently, $s \vee t$ is a clause of φ' and $st \in E(H)$. However, $s, t \notin X$, a contradiction.

2.27 If $\emptyset \in \mathcal{A}$, then conclude that we are dealing with a no-instance. Otherwise, for every set $X \in \mathcal{A}$ of size $|X| < d$, create $d - |X|$ new elements and add them to X .

2.28 There are two different ways to solve this exercise. First, you can treat the input instance as a d -HITTING SET instance, proceed as in Section 2.6.1, and at the end apply the solution of Exercise 2.27 to the obtained kernel, in order to get an Ed -HITTING SET instance.

In a second approach, try to find a sunflower with $k + 2$ sets, instead of $k + 1$ as in Section 2.6.1. If a sunflower is found, then discard one of the sets: the remaining $k + 1$ sets still ensure that the core needs to be hit in any solution of size at most k .

2.29 Show that in a $(dk + 2)$ -sunflower, one set can be discarded without changing the answer to the problem.

2.30 Proceed as in Exercise 2.27: pad every set $X \in \mathcal{A}$ with $d - |X|$ newly created elements.

2.31 Show that, in a split graph, every path can be shortened to a path on at most four vertices. Thus, for every $1 \leq i \leq k$, we have a family $\mathcal{F}_i$ of vertex sets of possible paths between s_i and t_i , and this family is of size $\mathcal{O}(n^4)$. Interpret the problem as a d -SET PACKING instance for some constant d and family $\mathcal{F} = \bigcup_{i=1}^k \mathcal{F}_i$. Run the kernelization algorithm from the previous exercise, and discard all vertices that are not contained in any set in the obtained kernel.

2.32 Show that, in a graph excluding a d -vertex path as an induced subgraph, every path in a solution can be shortened to a path on at most $d - 1$ vertices. Proceed then as in Exercise 2.31.

2.33 Let $\mathcal{A}$ be a family of all vertex sets of a P_3 (induced path on three vertices) in G . In this manner, CLUSTER VERTEX DELETION becomes a 3-HITTING SET problem on family $\mathcal{A}$, as we need to hit all induced P_3 s in G . Reduce $\mathcal{A}$, but not exactly as in the d -HITTING SET case: repeatedly find a $k + 2$ sunflower and delete one of its elements from $\mathcal{A}$. Show that this reduction is safe for CLUSTER VERTEX DELETION. Moreover, show that, if $\mathcal{A}'$ is the family after the reduction is exhaustively applied, then $(G[\bigcup \mathcal{A}'], k)$ is the desired kernel.

2.34 Use the following observation: a perfect graph is bipartite if and only if it does not contain a triangle. Thus, the problem reduces to hitting all triangles in the input graph, which is a 3-HITTING SET instance.

2.35 Proceed as in the case of CLUSTER VERTEX DELETION: interpret a SPLIT VERTEX DELETION instance as a 5-HITTING SET instance.

2.36 Let $\{C_4, C_5, 2K_2\}$ be the set of *forbidden induced subgraphs* for split graphs. That is, a graph is a split graph if it contains none of these three graphs as an induced subgraph.

You may need (some of) the following reduction rules. (Note that the safeness of some of them is not so easy.)

1. The “standard” sunflower-like: if more than k forbidden induced subgraphs share a single edge (and otherwise are pairwise edge-disjoint), delete the edge in question.
2. The irrelevant vertex rule: if a vertex is not part of any forbidden induced subgraph, then delete the vertex in question. (Safeness is not obvious here!)
3. If two adjacent edges uv and uw are contained in more than k induced C_4 s, then delete uv and uw , and replace them with edges va and wb , where a and b are new degree-1 vertices.
4. If two adjacent edges uv and uw are contained in more than k pairwise edge-disjoint induced C_5 s, then delete uv and uw , and decrease k by 2.
5. If the edges v_1v_2 , v_2v_3 and v_3v_4 are contained in more than k induced C_5 s, delete v_2v_3 and decrease k by 1.

2.37 By induction, show that every graph on at least 4^k vertices has either a clique or an independent set on k vertices. Observe that this implies a kernel of exponential size for the problem.

2.38 Show that every tournament on n vertices has a transitive subtournament on $\mathcal{O}(\log n)$ vertices. Then, use this fact to show that every oriented directed graph on n vertices has an induced directed acyclic subgraph on $\log n$ vertices. Finally, obtain an exponential kernel for the considered problem.

Bibliographic notes

The history of preprocessing, such as that of applying reduction rules to simplify truth functions, can be traced back to the origins of computer science—the 1950 work of Quine [391]. A modern example showing the striking power of efficient preprocessing is the commercial integer linear program solver CPLEX. The name “lost continent of polynomial-time preprocessing” is due to Mike Fellows [175].

Lemma 2.2 on equivalence of kernelization and fixed-parameter tractability is due to Cai, Chen, Downey, and Fellows [66]. The reduction rules for VERTEX COVER discussed in this chapter are attributed to Buss in [62] and are often referred to as Buss kernelization in the literature. A more refined set of reduction rules for VERTEX COVER was introduced in [24]. The kernelization algorithm for FAST in this chapter follows the lines provided by Dom, Guo, Hüffner, Niedermeier and Truß [140]. The improved kernel with $(2 + \varepsilon)k$ vertices, for $\varepsilon > 0$, is obtained by Bessy, Fomin, Gaspers, Paul, Perez, Saurabh, and Thomassé in [32]. The exponential kernel for EDGE CLIQUE COVER is taken from [234], see also Gyárfás [253]. Cygan, Kratsch, Pilipczuk, Pilipczuk and Wahlström [114] showed that EDGE CLIQUE COVER admits no kernel of polynomial size unless $\text{NP} \subseteq \text{coNP}/\text{poly}$ (see Exercise 15.4, point 16). Actually, as we will see in Chapter 14 (Exercise 14.10), one can prove a stronger result: no subexponential kernel exists for this problem unless $\text{P} = \text{NP}$.

König’s theorem (found also independently in a more general setting of weighted graphs by Egerváry) and Hall’s theorem [256, 303] are classic results from graph theory, see also the book of Lovász and Plummer [338] for a general overview of matching theory. The crown rule was introduced by Chor, Fellows and Juedes in [94], see also [174]. Implementation issues of kernelization algorithms for vertex cover are discussed in [4]. The kernel for MAXIMUM SATISFIABILITY (Theorem 2.16) is taken from [323]. Abu-Khzam used crown decomposition to obtain a kernel for d -HITTING SET with at most $(2d - 1)k^{d-1} + k$ elements [3] and for d -SET PACKING with $\mathcal{O}(k^{d-1})$ elements [2]. Crown Decomposition and its variations were used to obtain kernels for different problems by Wang, Ning, Feng and Chen [431], Prieto and Sloper [388, 389], Fellows, Heggeres, Rosamond, Sloper and Telle [178], Moser [369], Chlebík and Chlebíková [93]. The expansion lemma, in a slightly different form, appears in the PhD thesis of Prieto [387], see also Thomassé [420, Theorem 2.3] and Halmos and Vaughan [257].

The Nemhauser-Trotter theorem is a classical result from combinatorial optimization [375]. Our proof of this theorem mimics the proof of Khuller from [289]. The application of the Nemhauser-Trotter theorem in kernelization was observed by Chen, Kanj and Jia [81]. The sunflower lemma is due to Erdős and Rado [167]. Our kernelization for d -HITTING SET follows the lines of [189].

Exercise 2.11 is from [392], Exercise 2.15 is from [121, 383], Exercise 2.16 is from [281], Exercise 2.18 is from [251] and Exercise 2.23 is from [190]. An improved kernel for the above guarantee variant of MAXIMUM BISECTION, discussed in Exercise 2.18, is obtained by Mních and Zenklusen [364]. A polynomial kernel for SPLIT EDGE DELETION (Exercise 2.36) was first shown by Guo [240]. An improved kernel, as well as a smaller kernel for SPLIT VERTEX DELETION, was shown by Ghosh, Kolay, Kumar, Misra, Panolan, Rai, and Ramanujan [228]. It is worth noting that the very simple kernel of Exercise 2.4 is probably optimal by the result of Kratsch, Philip, and Ray [308].

Chapter 3

Bounded search trees

In this chapter we introduce a variant of exhaustive search, namely the method of bounded search trees. This is one of the most commonly used tools in the design of fixed-parameter algorithms. We illustrate this technique with algorithms for two different parameterizations of VERTEX COVER, as well as for the problems (undirected) FEEDBACK VERTEX SET and CLOSEST STRING.



Bounded search trees, or simply *branching*, is one of the simplest and most commonly used techniques in parameterized complexity that originates in the general idea of backtracking. The algorithm tries to build a feasible solution to the problem by making a sequence of decisions on its shape, such as whether to include some vertex into the solution or not. Whenever considering one such step, the algorithm investigates many possibilities for the decision, thus effectively *branching* into a number of subproblems that are solved one by one. In this manner the execution of a branching algorithm can be viewed as a *search tree*, which is traversed by the algorithm up to the point when a solution is discovered in one of the leaves. In order to justify the correctness of a branching algorithm, one needs to argue that in case of a yes-instance some sequence of decisions captured by the algorithm leads to a feasible solution. If the total size of the search tree is bounded by a function of the parameter alone, and every step takes polynomial time, then such a branching algorithm runs in FPT time. This is indeed the case for many natural backtracking algorithms.

More precisely, let I be an instance of a minimization problem (such as VERTEX COVER). We associate a measure $\mu(I)$ with the instance I , which, in the case of FPT algorithms, is usually a function of k alone. In a branch step we generate from I simpler instances $I_1, \dots, I_\ell$ ($\ell \geq 2$) of the same problem such that the following hold.

1. Every feasible solution S of I_i , $i \in \{1, \dots, \ell\}$, corresponds to a feasible solution $h_i(S)$ of I . Moreover, the set

$$\left\{ h_i(S) : 1 \leq i \leq \ell \text{ and } S \text{ is a feasible solution of } I_i \right\}$$

contains at least one optimum solution for I . Informally speaking, a branch step splits problem I into subproblems $I_1, \dots, I_\ell$, possibly taking some (formally justified) greedy decisions.

2. The number ℓ is *small*, e.g., it is bounded by a function of $\mu(I)$ alone.
3. Furthermore, for every I_i , $i \in \{1, \dots, \ell\}$, we have that $\mu(I_i) \leq \mu(I) - c$ for some constant $c > 0$. In other words, in every branch we *substantially* simplify the instance at hand.

In a branching algorithm, we recursively apply branching steps to instances $I_1, I_2, \dots, I_\ell$, until they become simple or even trivial. Thus, we may see an execution of the algorithm as a *search tree*, where each recursive call corresponds to a node: the calls on instances $I_1, I_2, \dots, I_\ell$ are children of the call on instance I . The second and third conditions allow us to bound the number of nodes in this search tree, assuming that the instances with non-positive measure are simple. Indeed, the third condition allows us to bound the depth of the search tree in terms of the measure of the original instance, while the second condition controls the number of branches below every node. Because of these properties, search trees of this kind are often called *bounded search trees*. A branching algorithm with a cleverly chosen branching step often offers a drastic improvement over a straightforward exhaustive search.

We now present a typical scheme of applying the idea of bounded search trees in the design of parameterized algorithms. We first identify, in polynomial time, a small (typically of size that is constant, or bounded by a function of the parameter) subset S of elements of which at least one must be in *some* or *every* feasible solution of the problem. Then we solve $|S|$ subproblems: for each element e of S , create one subproblem in which we include e in the solution, and solve the remaining task with a reduced parameter value. We also say that we *branch* on the element of S that belongs to the solution. Such search trees are analyzed by measuring the drop of the parameter in each branch. If we ensure that the parameter (or some measure bounded by a function of the parameter) decreases in each branch by at least a constant value, then we will be able to bound the depth of the search tree by a function of the parameter, which results in an FPT algorithm.

It is often convenient to think of branching as of “guessing” the right branch. That is, whenever performing a branching step, the algorithm guesses the right part of an (unknown) solution in the graph, by trying all possibilities. What we need to ensure is that there will be a sequence of guesses that uncovers the whole solution, and that the total time spent on wrong guesses is not too large.

We apply the idea of bounded search trees to VERTEX COVER in Section 3.1. Section 3.2 briefly discusses methods of bounding the number of

nodes of a search tree. In Section 3.3 we give a branching algorithm for FEEDBACK VERTEX SET in undirected graphs. Section 3.4 presents an algorithm for a different parameterization of VERTEX COVER and shows how this algorithm implies algorithms for other parameterized problems such as ODD CYCLE TRANSVERSAL and ALMOST 2-SAT. Finally, in Section 3.5 we apply this technique to a non-graph problem, namely CLOSEST STRING.

3.1 VERTEX COVER

As the first example of branching, we use the strategy on VERTEX COVER. In Chapter 2 (Lemma 2.23), we gave a kernelization algorithm which in time $\mathcal{O}(n\sqrt{m})$ constructs a kernel on at most $2k$ vertices. Kernelization can be easily combined with a brute-force algorithm to solve VERTEX COVER in time $\mathcal{O}(n\sqrt{m} + 4^k k^{\mathcal{O}(1)})$. Indeed, there are at most $2^{2k} = 4^k$ subsets of size at most k in a $2k$ -vertex graph. Thus, by enumerating all vertex subsets of size at most k in the kernel and checking whether any of these subsets forms a vertex cover, we can solve the problem in time $\mathcal{O}(n\sqrt{m} + 4^k k^{\mathcal{O}(1)})$. We can easily obtain a better algorithm by branching. Actually, this algorithm was already presented in Chapter 1 under the cover of the BAR FIGHT PREVENTION problem.

Let (G, k) be a VERTEX COVER instance. Our algorithm is based on the following two simple observations.

- For a vertex v , any vertex cover must contain either v or *all* of its neighbors $N(v)$.
- VERTEX COVER becomes trivial (in particular, can be solved optimally in polynomial time) when the maximum degree of a graph is at most 1.

We now describe our recursive branching algorithm. Given an instance (G, k) , we first find a vertex $v \in V(G)$ of maximum degree in G . If v is of degree 1, then every connected component of G is an isolated vertex or an edge, and the instance has a trivial solution. Otherwise, $|N(v)| \geq 2$ and we recursively branch on two cases by considering

either v , or $N(v)$ in the vertex cover.

In the branch where v is in the vertex cover, we can delete v and reduce the parameter by 1. In the second branch, we add $N(v)$ to the vertex cover, delete $N[v]$ from the graph and decrease k by $|N(v)| \geq 2$.

The running time of the algorithm is bounded by

(the number of nodes in the search tree) $\times$ (time taken at each node).

Clearly, the time taken at each node is bounded by $n^{\mathcal{O}(1)}$. Thus, if $\tau(k)$ is the number of nodes in the search tree, then the total time used by the algorithm is at most $\tau(k)n^{\mathcal{O}(1)}$.

In fact, in every search tree $\mathcal{T}$ that corresponds to a run of a branching algorithm, every internal node of $\mathcal{T}$ has at least two children. Thus, if $\mathcal{T}$ has ℓ leaves, then the number of nodes in the search tree is at most $2\ell - 1$. Hence, to bound the running time of a branching algorithm, it is sufficient to bound the number of leaves in the corresponding search tree.

In our case, the tree $\mathcal{T}$ is the search tree of the algorithm when run with parameter k . Below its root, it has two subtrees: one for the same algorithm run with parameter $k - 1$, and one recursive call with parameter at most $k - 2$. The same pattern occurs deeper in $\mathcal{T}$. This means that if we define a function $T(k)$ using the recursive formula

$$T(i) = \begin{cases} T(i-1) + T(i-2) & \text{if } i \geq 2, \\ 1 & \text{otherwise,} \end{cases}$$

then the number of leaves of $\mathcal{T}$ is bounded by $T(k)$.

Using induction on k , we prove that $T(k)$ is bounded by 1.6181^k . Clearly, this is true for $k = 0$ and $k = 1$, so let us proceed for $k \geq 2$:

$$\begin{aligned} T(k) &= T(k-1) + T(k-2) \leq 1.6181^{k-1} + 1.6181^{k-2} \\ &\leq 1.6181^{k-2}(1.6181 + 1) \leq 1.6181^{k-2}(1.6181)^2 \leq 1.6181^k. \end{aligned}$$

This proves that the number of leaves is bounded by 1.6181^k . Combined with kernelization, we arrive at an algorithm solving VERTEX COVER in time $\mathcal{O}(n\sqrt{m} + 1.6181^k k^{\mathcal{O}(1)})$.

A natural question is how did we know that 1.6181^k is a solution to the above recurrence. Suppose that we are looking for an upper bound on function $T(k)$ of the form $T(k) \leq c \cdot \lambda^k$, where $c > 0$, $\lambda > 1$ are some constants. Clearly, we can set constant c so that the initial conditions in the definition of $T(k)$ are satisfied. Then, we are left with proving, using induction, that this bound holds for every k . This boils down to proving that

$$c \cdot \lambda^k \geq c \cdot \lambda^{k-1} + c \cdot \lambda^{k-2}, \quad (3.1)$$

since then we will have

$$T(k) = T(k-1) + T(k-2) \leq c \cdot \lambda^{k-1} + c \cdot \lambda^{k-2} \leq c \cdot \lambda^k.$$

Observe that (3.1) is equivalent to $\lambda^2 \geq \lambda + 1$, so it makes sense to look for the lowest possible value of λ for which this inequality is satisfied; this is actually the one for which equality holds. By solving equation $\lambda^2 = \lambda + 1$ for $\lambda > 1$, we find that $\lambda = \frac{1+\sqrt{5}}{2} < 1.6181$, so for this value of λ the inductive proof works.

The running time of the above algorithm can be easily improved using the following argument, whose proof we leave as Exercise 3.1.

Proposition 3.1. *VERTEX COVER can be solved optimally in polynomial time when the maximum degree of a graph is at most 2.*

Thus, we branch only on the vertices of degree at least 3, which immediately brings us to the following upper bound on the number of leaves in a search tree:

$$T(k) = \begin{cases} T(k-1) + T(k-3) & \text{if } k \geq 3, \\ 1 & \text{otherwise.} \end{cases}$$

Again, an upper bound of the form $c \cdot \lambda^k$ for the above recursive function can be obtained by finding the largest root of the polynomial equation $\lambda^3 = \lambda^2 + 1$. Using standard mathematical techniques (and/or symbolic algebra packages) the root is estimated to be at most 1.4656. Combined with kernelization, this gives us the following theorem.

Theorem 3.2. *VERTEX COVER can be solved in time $\mathcal{O}(n\sqrt{m} + 1.4656^k k^{\mathcal{O}(1)})$.*

Can we apply a similar strategy for graphs of vertex degree at most 3? Well, this becomes more complicated as VERTEX COVER is NP-hard on this class of graphs. But there are more involved branching strategies, and there are faster branching algorithms than the one given in Theorem 3.2.

3.2 How to solve recursive relations

For algorithms based on the bounded search tree technique, we need to bound the number of nodes in the search tree to obtain an upper bound on the running time of the algorithm. For this, recurrence relations are used. The most common case in parameterized branching algorithms is when we use linear recurrences with constant coefficients. There exists a standard technique to bound the number of nodes in the search tree for this case. If the algorithm solves a problem of size n with parameter k and calls itself recursively on problems with decreased parameters $k - d_1, k - d_2, \dots, k - d_p$, then $(d_1, d_2, \dots, d_p)$ is called the *branching vector* of this recursion. For example, we used a branching vector $(1, 2)$ to obtain the first algorithm for VERTEX COVER in the previous section, and a branching vector $(1, 3)$ for the second one. For a branching vector $(d_1, d_2, \dots, d_p)$, the upper bound $T(k)$

on the number of leaves in the search tree is given by the following linear recurrence:

$$T(k) = T(k - d_1) + T(k - d_2) + \cdots + T(k - d_p).$$

Again, for $k < d$, where $d = \max_{i=1,2,\dots,p} d_i$, we put the initial condition $T(k) = 1$. Assuming that new subproblems with smaller parameters can be computed in time polynomial in n , the running time of such recursive algorithm is $T(k) \cdot n^{\mathcal{O}(1)}$.

If we now look for an upper bound of the form $T(k) \leq c \cdot \lambda^k$, then the inductive step boils down to proving the following inequality:

$$\lambda^k \geq \lambda^{k-d_1} + \lambda^{k-d_2} + \cdots + \lambda^{k-d_p}. \quad (3.2)$$

Inequality (3.2) can be rewritten as $P(\lambda) \geq 0$, where

$$P(\lambda) = \lambda^d - \lambda^{d-d_1} - \lambda^{d-d_2} - \cdots - \lambda^{d-d_p} \quad (3.3)$$

is the *characteristic polynomial* of the recurrence for $T(k)$ (recall $d = \max_{i=1,2,\dots,p} d_i$). Using standard techniques of calculus it is not hard to show that if a polynomial P has a form as in (3.3), then P has a unique positive root λ_0 , and moreover $P(\lambda) < 0$ for $0 < \lambda < \lambda_0$ and $P(\lambda) > 0$ for $\lambda > \lambda_0$. This means that λ_0 is the best possible value that can be used in an upper bound for $T(k)$. In the bibliographic notes we provide further references to the relevant literature on solving recursive formulas.

The root λ_0 is often called the *branching number* corresponding to the branching vector $(d_1, d_2, \dots, d_p)$. Hence, the running time of the considered branching algorithm is bounded by $\lambda_0^k n^{\mathcal{O}(1)}$. In Table 3.1, we give branching numbers corresponding to branching vectors (i, j) for $i, j \in \{1, \dots, 6\}$.

(i, j)	1	2	3	4	5	6
1	2.0000	1.6181	1.4656	1.3803	1.3248	1.2852
2		1.4143	1.3248	1.2721	1.2366	1.2107
3			1.2560	1.2208	1.1939	1.1740
4				1.1893	1.1674	1.1510
5					1.1487	1.1348
6						1.1225

Table 3.1: A table of branching numbers (rounded up)

Two natural questions arise:

- How good is the estimation of $T(k)$ using the exponent of the corresponding branching number?
- How well does $T(k)$ estimate the actual size of the search tree?

The answer to the first question is “it is good”: up to a polynomial factor, the estimation is tight. The second question is much more difficult, since the actual way a branching procedure explores the search space may be more complex than our estimation of its behavior using recursive formulas. If, say, a branching algorithm uses several ways of branching into subproblems (so-called branching rules) that correspond to different branching vectors, and/or is combined with local reduction rules, then so far we do not know how to estimate the running time better than by using the branching number corresponding to the worst branching vector. However, the delicate interplay between different branching rules and reduction rules may lead to a much smaller tree than what follows from our imprecise estimations.

3.3 FEEDBACK VERTEX SET

For a given graph G and a set $X \subseteq V(G)$, we say that X is a *feedback vertex set* of G if $G - X$ is an acyclic graph (i.e., a forest). In the FEEDBACK VERTEX SET problem, we are given an undirected graph G and a nonnegative integer k , and the objective is to determine whether there exists a feedback vertex set of size at most k in G . In this section, we give a branching algorithm solving FEEDBACK VERTEX SET in time $k^{\mathcal{O}(k)} \cdot n^{\mathcal{O}(1)}$.

It is more convenient for us to consider this problem in the more general setting of *multigraphs*, where the input graph G may contain multiple edges and loops. We note that both a double edge and a loop are cycles. We also use the convention that a loop at a vertex v contributes 2 to the degree of v .

We start with some simple reduction rules that clean up the graph. At any point, we use the lowest-numbered applicable rule. We first deal with the multigraph parts of G . Observe that any vertex with a loop needs to be contained in any solution set X .

Reduction FVS.1. If there is a loop at a vertex v , delete v from the graph and decrease k by 1.

Moreover, notice that the multiplicity of a multiple edge does not influence the set of feasible solutions to the instance (G, k) .

Reduction FVS.2. If there is an edge of multiplicity larger than 2, reduce its multiplicity to 2.

We now reduce vertices of low degree. Any vertex of degree at most 1 does not participate in any cycle in G , so it can be deleted.

Reduction FVS.3. If there is a vertex v of degree at most 1, delete v .

Concerning vertices of degree 2, observe that, instead of including into the solution any such vertex, we may as well include one of its neighbors. This leads us to the following reduction.

Reduction FVS.4. If there is a vertex v of degree 2, delete v and connect its two neighbors by a new edge.

Two remarks are in place. First, a vertex v in Reduction FVS.4 cannot have a loop, as otherwise Reduction FVS.1 should be triggered on v instead. This ensures that the neighbors of v are distinct from v , hence the rule may be applied and the safeness argument works. Second, it is possible that v is incident to a double edge; in this case, the reduction rule deletes v and adds a loop to a sole “double” neighbor of v . Observe that in this case Reduction FVS.1 will trigger subsequently on this neighbor.

We remark that after exhaustively applying these four reduction rules, the resulting graph G

- (P1) contains no loops,
- (P2) has only single and double edges, and
- (P3) has minimum vertex degree at least 3.

Moreover, all rules are trivially applicable in polynomial time. From now on we assume that in the input instance (G, k) , graph G satisfies properties (P1)–(P3).

We remark that for the algorithm in this section, we do not need properties (P1) and (P2). However, we will need these properties later for the kernelization algorithm in Section 9.1.

Finally, we need to add a rule that stops the algorithm if we already exceeded our budget.

Reduction FVS.5. If $k < 0$, terminate the algorithm and conclude that (G, k) is a no-instance.

The intuition behind the algorithm we are going to present is as follows. Observe that if X is a feedback vertex set of G , then $G - X$ is a forest. However, $G - X$ has at most $|V(G)| - |X| - 1$ edges and thus $G - X$ cannot have “many” vertices of high degree. Thus, if we pick some $f(k)$ vertices with the highest degrees in the graph, then every solution of size at most k must contain one of these high-degree vertices. In what follows we make this intuition work.

Let $(v_1, v_2, \dots, v_n)$ be a descending ordering of $V(G)$ according to vertex degrees, i.e., $d(v_1) \geq d(v_2) \geq \dots \geq d(v_n)$. Let $V_{3k} = \{v_1, \dots, v_{3k}\}$. Let us recall that the minimum vertex degree of G is at least 3. Our algorithm for FEEDBACK VERTEX SET is based on the following lemma.

Lemma 3.3. *Every feedback vertex set in G of size at most k contains at least one vertex of V_{3k} .*

Proof. To prove this lemma we need the following simple claim.

Claim 3.4. *For every feedback vertex set X of G ,*

$$\sum_{v \in X} (d(v) - 1) \geq |E(G)| - |V(G)| + 1.$$

Proof. Graph $F = G - X$ is a forest and thus the number of edges in F is at most $|V(G)| - |X| - 1$. Every edge of $E(G) \setminus E(F)$ is incident to a vertex of X . Hence

$$\sum_{v \in X} d(v) + |V(G)| - |X| - 1 \geq |E(G)|.$$

┘

Targeting a contradiction, let us assume that there is a feedback vertex set X of size at most k such that $X \cap V_{3k} = \emptyset$. By the choice of V_{3k} , for every $v \in X$, $d(v)$ is at most the minimum of vertex degrees from V_{3k} . Because $|X| \leq k$, by Claim 3.4 we have that

$$\sum_{i=1}^{3k} (d(v_i) - 1) \geq 3 \cdot \left(\sum_{v \in X} (d(v) - 1) \right) \geq 3 \cdot (|E(G)| - |V(G)| + 1).$$

In addition, we have that $X \subseteq V(G) \setminus V_{3k}$, and hence

$$\sum_{i > 3k} (d(v_i) - 1) \geq \sum_{v \in X} (d(v) - 1) \geq (|E(G)| - |V(G)| + 1).$$

Therefore,

$$\sum_{i=1}^n (d(v_i) - 1) \geq 4 \cdot (|E(G)| - |V(G)| + 1).$$

However, observe that $\sum_{i=1}^n d(v_i) = 2|E(G)|$: every edge is counted twice, once for each of its endpoints. Thus we obtain

$$4 \cdot (|E(G)| - |V(G)| + 1) \leq \sum_{i=1}^n (d(v_i) - 1) = 2|E(G)| - |V(G)|,$$

which implies that $2|E(G)| < 3|V(G)|$. However, this contradicts the fact that every vertex of G is of degree at least 3. $\square$

We use Lemma 3.3 to obtain the following algorithm for FEEDBACK VERTEX SET.

Theorem 3.5. *There exists an algorithm for FEEDBACK VERTEX SET running in time $(3k)^k \cdot n^{\mathcal{O}(1)}$.*

Proof. Given an undirected graph G and an integer $k \geq 0$, the algorithm works as follows. It first applies Reductions FVS.1, FVS.2, FVS.3, FVS.4,

and FVS.5 exhaustively. As the result, we either already conclude that we are dealing with a no-instance, or obtain an equivalent instance (G', k') such that G' has minimum degree at least 3 and $k' \leq k$. If G' is empty, then we conclude that we are dealing with a yes-instance, as $k' \geq 0$ and an empty set is a feasible solution. Otherwise, let $V_{3k'}$ be the set of $3k'$ vertices of G' with largest degrees. By Lemma 3.3, every solution X to the FEEDBACK VERTEX SET instance (G', k') contains at least one vertex from $V_{3k'}$. Therefore, we branch on the choice of one of these vertices, and for every vertex $v \in V_{3k'}$, we recursively apply the algorithm to solve the FEEDBACK VERTEX SET instance $(G' - v, k' - 1)$. If one of these branches returns a solution X' , then clearly $X' \cup \{v\}$ is a feedback vertex set of size at most k' for G' . Else, we return that the given instance is a no-instance.

At every recursive call we decrease the parameter by 1, and thus the height of the search tree does not exceed k' . At every step we branch in at most $3k'$ subproblems. Hence the number of nodes in the search tree does not exceed $(3k')^{k'} \leq (3k)^k$. This concludes the proof. $\square$

3.4 VERTEX COVER ABOVE LP

Recall the integer linear programming formulation of VERTEX COVER and its relaxation LPVC(G):

$$\begin{aligned} \min \quad & \sum_{v \in V(G)} x_v \\ \text{subject to} \quad & x_u + x_v \geq 1 \quad \text{for every } uv \in E(G), \\ & 0 \leq x_v \leq 1 \quad \text{for every } v \in V(G). \end{aligned}$$

These programs were discussed in Section 2.2.1. If the minimum value of LPVC(G) is $\text{vc}^*(G)$, then the size of a minimum vertex cover is at least $\text{vc}^*(G)$. This leads to the following parameterization of VERTEX COVER, which we call VERTEX COVER ABOVE LP: Given a graph G and an integer k , we ask for a vertex cover of G of size at most k , but instead of seeking an FPT algorithm parameterized by k as for VERTEX COVER, the parameter now is $k - \text{vc}^*(G)$. In other words, the goal of this section is to design an algorithm for VERTEX COVER on an n -vertex graph G with running time $f(k - \text{vc}^*(G)) \cdot n^{\mathcal{O}(1)}$ for some computable function f .

The parameterization by $k - \text{vc}^*(G)$ falls into a more general theme of *above guarantee parameterization*, where, instead of parameterizing purely by the solution size k , we look for some (computable in polynomial time) lower bound ℓ for the solution size, and use a more refined parameter $k - \ell$, the *excess* above the lower bound. Such a parameterization makes perfect sense in problems where the solution size is often quite large and, consequently, FPT algorithms in parameterization by k may not be very efficient. In the VERTEX COVER problem, the result of Section 2.5 — a kernel with at

most $2k$ vertices — can be on one hand seen as a very good result, but on the other hand can be an evidence that the solution size parameterization for VERTEX COVER may not be the most meaningful one, as the parameter can be quite large. A more striking example is the MAXIMUM SATISFIABILITY problem, studied in Section 2.3.2: here an instance with at least $2k$ clauses is trivially a yes-instance. In Section 9.2 we study an above guarantee parameterization of a variant of MAXIMUM SATISFIABILITY, namely MAX-ER-SAT. In Exercise 9.3 we also ask for an FPT algorithm for MAXIMUM SATISFIABILITY parameterized by $k - m/2$, where m is the number of clauses in the input formula. The goal of this section is to study the above guarantee parameterization of VERTEX COVER, where the lower bound is the cost of an optimum solution to an LP relaxation.

Before we describe the algorithm, we fix some notation. By *optimum solution* $\mathbf{x} = (x_v)_{v \in V(G)}$ to LPVC(G), we mean a feasible solution with $1 \geq x_v \geq 0$ for all $v \in V(G)$ that minimizes the objective function (sometimes called the *cost*) $\mathbf{w}(\mathbf{x}) = \sum_{v \in V(G)} x_v$. By Proposition 2.24, for any graph G there exists an optimum half-integral solution of LPVC(G), i.e., a solution with $x_v \in \{0, \frac{1}{2}, 1\}$ for all $v \in V(G)$, and such a solution can be found in polynomial time.

Let $\text{vc}(G)$ denote the size of a minimum vertex cover of G . Clearly, $\text{vc}(G) \geq \text{vc}^*(G)$. For a half-integral solution $\mathbf{x} = (x_v)_{v \in V(G)}$ and $i \in \{0, \frac{1}{2}, 1\}$, we define $V_i^{\mathbf{x}} = \{v \in V : x_v = i\}$. We also say that $\mathbf{x} = \{x_v\}_{v \in V(G)}$ is *all- $\frac{1}{2}$ -solution* if $x_v = \frac{1}{2}$ for every $v \in V(G)$. Because the all- $\frac{1}{2}$ -solution is a feasible solution, we have that $\text{vc}^*(G) \leq \frac{|V(G)|}{2}$. Furthermore, we define the *measure* of an instance (G, k) to be our parameter of interest $\mu(G, k) = k - \text{vc}^*(G)$.

Recall that in Section 2.5 we have developed Reduction VC.4 for VERTEX COVER. This reduction, if restricted to half-integral solutions, can be stated as follows: for an optimum half-integral LPVC(G) solution $\mathbf{x}$, we (a) conclude that the input instance (G, k) is a no-instance if $\mathbf{w}(\mathbf{x}) > k$; and (b) delete $V_0^{\mathbf{x}} \cup V_1^{\mathbf{x}}$ and decrease k by $|V_1^{\mathbf{x}}|$ otherwise. As we are now dealing with measure $\mu(G, k) = k - \text{vc}^*(G)$, we need to understand how this parameter changes under Reduction VC.4.

Lemma 3.6. *Assume an instance (G', k') is created from an instance (G, k) by applying Reduction VC.4 to a half-integral optimum solution $\mathbf{x}$. Then $\text{vc}^*(G) - \text{vc}^*(G') = \text{vc}(G) - \text{vc}(G') = |V_1^{\mathbf{x}}| = k - k'$. In particular, $\mu(G', k') = \mu(G, k)$.*

We remark that, using Exercise 2.25, the statement of Lemma 3.6 is true for any optimum solution $\mathbf{x}$, not only a half-integral one (see Exercise 3.19). However, the proof for a half-integral solution is slightly simpler, and, thanks to Proposition 2.24, we may work only with half-integral solutions.

Proof (of Lemma 3.6). Observe that every edge of G incident to $V_0^{\mathbf{x}}$ has its second endpoint in $V_1^{\mathbf{x}}$. Hence, we have the following:

- For every vertex cover Y of G' , $Y \cup V_1^{\mathbf{x}}$ is a vertex cover of G and, consequently, $\text{vc}(G) \leq \text{vc}(G') + |V_1^{\mathbf{x}}|$.
- For every feasible solution $\mathbf{y}'$ to $\text{LPVC}(G')$, if we define a vector $\mathbf{y} = (y_v)_{v \in V(G)}$ as $y_v = y'_v$ for every $v \in V(G')$ and $y_v = x_v$ for every $v \in V(G) \setminus V(G')$, then we obtain a feasible solution to $\text{LPVC}(G)$ of cost $\mathbf{w}(\mathbf{y}') + |V_1^{\mathbf{x}}|$; consequently, $\text{vc}^*(G) \leq \text{vc}^*(G') + |V_1^{\mathbf{x}}|$.

Now it suffices to prove the reversed versions of the two inequalities obtained above. Theorem 2.19 ensures that $\text{vc}(G') \leq \text{vc}(G) - |V_1^{\mathbf{x}}|$. Moreover, since $\mathbf{x}$ restricted to $V(G')$ is a feasible solution to $\text{LPVC}(G')$ of cost $\text{vc}^*(G) - |V_1^{\mathbf{x}}|$, we have $\text{vc}^*(G') \leq \text{vc}^*(G) - |V_1^{\mathbf{x}}|$. $\square$

In Theorem 2.21 we simply solved $\text{LPVC}(G)$, and applied Reduction VC.4 to obtain a kernel with at most $2k$ vertices. In this section we would like to do something slightly stronger: to apply Reduction VC.4 as long as there exists some half-integral optimum solution $\mathbf{x}$ that is not the all- $\frac{1}{2}$ -solution. Luckily, by a simple self-reduction trick, we can always detect such solutions.

Lemma 3.7. *Given a graph G , one can in $\mathcal{O}(mn^{3/2})$ time find an optimum solution to $\text{LPVC}(G)$ which is not the all- $\frac{1}{2}$ -solution, or correctly conclude that the all- $\frac{1}{2}$ -solution is the unique optimum solution to $\text{LPVC}(G)$. Moreover, the returned optimum solution is half-integral.*

Proof. First, use Proposition 2.24 to solve $\text{LPVC}(G)$, obtaining an optimum half-integral solution $\mathbf{x}$. If $\mathbf{x}$ is not the all- $\frac{1}{2}$ -solution, then return $\mathbf{x}$. Otherwise, proceed as follows.

For every $v \in V(G)$, use Proposition 2.24 again to solve $\text{LPVC}(G - v)$, obtaining an optimum half-integral solution $\mathbf{x}^v$. Define a vector $\mathbf{x}^{v,\circ}$ as $\mathbf{x}^v$, extended with a value $x_v = 1$. Note that $\mathbf{x}^{v,\circ}$ is a feasible solution to $\text{LPVC}(G)$ of cost $\mathbf{w}(\mathbf{x}^{v,\circ}) = \mathbf{w}(\mathbf{x}^v) + 1 = \text{vc}^*(G - v) + 1$. Thus, if for some $v \in V(G)$ we have $\mathbf{w}(\mathbf{x}^v) = \text{vc}^*(G - v) \leq \text{vc}^*(G) - 1$, then $\mathbf{x}^{v,\circ}$ is an optimum solution for $\text{LPVC}(G)$. Moreover, $\mathbf{x}^{v,\circ}$ is half-integral, but is not equal to the all- $\frac{1}{2}$ -solution due to the value at vertex v . Hence, we may return $\mathbf{x}^{v,\circ}$.

We are left with the case

$$\text{vc}^*(G - v) > \text{vc}^*(G) - 1 \quad \text{for every } v \in V(G). \quad (3.4)$$

We claim that in this case the all- $\frac{1}{2}$ -solution is the unique solution to $\text{LPVC}(G)$; note that such a claim would conclude the proof of the lemma, as the computation time used so far is bounded by $\mathcal{O}(mn^{3/2})$ ($n+1$ applications of Proposition 2.24). Observe that, due to Proposition 2.24, both $2\text{vc}^*(G - v)$ and $2\text{vc}^*(G)$ are integers and, consequently, we obtain the following strengthening of (3.4):

$$\text{vc}^*(G - v) \geq \text{vc}^*(G) - \frac{1}{2} \quad \text{for every } v \in V(G). \quad (3.5)$$

By contradiction, let $\mathbf{x}$ be an optimum solution to $\text{LPVC}(G)$ that is not the all- $\frac{1}{2}$ -solution. As the all- $\frac{1}{2}$ -solution is an optimum solution to $\text{LPVC}(G)$,

for some $v \in V(G)$ we have $x_v > \frac{1}{2}$. However, then $\mathbf{x}$, restricted to $G - v$, is a feasible solution to $\text{LPVC}(G - v)$ of cost $\mathbf{w}(\mathbf{x}) - x_v = \text{vc}^*(G) - x_v < \text{vc}^*(G) - \frac{1}{2}$, a contradiction to (3.5). $\square$

Lemma 3.7 allows us to enhance Reduction VC.4 to the following form.

Reduction VC.5. Invoke the algorithm of Lemma 3.7 and, if an optimum solution $\mathbf{x}$ is returned, then apply Reduction VC.4 to G and solution $\mathbf{x}$.

If Reduction VC.5 is not applicable, then we know not only that the all- $\frac{1}{2}$ -solution is an optimum solution to $\text{LPVC}(G)$, but also that this is the *unique* optimum solution. This property is crucial for our branching algorithm, encapsulated in the next theorem.

Theorem 3.8. *There exists an algorithm solving VERTEX COVER ABOVE LP in time $4^{k - \text{vc}^*(G)} \cdot n^{\mathcal{O}(1)}$.*

Proof. Our algorithm for VERTEX COVER ABOVE LP is almost the same as the branching algorithm described for VERTEX COVER in Section 3.1. After Reduction VC.5 is applied exhaustively, we pick an arbitrary vertex v in the graph and branch on it. In other words, in one branch, we add v into the vertex cover, decrease k by 1, and delete v from the graph, and in the other branch, we add $N(v)$ into the vertex cover, decrease k by $|N(v)|$, and delete $N[v]$ from the graph. The correctness of this algorithm follows from the safeness of Reduction VC.5 and the fact that the branching is exhaustive.

Although the algorithm is very similar to the one of Section 3.1, we analyze our algorithm in a completely different way, using the measure $\mu(G, k) = k - \text{vc}^*(G)$. In Lemma 3.6 we have proved that Reduction VC.4 does not change the measure; as Reduction VC.5 is in fact an application of Reduction VC.4 to a specific half-integral solution $\mathbf{x}$, the same holds for Reduction VC.5. Hence, it remains to analyze how the measure changes in a branching step.

Consider first the case when we pick v to the vertex cover, obtaining an instance $(G', k') = (G - v, k - 1)$. We claim that $\text{vc}^*(G') \geq \text{vc}^*(G) - \frac{1}{2}$. Suppose that this is not the case. Let $\mathbf{x}'$ be an optimum solution to $\text{LPVC}(G')$. We have $\mathbf{w}(\mathbf{x}') \leq \text{vc}^*(G) - 1$ and we can obtain an optimum solution $\mathbf{x}''$ to $\text{LPVC}(G)$ by extending $\mathbf{x}'$ with a new variable $x_v = 1$ corresponding to v . But this contradicts our assumption that the all- $\frac{1}{2}$ -solution is the unique optimum solution to $\text{LPVC}(G)$. Hence, $\text{vc}^*(G') \geq \text{vc}^*(G) - \frac{1}{2}$, which implies that $\mu(G', k') \leq \mu(G, k) - \frac{1}{2}$.

In the second case, let $p = |N(v)|$. Recall that here the new instance is $(G', k') = (G - N[v], k - p)$. We claim that $\text{vc}^*(G') \geq \text{vc}^*(G) - p + \frac{1}{2}$, which would imply again $\mu(G', k') \leq \mu(G, k) - \frac{1}{2}$. Assume the contrary: let $\mathbf{x}'$ be an optimum solution to $\text{LPVC}(G')$ and suppose that $\mathbf{w}(\mathbf{x}') \leq \text{vc}^*(G) - p$. Define a feasible solution $\mathbf{x}''$ to $\text{LPVC}(G)$ by extending $\mathbf{x}'$ with $x_v = 0$ and $x_u = 1$ for every $u \in N(v)$. Clearly, $\mathbf{w}(\mathbf{x}'') = \mathbf{w}(\mathbf{x}') + p$, thus $\mathbf{x}''$ is an optimum solution to $\text{LPVC}(G)$, contradicting the assumption that the all- $\frac{1}{2}$ -solution is the unique optimum solution.

We have shown that the preprocessing rule does not increase the measure $\mu(G, k)$, and that the branching step results in a $(\frac{1}{2}, \frac{1}{2})$ decrease in $\mu(G, k)$. As a result, we obtain recurrence $T(\mu) \leq 2T(\mu - \frac{1}{2})$ for the number of leaves in the search tree. This recurrence solves to $4^\mu = 4^{k - \text{vc}^*(G)}$, and we obtain a $4^{(k - \text{vc}^*(G))} \cdot n^{\mathcal{O}(1)}$ -time algorithm for VERTEX COVER ABOVE LP. $\square$

Let us now consider a different lower bound on the size of a minimum vertex cover in a graph, namely the size of a maximum matching. Observe that, if graph G contains a matching M , then for $k < |M|$ the instance (G, k) is a trivial no-instance of VERTEX COVER. Thus, for a graph G with a large maximum matching (e.g., when G has a perfect matching) the FPT algorithm for VERTEX COVER of Section 3.1 is not practical, as in this case k has to be quite large.

This leads to a second above guarantee variant of the VERTEX COVER problem, namely the VERTEX COVER ABOVE MATCHING problem. On input, we are given an undirected graph G , a maximum matching M and a positive integer k . As in VERTEX COVER, the objective is to decide whether G has a vertex cover of size at most k ; however, now the parameter is $k - |M|$. By the weak duality of linear programs, it follows that $\text{vc}^*(G) \geq |M|$ (see Exercise 2.24 and the corresponding hint for a self-contained argument) and thus we have that $k - \text{vc}^*(G) \leq k - |M|$. Consequently, any parameterized algorithm for VERTEX COVER ABOVE LP is also a parameterized algorithm for VERTEX COVER ABOVE MATCHING, and Theorem 3.8 yields the following interesting observation.

Theorem 3.9. VERTEX COVER ABOVE MATCHING *can be solved in time $4^{k - |M|} \cdot n^{\mathcal{O}(1)}$.*

The VERTEX COVER ABOVE MATCHING problem has been at the center of many developments in parameterized algorithms. The reason is that faster algorithms for this problem also yield faster algorithms for a host of other problems. Just to show its importance, we design algorithms for ODD CYCLE TRANSVERSAL and ALMOST 2-SAT by making use of the algorithm for VERTEX COVER ABOVE MATCHING.

A subset $X \subseteq V(G)$ is called an *odd cycle transversal* of G if $G - X$ is a bipartite graph. In ODD CYCLE TRANSVERSAL, we are given an undirected graph G with a positive integer k , and the goal is to determine whether G has an odd cycle transversal of size at most k .

For a given graph G , we define a new graph $\tilde{G}$ as follows. Let $V_i = \{u_i : u \in V(G)\}$ for $i \in \{1, 2\}$. The vertex set $V(\tilde{G})$ consists of two copies of $V(G)$, i.e. $V(\tilde{G}) = V_1 \cup V_2$, and

$$E(\tilde{G}) = \{u_1 u_2 : u \in V(G)\} \cup \{u_i v_i : uv \in E(G), i \in \{1, 2\}\}.$$

In other words, $\tilde{G}$ is obtained by taking two disjoint copies of G and by adding a perfect matching such that the endpoints of every matching edge are the

copies of the same vertex. In particular, $M = \{u_1u_2 : u \in V(G)\}$ is a perfect matching in $\tilde{G}$ and $|M| = n$. The graph $\tilde{G}$ has some interesting properties which are useful for designing an algorithm for ODD CYCLE TRANSVERSAL.

Lemma 3.10. *Let G be a graph on n vertices. Then G has an odd cycle transversal of size at most k if and only if $\tilde{G}$ has a vertex cover of size at most $n + k$.*

Proof. Let X be an odd cycle transversal of size at most k in G , and let (A, B) be a bipartition of $G - X$. For $i \in \{1, 2\}$, let A_i, B_i and X_i be the copies of A, B and X in V_i , respectively. Observe that $A_1 \cup B_2$ is an independent set in $\tilde{G}$ and thus $A_2 \cup B_1 \cup X_1 \cup X_2$ is a vertex cover of $\tilde{G}$. However,

$$(|A_2| + |B_1|) + |X_1| + |X_2| \leq (n - k) + k + k \leq n + k.$$

This completes the forward direction of the proof.

In the other direction, let Y be a vertex cover of $\tilde{G}$ of size at most $n + k$. Then $I = V(\tilde{G}) \setminus Y$ is an independent set of $\tilde{G}$ of size at least $n - k$. Let $\tilde{A} = I \cap V_1$ and $\tilde{B} = I \cap V_2$. Let A and B denote the vertices corresponding to $\tilde{A}$ and $\tilde{B}$ in $V(G)$, respectively. Since $\tilde{A} \cup \tilde{B}$ is independent, we have that $A \cap B = \emptyset$. Therefore, $A \cup B$ is of size at least $n - k$. Moreover, $G[A \cup B]$ is bipartite, as $G[A]$ and $G[B]$ are independent sets. Thus, $V(G) \setminus (A \cup B)$ is an odd cycle transversal of size at most k for G . This completes the proof. $\square$

Using Lemma 3.10 and Theorem 3.9 we get the following result for ODD CYCLE TRANSVERSAL.

Theorem 3.11. ODD CYCLE TRANSVERSAL can be solved in time $4^k n^{\mathcal{O}(1)}$.

Proof. Given an input (G, k) to ODD CYCLE TRANSVERSAL, we first construct the graph $\tilde{G}$. Let $M = \{u_1u_2 : u \in V(G)\}$ be a perfect matching of $\tilde{G}$. Now we apply Theorem 3.9 to the instance $(G, M, n + k)$ to check in time $4^{n+k-n} \cdot n^{\mathcal{O}(1)} = 4^k \cdot n^{\mathcal{O}(1)}$ whether $\tilde{G}$ has a vertex cover of size at most $n + k$, where $n = |V(G)|$. If a vertex cover of size at most $n + k$ in $\tilde{G}$ is found, then using Lemma 3.10 we obtain an odd cycle transversal of size at most k in G . Otherwise, by Lemma 3.10, G has no odd cycle transversal of size at most k . $\square$

In fact, the fastest currently known FPT algorithm for ODD CYCLE TRANSVERSAL [328, 373] uses an improved version of the algorithm described in Theorem 3.9 as a subroutine. We will give a $3^k n^{\mathcal{O}(1)}$ -time algorithm for this problem in the next chapter using a different technique, called iterative compression.

Next, we give an algorithm for ALMOST 2-SAT using the $4^k n^{\mathcal{O}(1)}$ -time algorithm for VERTEX COVER ABOVE MATCHING. In the ALMOST 2-SAT problem, we are given a 2-CNF formula φ and an integer k , and the question is whether one can delete at most k clauses from φ to make it satisfiable.

In a variable-deletion variant, called **VARIABLE DELETION ALMOST 2-SAT**, we instead allow deleting at most k variables; a variable is deleted together with all clauses containing it. (One can think of deleting a variable as setting it both to true and false at the same time, in this way satisfying all the clauses containing it.) Exercises 3.21 and 3.22 ask you to prove that these two variants are equivalent, and an $f(k)n^{\mathcal{O}(1)}$ -time algorithm for one of them implies an $f(k)n^{\mathcal{O}(1)}$ -time algorithm for the other one (with the same function f). Hence, it suffices to focus only on the variable-deletion variant.

Theorem 3.12. **VARIABLE DELETION ALMOST 2-SAT** can be solved in $4^k n^{\mathcal{O}(1)}$ time.

Proof. We proceed similarly as in the case of **ODD CYCLE TRANSVERSAL**: in polynomial time, we construct a **VERTEX COVER ABOVE MATCHING** instance $(G, M, |M| + k)$ that is equivalent to the input **VARIABLE DELETION ALMOST 2-SAT** instance (φ, k) . Then the theorem follows from an application of the algorithm of Theorem 3.9 to the instance $(G, M, |M| + k)$.

We construct the graph G as follows. For every variable x , and for every literal $\ell \in \{x, \neg x\}$, we construct a vertex v_ℓ , and connect the vertices v_x and $v_{\neg x}$ by an edge. Let M be a set of edges defined in this step; note that M is a matching in G . The intuitive meaning of vertices v_ℓ is that picking v_ℓ into a vertex cover corresponds to valuating the variable of ℓ so that ℓ is true. For every edge $v_x v_{\neg x} \in M$, we expect to take one endpoint into a vertex cover, which corresponds to setting the value of variable x . However, in k edges $v_x v_{\neg x} \in M$ we are allowed to take both endpoints into a vertex cover, corresponding to deleting x or, equivalently, setting x both to true and false at once.

To encode clauses, we proceed as follows. For every binary clause $\ell_1 \vee \ell_2$, we add an edge $v_{\ell_1} v_{\ell_2}$, so that either v_{ℓ_1} or v_{ℓ_2} needs to be taken into a vertex cover. For every unary clause with literal ℓ , we add a new vertex of degree 1, adjacent to v_ℓ .

This concludes the construction of the graph G and the instance $(G, M, |M| + k)$. We refer to Fig. 3.1 for an illustration of the construction. We claim that (φ, k) is a yes-instance of **VARIABLE DELETION ALMOST 2-SAT** if and only if $(G, M, |M| + k)$ is a yes-instance of **VERTEX COVER ABOVE MATCHING**.

In one direction, let X be a set of at most k variables of φ such that $\varphi - X$, the formula φ with all variables of X deleted, has a satisfying assignment ψ . We define a set $Y \subseteq V(G)$ as follows. First, for every $x \in X$, we put both v_x and $v_{\neg x}$ into Y . Second, for every variable $x \notin X$, we put v_x into Y if $\psi(x) = \top$ and otherwise, if $\psi(x) = \perp$, we put $v_{\neg x}$ into Y . Clearly $|Y| = |M| + |X| \leq |M| + k$.

It remains to argue that Y is a vertex cover of G . By construction, Y contains at least one endpoint of every edge in M . For the remaining edges, consider a clause C . Since X is a feasible solution to (φ, k) , there exists a literal ℓ in C that is either evaluated to true by ψ , or its variable is deleted

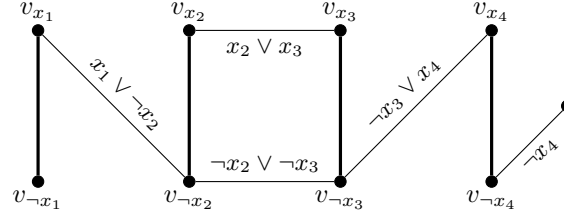


Fig. 3.1: The graph G for the formula $\varphi = (x_1 \vee \neg x_2) \wedge (x_2 \vee x_3) \wedge (\neg x_2 \vee \neg x_3) \wedge (\neg x_3 \vee x_4) \wedge (\neg x_4)$. The thick edges are the edges of the matching M

(belongs to X). In both cases, $v_\ell \in Y$ and the edge corresponding to the clause C is covered by Y .

In the other direction, let Y be a vertex cover of G , and assume $|Y| \leq |M| + k$. Without loss of generality, we may assume that $Y \subseteq V(M)$: if, for some unary clause $C = \ell$, the corresponding degree 1 neighbor of v_ℓ belongs to Y , we replace it with the vertex v_ℓ . Let X be the set of these variables x for which both v_x and $v_{\neg x}$ belong to Y . Since Y needs to contain at least one endpoint of every edge of M , we have $|X| \leq k$. Let us define an assignment ψ on the variables outside X as follows: $\psi(x) = \top$ if $v_x \in Y$, and $\psi(x) = \perp$ if $v_{\neg x} \in Y$. We claim that ψ is a satisfying assignment of $\varphi - X$; note that such a statement implies that X is a feasible solution to the VARIABLE DELETION ALMOST 2-SAT instance (φ, k) , concluding the proof of the theorem.

To this end, consider a clause C , and assume no variable of C belongs to X . If $C = \ell_1 \vee \ell_2$ is a binary clause, then the edge $v_{\ell_1} v_{\ell_2}$ ensures that either v_{ℓ_1} or v_{ℓ_2} belongs to Y , and the corresponding literal is evaluated to true in the assignment ψ . If $C = \ell$ is a unary clause, then the corresponding edge incident to v_ℓ , together with the assumption that $Y \subseteq V(M)$, implies that $v_\ell \in Y$ and ℓ is evaluated to true by ψ .

Consequently, (φ, k) is a yes-instance to VARIABLE DELETION ALMOST 2-SAT if and only if $(G, M, |M| + k)$ is a yes-instance to VERTEX COVER ABOVE MATCHING. This concludes the proof of the theorem. $\square$

3.5 CLOSEST STRING

In the last part of this chapter we give yet another example of branching algorithms, this time for a string problem called CLOSEST STRING. Here, we are given k strings $x_1, \dots, x_k$, each string over an alphabet Σ and of length L , and an integer d . The question is whether there exists a string y of length L over Σ such that $d_H(y, x_i) \leq d$ for all $i \in \{1, \dots, k\}$. Here, $d_H(x, y)$ is the *Hamming distance* between strings x and y , that is, the number of

positions where x and y differ. We call any such string y a *center string*. In this section we consider the parameterization by d , the maximum allowed distance between the center string and the input strings.

Let x be a string over alphabet Σ . We denote the letter on the p th position of x as $x[p]$. Thus $x = x[1]x[2] \cdots x[L]$ for a string of length L . We say that string x and y *differ* on the p -th position if $x[p] \neq y[p]$.

Given a set of k strings, each of length L , we can think of these strings as a $k \times L$ character matrix. By *columns* of the set of strings, we mean the columns of this matrix. That is, the j -th column is the sequence of letters $x_1[j], x_2[j], \dots, x_k[j]$. We call a column *bad* if it contains at least two different symbols from alphabet Σ , and *good* otherwise. Clearly, if the j -th column is a good one, then we have an obvious greedy choice for the j -th letter of the solution: $y[j] = x_1[j] = x_2[j] = \dots = x_k[j]$. Thus, we obtain the following reduction rule.

Reduction CS.1. Delete all good columns.

It is straightforward to implement Reduction CS.1 in linear time.

We now observe that we cannot have too many bad columns in a yes-instance.

Lemma 3.13. *For every yes-instance of CLOSEST STRING, the corresponding $k \times L$ matrix contains at most kd bad columns.*

Proof. Fix a center string y . For every bad column j there exists a string $x_{i(j)}$ such that $x_{i(j)}[j] \neq y[j]$. Since every string x_i differs from y on at most d positions, for every i we have $i(j) = i$ for at most d positions j . Consequently, there are at most kd bad columns. $\square$

Reduction CS.2. If there are more than kd bad columns, then conclude that we are dealing with a no-instance.

We now give an intuition behind the algorithm.

Fix a center string y . The idea of our algorithm is to start with one of the given strings, say x_1 , as a “candidate string”, denoted by z . As long as there is a string x_i , $i \in \{1, \dots, k\}$, such that $d_H(x_i, z) \geq d + 1$, then for at least one of the positions p where z and x_i differ, we have that $y[p] = x_i[p]$. Thus we can try recursively $d + 1$ ways to move the candidate z string “closer” to y ; moving closer here means that we select a position p on which the candidate string z and x_i differ and set $z[p] := x_i[p]$. As at every step we move closer to y , and at the beginning $z = x_1$ and $d_H(x_1, y) \leq d$, we obtain a bounded number of possibilities.

Let us move to formal details.

Theorem 3.14. CLOSEST STRING can be solved in time $\mathcal{O}(kL + kd(d+1)^d)$.

Proof. First, we apply Reductions CS.1 and CS.2. Unless we have resolved the instance already, we are left with k strings $x_1, x_2, \dots, x_k$, each of length $L \leq kd$. The time spent so far is $\mathcal{O}(kL)$, that is, linear in the input size.

Recursively, we solve the following augmented problem: given a candidate string z and an integer $\ell \leq d$, determine whether there exists a center string y for the strings $x_1, x_2, \dots, x_k$ with the additional property $d_H(y, z) \leq \ell$. Observe that, to solve CLOSEST STRING on our instance, it suffices to solve our augmented problem for $z = x_1$ and $\ell = d$.

Given z and ℓ , we first perform the following checks. If z is a center string itself, then we return z . Otherwise, if $\ell = 0$, then we return that there is no such center string y . In the remaining case, $\ell > 0$ and there exists a string x_i with $d_H(x_i, z) > d$. Let $\mathcal{P}$ be a set of arbitrary $d+1$ positions on which x_i and z differ. Observe that for every center string y we have $y[p] = x_i[p]$ for at least one position $p \in \mathcal{P}$. Hence, we branch into $|\mathcal{P}| = d+1$ subcases: for every $p \in \mathcal{P}$, we define z_p to be equal z except for position p where $z_p[p] = x_i[p]$, and we recursively solve our augmented problem for the pair $(z_p, \ell - 1)$. To show correctness of this branching, observe that if there exists a center string y with $d_H(z, y) \leq \ell$, then for a position $p \in \mathcal{P}$ satisfying $x_i[p] = y[p]$ we have $d_H(z_p, y) \leq d_H(z, y) - 1 \leq \ell - 1$.

Concerning the running time of the algorithm, note that we build a search tree of depth at most d , and every node of the search tree has at most $d+1$ children. Thus, the size of the search tree does not exceed $\mathcal{O}((d+1)^d)$. With small technical work which we omit here, every step of the algorithm can be implemented in linear time. This completes the proof. $\square$

Exercises

3.1 ($\mathcal{E}$). Prove Proposition 3.1.

3.2 ($\mathcal{E}$). Show that CLIQUE and INDEPENDENT SET, parameterized by the solution size k , are FPT on r -regular graphs for every fixed integer r . Also show that these problems are FPT with combined parameters $k + r$.

3.3. Show that any graph has at most 2^k inclusion-wise minimal vertex covers of size at most k . Furthermore, show that given G and k , we can enumerate all inclusion-wise minimal vertex covers of G of size at most k in time $2^k n^{\mathcal{O}(1)}$.

3.4 ($\mathcal{E}$). In the CLUSTER VERTEX DELETION problem, we are given a graph G and an integer k , and the task is to delete at most k vertices from G to obtain a cluster graph (a disjoint union of cliques). Obtain a $3^k n^{\mathcal{O}(1)}$ -time algorithm for CLUSTER VERTEX DELETION.

3.5 ($\mathcal{E}$). In the CLUSTER EDITING problem, we are given a graph G and an integer k , and the objective is to check whether we can turn G into a cluster graph (a disjoint union of cliques) by making at most k edge editions, where each edition is adding or deleting one edge. Obtain a $3^k n^{\mathcal{O}(1)}$ -time algorithm for CLUSTER EDITING.

3.6 ($\mathcal{B}$). An undirected graph G is called *perfect* if for every induced subgraph H of G , the size of the largest clique in H is the same as the chromatic number of H . In this exercise we consider the ODD CYCLE TRANSVERSAL problem, restricted to perfect graphs.

Recall that Exercise 2.34 asked for a kernel with $\mathcal{O}(k)$ vertices for this problem. In this exercise, we ask for a $3^k n^{\mathcal{O}(1)}$ -time branching algorithm.

3.7. Let $\mathcal{F}$ be a set of graphs. We say that a graph G is $\mathcal{F}$ -free if G does not contain any induced subgraph isomorphic to a graph in $\mathcal{F}$; in this context the elements of $\mathcal{F}$ are sometimes called *forbidden induced subgraphs*. For a fixed set $\mathcal{F}$, consider a problem where, given a graph G and an integer k , we ask to turn G into a $\mathcal{F}$ -free graph by:

- (vertex deletion) deleting at most k vertices;
- (edge deletion) deleting at most k edges;
- (completion) adding at most k edges;
- (edition) performing at most k editions, where every edition is adding or deleting one edge.

Prove that, if $\mathcal{F}$ is finite, then for each of the four aforementioned problems there exists a $2^{\mathcal{O}(k)} n^{\mathcal{O}(1)}$ -time FPT algorithm. (Note that the constants hidden in the $\mathcal{O}()$ -notation may depend on the set $\mathcal{F}$.)

3.8. In the VERTEX COVER/OCT problem, we are given an undirected graph G , an integer ℓ , and an odd cycle transversal Z of size at most k , and the objective is to test whether G has a vertex cover of size at most ℓ . Show that VERTEX COVER/OCT admits an algorithm with running time $2^k n^{\mathcal{O}(1)}$.

3.9. In this exercise we consider FPT algorithms for FEEDBACK ARC SET IN TOURNAMENTS and FEEDBACK VERTEX SET IN TOURNAMENTS. Recall that a *tournament* is a directed graph, where every pair of vertices is connected by exactly one directed edge (in one of the directions).

1. Let G be a digraph that can be made into a tournament by adding at most $k \geq 2$ directed edges. Show that if G has a cycle then it has a directed cycle of length at most $3\sqrt{k}$.
2. Show that FEEDBACK ARC SET IN TOURNAMENTS admits a branching algorithm with running time $(3\sqrt{k})^k n^{\mathcal{O}(1)}$.
3. Show that FEEDBACK VERTEX SET IN TOURNAMENTS admits a branching algorithm with running time $3^k n^{\mathcal{O}(1)}$.
4. Observe that, in the FEEDBACK ARC SET IN TOURNAMENTS problem, we can equivalently think of *reversing* an edge instead of deleting it. Use this observation to show a branching algorithm for FEEDBACK ARC SET IN TOURNAMENTS with running time $3^k n^{\mathcal{O}(1)}$.

3.10 ($\mathcal{B}$). A *bipartite tournament* is an orientation of a complete bipartite graph, meaning its vertex set is a union of two disjoint sets V_1 and V_2 and there is exactly one arc between every pair of vertices u and v such that $u \in V_1$ and $v \in V_2$.

1. Show that a bipartite tournament has a directed cycle if and only if it has a directed cycle on four vertices.
2. Show that DIRECTED FEEDBACK VERTEX SET and DIRECTED FEEDBACK ARC SET admit algorithms with running time $4^k n^{\mathcal{O}(1)}$ on bipartite tournaments.

3.11 ($\mathcal{B}$). A graph is *chordal* if it does not contain a cycle on at least four vertices as an induced subgraph. A *triangulation* of a graph G is a set of edges whose addition turns G into a chordal graph. In the CHORDAL COMPLETION problem, given a graph G and an integer k , we ask whether G admits a triangulation of size at most k .

1. Show that CHORDAL COMPLETION admits an algorithm with running time $k^{\mathcal{O}(k)}n^{\mathcal{O}(1)}$.
2. Show that there is a one-to-one correspondence between inclusion-wise minimal triangulations of a cycle with ℓ vertices and binary trees with $\ell - 2$ internal nodes. Using this correspondence show that a cycle on ℓ vertices has at most $4^{\ell-2}$ minimal triangulations.
3. Use the previous point to obtain an algorithm with running time $2^{\mathcal{O}(k)}n^{\mathcal{O}(1)}$ for CHORDAL COMPLETION.

You may use the fact that, given a graph G , one can in polynomial time check if G is a chordal graph and, if this is not the case, find in G an induced cycle on at least four vertices.

3.12. In the MIN-ONES- r -SAT problem, we are given an r -CNF formula ϕ and an integer k , and the objective is to decide whether there exists a satisfying assignment for ϕ with at most k variables set to true. Show that MIN-ONES- r -SAT admits an algorithm with running time $f(r, k)n^{\mathcal{O}(1)}$ for some computable function f .

3.13. In the MIN-2-SAT problem, we are given a 2-CNF formula ϕ and an integer k , and the objective is to decide whether there exists an assignment for ϕ that satisfies *at most* k clauses. Show that MIN-2-SAT can be solved in time $2^k n^{\mathcal{O}(1)}$.

3.14. In the MINIMUM MAXIMAL MATCHING problem, we are given a graph G and an integer k , and the task is to check if G admits an (inclusion-wise) maximal matching with at most k edges.

1. Show that if G has a maximal matching of size at most k , then $V(M)$ is a vertex cover of size at most $2k$.
2. Let M be a maximal matching in G and let $X \subseteq V(M)$ be a minimal vertex cover in G . Furthermore, let M_1 be a maximum matching of $G[X]$ and M_2 be a maximum matching of $G[V(G) \setminus V(M_1)]$. Show that $M_1 \cup M_2$ is a maximal matching in G of size at most $|M|$.
3. Obtain a $4^k n^{\mathcal{O}(1)}$ -time algorithm for MINIMUM MAXIMAL MATCHING.

3.15 () In the MAX LEAF SPANNING TREE problem, we are given a connected graph G and an integer k , and the objective is to test whether there exists a spanning tree of G with at least k leaves. Obtain an algorithm with running time $4^k n^{\mathcal{O}(1)}$ for MAX LEAF SPANNING TREE.

3.16 () An *out-tree* is an oriented tree with only one vertex of indegree zero called the *root*. In the DIRECTED MAX LEAF problem, we are given a directed graph G and an integer k , and the objective is to test whether there exists an out-tree in G with at least k leaves (vertices of outdegree zero). Show that DIRECTED MAX LEAF admits an algorithm with running time $4^k n^{\mathcal{O}(1)}$.

3.17. Describe an algorithm running in time $\mathcal{O}(1.381^n)$ which finds the *number* of independent sets (or, equivalently, vertex covers) in a given n -vertex graph.

3.18. Show that if a graph on n vertices has minimum degree at least 3, then it contains a cycle of length at most $2\lceil \log n \rceil$. Use this to design a $(\log n)^{\mathcal{O}(k)}n^{\mathcal{O}(1)}$ -time algorithm for FEEDBACK VERTEX SET on undirected graphs. Is this an FPT algorithm for FEEDBACK VERTEX SET?

3.19. Prove that the statement of Lemma 3.6 is true for any optimum solution $\mathbf{x}$, not necessarily a half-integral one. (For a not half-integral solution $\mathbf{x}$, we define $V_1^{\mathbf{x}} = \{v \in V(G) : x_v > \frac{1}{2}\}$ and $V_0^{\mathbf{x}} = \{v \in V(G) : x_v < \frac{1}{2}\}$.)

3.20. A graph G is a *split graph* if $V(G)$ can be partitioned into sets C and I , such that C is a clique and I is an independent set in G . In the SPLIT VERTEX DELETION problem, given a graph G and an integer k , the task is to check if one can delete at most k vertices from G to obtain a split graph.

Give a polynomial-time algorithm that, given a SPLIT VERTEX DELETION instance (G, k) , produces an equivalent VERTEX COVER ABOVE MATCHING instance $(G', M, |M| + k)$. Use this to design a $4^k n^{O(1)}$ -time algorithm for SPLIT VERTEX DELETION.

3.21. Show how to, given an ALMOST 2-SAT instance (φ, k) , compute in polynomial time an equivalent instance (φ', k) of VARIABLE DELETION ALMOST 2-SAT.

3.22. Show the reverse of the previous exercise. That is, show how to, given an instance (φ, k) of VARIABLE DELETION ALMOST 2-SAT, compute in polynomial time an equivalent instance (φ', k) of ALMOST 2-SAT.

3.23 (🔗). For an independent set I in a graph G , the *surplus* of I is defined as $|N(I)| - |I|$.

1. Show that a graph reduced with respect to Reduction VC.5 (i.e., this reduction cannot be further applied) does not admit independent sets with nonpositive surplus.
2. Show how to detect independent sets of surplus 1 in such a graph using the LP relaxation of VERTEX COVER.
3. Design a reduction rule for VERTEX COVER that handles a (given) independent set of surplus 1. How does the measure $\mu(G, k) = k - \text{vc}^*(G)$ behave in your reduction rule?
4. Show that, if a graph does not admit an independent set of surplus at most 1, then the branching of Theorem 3.8 has branching vector $(\frac{1}{2}, 1)$, and the algorithm runs in time $2.6181^{k - \text{vc}^*(G)} n^{O(1)}$.

3.24 (🔗). Consider the CLOSEST STRING problem, where in the input there are two strings x_i and x_j with $d_H(x_i, x_j) = 2d$. Show that in this case the CLOSEST STRING problem can be solved in time $O(kL + kd4^d)$.

3.25 (🔗). Consider a generalization of CLOSEST STRING where, apart from the strings $x_1, x_2, \dots, x_k$, each over alphabet Σ and of length L , we are given integers $d_1, d_2, \dots, d_k$, and we ask for a center string y of length L such that $d_H(y, x_i) \leq d_i$ for every $1 \leq i \leq k$. Consider the following recursive algorithm for this generalization.

1. First, try if x_1 is a center string. If this is the case, then return x_1 and finish.
2. Otherwise, if $d_1 = 0$, then return that there is no solution.
3. In the remaining case, take any $j \geq 2$ such that $d(x_1, x_j) > d_j$. Let $\mathcal{P}$ be the set of positions on which x_1 and x_j differ. If $|\mathcal{P}| > d_1 + d_j$, return that there is no solution.
4. Guess the values $y[p]$ for every $p \in \mathcal{P}$, where y is the center string we are looking for, in such a way that y differs from x_i on at most d_i positions from $\mathcal{P}$ for every $1 \leq i \leq k$.
5. For every guess $(y[p])_{p \in \mathcal{P}}$, define $d'_i = d_i - |\{p \in \mathcal{P} : y[p] \neq x_i[p]\}|$ and let x'_i be the string x_i with letters on positions of $\mathcal{P}$ deleted.
6. Observe that $x'_1 = x'_j$. Discard the string x'_j , and set $d'_1 := \min(d'_1, d'_j)$.
7. Recurse on the strings x'_i and integers d'_i , for every guess $(y[p])_{p \in \mathcal{P}}$.

Denote $d = \max_{1 \leq i \leq k} d_i$.

1. Show that in every recursive step it holds that $d'_1 < d_1/2$.
2. Use this fact to show that this algorithm solves the generalization of CLOSEST STRING in time $2^{O(d)} |\Sigma|^d (kL)^{O(1)}$.

Hints

3.3 Consider the following branching algorithm: given an instance (G, k) , pick an arbitrary edge $uv \in E(G)$ and branch on two instances $(G - u, k - 1)$ and $(G - v, k - 1)$. Stop when G becomes edgeless or $k < 0$. Show that in this manner you generate at most 2^k leaves of the search tree, and every minimal vertex cover of G of size at most k appears in some leaf.

3.4 Observe that a graph is a cluster graph if and only if it does not contain a path on three vertices (P_3) as an induced subgraph. As long as there exists a P_3 as an induced subgraph, branch by choosing one of its vertices to delete.

3.5 Proceed as in Exercise 3.4, and observe that you can break a P_3 in three different ways.

3.6 Observe that the class of perfect graphs is closed under taking induced subgraphs, and a perfect graph is bipartite if and only if it does not contain a triangle.

3.7 Observe that, if $\mathcal{F}$ is finite and fixed, then

1. we can in polynomial time verify if a graph G is $\mathcal{F}$ -free and, if not, find an induced subgraph of G that is isomorphic to a member of $\mathcal{F}$;
2. there are only $\mathcal{O}(1)$ ways to break such a forbidden induced subgraph in all four considered problem variants.

3.8 For every $v \in Z$, branch on whether to include v or $N(v)$ into the desired vertex cover. Observe that the remaining graph is bipartite.

3.9 For the first point, observe that a shortest cycle in a directed graph cannot have any chord. Moreover, if ℓ is the length of some chordless cycle in D , then there are at least $\ell(\ell - 3)/2$ pairs of nonadjacent vertices in D . To solve the second point, show how to find such a chordless cycle, and branch choosing which edge to delete from it.

For the third and fourth point, observe that a tournament is acyclic if and only if it does not contain a directed triangle.

3.10 Use again the observation that in the DIRECTED FEEDBACK ARC SET problem you can alternatively reverse edges instead of deleting them. Thus, the graph remains a bipartite tournament in the course of the algorithm, and every directed cycle on four vertices can be broken in exactly four ways in both considered problems.

3.11 First, design an algorithm that either verifies that G is a chordal graph, or finds an induced cycle on at least four vertices in G . Second, observe that such a cycle on more than $k + 3$ vertices is a certificate that (G, k) is a no-instance. For the first point, it suffices to branch choosing one edge to add to the cycle found. For the last point, the branching should consider all minimal triangulations of the cycle.

3.12 Start with an assignment that sets all variables to false. As long as there exists an unsatisfied clause C , branch on C ; in each subcases choosing one positive literal of C to be satisfied. In this way, you obtain an $r^k n^{\mathcal{O}(1)}$ -time algorithm.

3.13 First, observe that if some variable appears only positively (or negatively), then we can assign its value in such a way that it does not satisfy any clause. Second, note that if the aforementioned preprocessing rule is not applicable, then branching on any variable makes at least one clause satisfied, decreasing the budget k by at least 1.

3.14 The first point is straightforward. The fact that $M_1 \cup M_2$ is a maximal matching follows directly from the fact that M_2 is a maximum matching in $G - V(M_1)$. For the cardinality bound, let $M_X \subseteq M$ be the set of edges of M that have both endpoints in

X . Observe that $|M| = |M_X| + |M \setminus M_X| = |M_X| + (|X| - 2|M_X|) = |X| - |M_X|$, while $|M_1| \geq |M_X|$ and $|M_2| \leq |X| - |V(M_1)| = |X| - 2|M_1|$. For the last point, use Exercise 3.3 to enumerate all minimal vertex covers of G of size at most $2k$.

3.15 Start with the following observation: if G is connected and has a subtree T , rooted at some vertex r , with at least k leaves, then G has also a spanning tree with at least k leaves. In our algorithm, we start by guessing a non-leaf vertex r of the tree in question, and consider all further trees as rooted in r .

At every node of the search tree we have a subtree T of G rooted at node r . Let $L(T)$ denote the set of leaves of T . Our objective in this branch is to find a subtree with k leaves (if it exists) that is a supergraph of T .

First, observe that if there exists a leaf $v \in L(T)$ with at least two neighbors in $V(G) \setminus V(T)$, then we can branch on v . In one subcase, we consider v being an internal vertex of the tree in question, adding all edges vu for $u \in N(v) \setminus V(T)$ to the tree T . In the second subcase, we consider v being a leaf vertex of the tree in question, deleting all edges vu for $u \in N(v) \setminus V(T)$ from the graph G . Show that this branching is correct. Observe that in every branching step we either fix one vertex to be a leaf, or increase the number of leaves of T . Hence, this branching leads to at most 2^{2k} leaves of the search tree.

We are left with the case when all vertices $v \in L(T)$ have at most one neighbor outside $V(T)$. Prove that in this case a similar branching is still valid: for a vertex $v \in L(T)$, either proclaim v a leaf, or fix as internal vertices both v and all its descendants, up to the closest vertex of degree at least three, inclusive.

In the leaves of the search tree we have trees T with $|L(T)| \geq k$ (where we report that (G, k) is a yes-instance), or trees T where all the leaves have been fixed as leaves, but still $|L(T)| < k$ (where we report no solution in this branch).

3.16 Adapt the solution of Exercise 3.15 to the directed case.

3.17 Note that there is a bijection between vertex covers and maximal independent sets, since the complement of a vertex cover is an independent set. Show a *counting* counterpart of Proposition 3.1 and design a branching algorithm where the number of leaves in the search tree is bounded by the solution of the recurrence $T(n) = T(n-1) + T(n-4)$. Solve the recurrence by applying the techniques described in Section 3.2 (use Table 3.1).

3.18 To prove the desired upper bound on the length of the shortest cycle, consider a breadth-first search tree of the input graph.

To show that the obtained algorithm is an FPT one, consider two possibilities. If $n \leq k^k$, then $\log n \leq k \log k$ and $(\log n)^k \leq 2^{\mathcal{O}(k \log k)}$. Otherwise, $k = \mathcal{O}(\log n / \log \log n)$ and $(\log n)^k = 2^{\mathcal{O}(\log n)} = n^{\mathcal{O}(1)}$. Alternatively, you may use the Cauchy-Schwarz inequality to observe that

$$(\log n)^k = 2^{k \log \log n} \leq 2^{\frac{k^2 + (\log \log n)^2}{2}} = 2^{k^2/2} \cdot 2^{(\log \log n)^2/2} = 2^{k^2/2} \cdot n^{o(1)}.$$

3.19 Apply Exercise 2.25 to $\mathbf{x}$, obtaining a half-integral solution $\mathbf{y}$. Observe that Reduction VC.4 behaves the same way when fed with the solution $\mathbf{x}$ and with the solution $\mathbf{y}$.

3.20 In short, modify the reduction of Lemma 3.10.

For a given a graph G , we define a new graph $\tilde{G}$ as follows. Let $V_i = \{u_i : u \in V(G)\}$, $i \in \{1, 2\}$. The vertex set $V(\tilde{G})$ consists of two copies of $V(G)$, i.e., $V(\tilde{G}) = V_1 \cup V_2$ and

$$E(\tilde{G}) = \{u_1 u_2 : u \in V(G)\} \cup \{u_1 v_1 : uv \in E(G)\} \cup \{u_2 v_2 : uv \notin E(G)\}.$$

In other words, $\tilde{G}$ is obtained by taking a disjoint copy of G and a complement of G , and adding a perfect matching such that the endpoints of every matching edge are copies of the same vertex.

3.21 The main idea is to make, for every variable x , a different copy for every appearance of x in the formula φ . To create the formula φ' , we first copy all the clauses of φ , replacing every variable x with one of its copies so that no copy is used twice, and then we add clauses enforcing equality between every pair of copies of the same variable. Show that a deletion of a clause C in the original instance corresponds to the deletion of one of the copies of a variable appearing in C in the new instance.

3.22 The main idea is to replace every variable x with its two copies, $x^\top$ and $x^\perp$: $x^\top$ has the meaning “Is x true?”, whereas $x^\perp$ has the meaning “Is x false?”. We connect them by a clause $\neg x^\top \vee \neg x^\perp$ to enforce that only one value of x is chosen. Moreover, we replace every literal x with $x^\top$, and every literal $\neg x$ with $x^\perp$. Argue that in the new instance (φ', k) of ALMOST 2-SAT there exists a minimum solution that deletes only clauses of the form $\neg x^\top \vee \neg x^\perp$. Observe that deleting such a clause corresponds to setting x to both true and false at once in the input formula φ , satisfying all the clauses containing x .

3.23 Let I be an independent set in G of surplus a . Define the following vector $\mathbf{x} = (x_v)_{v \in V(G)}$: $x_v = 0$ for $v \in I$, $x_v = 1$ for $v \in N(I)$ and $x_v = \frac{1}{2}$ for $v \notin N[I]$. Observe that $\mathbf{x}$ is a feasible solution to LPVC(G) of cost $\frac{|V(G)|}{2} + \frac{a}{2}$. This proves the first point.

Moreover, it also proves the following: if we pick any vertex v and solve LPVC(G) with an additional constraint $x_v = 0$ (in other words, solve LPVC($G - N[v]$) and extend it with $x_v = 0$ and $x_u = 1$ for every $u \in N(v)$), obtaining a half-integral solution $\mathbf{x}$, then the set $V_0^\mathbf{x}$ is an independent set of minimum surplus among all independent sets containing v . In particular, this gives us a way to detect independent sets with surplus 1, as well as shows that, if the minimum surplus of an independent set containing v is at least 2, then in a branch where we exclude v from a vertex cover, the measure $\mu(G, k)$ drops by at least 1.

It remains to argue about the third point, tackling with a reduction rule. To this end, prove the following: if Reduction VC.5 is not applicable, and I is of surplus 1, then there exists a minimum vertex cover of G that contains the whole I or the whole $N(I)$. Recall $|I| + 1 = |N(I)|$. If $G[N(I)]$ contains an edge, then we can greedily take $N(I)$ into the vertex cover. Otherwise, we can replace $N[I]$ with a single vertex incident to $N(N[I])$ and decrease k by $|I|$. A direct check shows that the measure $\mu(G, k)$ does not increase in these steps.

3.24 Observe that, in this case, for every center string y , $y[p]$ equals $x_i[p]$ whenever $x_i[p] = x_j[p]$, whereas for every position p with $x_i[p] \neq x_j[p]$ we have $y[p] = x_i[p]$ or $y[p] = x_j[p]$. Thus, there are two options for each of $2d$ positions where x_i and x_j differ.

3.25 For the first point, consider two cases depending on for how many positions $p \in \mathcal{P}$ the character $y[p]$ has been guessed so that $y[p] = x_j[p]$. If there are more than $d_1/2$ such positions, then x_1 differs on all of them from y , and we have $d'_1 < d_1/2$. Show that in the second case, when there are at most $d_1/2$ such positions, it holds that $d'_j < d_1/2$. Use here the fact that $|\mathcal{P}| > d_j$.

For the second point, think of choosing $(y[p])_{p \in \mathcal{P}}$ as a three stage process. First, we choose an integer $0 \leq \ell \leq d_1$: y will differ from x_1 on exactly ℓ positions of $\mathcal{P}$. Second, we choose the set $\mathcal{Q}$ of these ℓ positions; there are $\binom{|\mathcal{P}|}{\ell} \leq \binom{d+d_1}{\ell}$ possibilities. Third, we choose $y[p]$ for every $p \in \mathcal{Q}$; there are $(|\Sigma| - 1)^\ell$ possibilities.

Using d_1 as a measure for the complexity of an instance, and using the first point, we obtain the following recurrence for time complexity.

$$\begin{aligned} T(d_1) &\leq \sum_{\ell=0}^{d_1} \binom{d+d_1}{\ell} (|\Sigma| - 1)^\ell \cdot T(\min(d_1 - \ell, d_1/2)) \\ &= \sum_{\ell=0}^{\lfloor d_1/2 \rfloor} \binom{d+d_1}{\ell} (|\Sigma| - 1)^\ell \cdot T(d_1/2) + \sum_{\ell=\lfloor d_1/2 \rfloor + 1}^{d_1} \binom{d+d_1}{\ell} (|\Sigma| - 1)^\ell \cdot T(d_1 - \ell). \end{aligned}$$

It remains to use the aforementioned recurrence to prove by induction that

$$T(d_1) \leq \binom{d + d_1}{d_1} 2^{6d_1} (|\Sigma| - 1)^{d_1}.$$

(Constant 6 in the exponent is not optimal; it has been chosen so that the proof goes smoothly.)

Bibliographic notes

The history of branching algorithms can be traced back to the work of Davis and Putnam [126] (see also [125]) on the design and analysis of algorithms to solve some satisfiability problems. Solving linear recurrences or linearly recurrent sequences is the subject of many textbooks in discrete mathematics including the books of Rosen [406] and Graham, Knuth, and Patashnik [232], see also [300, p. 88]. Branching algorithms and techniques for their analysis are discussed in detail in the book of Fomin and Kratsch [197].

A branching algorithm with running time $\mathcal{O}(2^k(n+m))$ for VERTEX COVER appears in the book of Mehlhorn [363]. After a long sequence of improvements, the current champion is the algorithm of Chen, Kanj, and Xia, running in time $\mathcal{O}(1.2738^k + kn)$ [82]. The branching algorithm for FEEDBACK VERTEX SET is from [409]. Our algorithm for CLOSEST STRING follows Gramm, Niedermeier, and Rossmanith [235].

Parameterizations above and below guarantees were introduced by Mahajan and Raman [342]. Other interesting examples of algorithms for above and below guarantee parameterizations include [11, 104, 105, 107, 108, 247, 250, 290, 343].

Our algorithm for VERTEX COVER ABOVE LP, as well as Exercise 3.23, follows the outline from [373]. More involved reduction rules and a detailed analysis lead to an improved running time bound of $2.32^k n^{\mathcal{O}(1)}$ for VERTEX COVER ABOVE LP [328, 373]. The “LP-guided branching” paradigm was introduced by Cygan, Pilipczuk, Pilipczuk, and Woitaszczyk [122] to solve VERTEX MULTIWAY CUT, and extended to a bigger set of problems by Wahlström [430]. The above algorithm is not only an example of the use of linear programming in parameterized algorithms, but also an example of the technique called “measure and conquer.” In a more sophisticated use of this technique, one comes up with a measure which is a function of the input and the parameter, and whose objective is to measure the progress in each branching step. We refer to [194] and [197] for an introduction to measure and conquer, as well as its applications to the VERTEX COVER and DOMINATING SET problems.

The observation that graph modification problems for hereditary graph classes with a finite set of forbidden induced subgraphs admit simple FPT algorithms (encapsulated as Exercise 3.7) is due to Cai [63].

Exercise 3.11 is from [281]; we note that there is also a subexponential parameterized algorithm for CHORDAL COMPLETION [211]. Exercise 3.25 is from [341]. Exercises 3.15 and 3.16 are from [298].